



OnGuard®

OpenAccess User Guide



Lenel® OnGuard® 8.0 OpenAccess User Guide

This guide is item number DOC-1057-EN-US, revision 11.032, October 2020.

© 2020 Carrier. All Rights Reserved. All trademarks are the property of their respective owners.

LenelS2 is a part of Carrier.

Information in this document is subject to change without notice. No part of this document may be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without the prior express written permission of UTC Fire & Security Americas Corporation, Inc., which such permission may have been granted in a separate agreement (i.e., end user license agreement or software license agreement for the particular application).

Non-English versions of LenelS2 documents are offered as a service to our global audiences. We have attempted to provide an accurate translation of the text, but the official text is the English text, and any differences in the translation are not binding and have no legal effect.

The software described in this document is furnished under a license agreement and may only be used in accordance with the terms of that agreement.

SAP® Crystal Reports® is the registered trademark of SAP SE or its affiliates in Germany and in several other countries.

Integral and FlashPoint are trademarks of Integral Technologies, Inc.

Portions of this product were created using LEADTOOLS ©1991-2011, LEAD Technologies, Inc. ALL RIGHTS RESERVED.

Active Directory, Microsoft, SQL Server, Windows, and Windows Server are either registered trademarks or trademarks of Microsoft Corporation in the United States and/or other countries.

Oracle is a registered trademark of Oracle International Corporation.

Amazon Web Services and the "Powered by AWS" logo are trademarks of Amazon.com, Inc. or its affiliates in the United States and/or other countries.

Other product names mentioned may be trademarks or registered trademarks of their respective companies and are hereby acknowledged.

Product Disclaimers and Warnings

THESE PRODUCTS ARE INTENDED FOR SALE TO, AND INSTALLATION BY, AN EXPERIENCED SECURITY PROFESSIONAL. LENELS2 CANNOT PROVIDE ANY ASSURANCE THAT ANY PERSON OR ENTITY BUYING ITS PRODUCTS, INCLUDING ANY "AUTHORIZED DEALER", IS PROPERLY TRAINED OR EXPERIENCED TO CORRECTLY INSTALL SECURITY RELATED PRODUCTS.

LENELS2 DOES NOT REPRESENT THAT SOFTWARE, HARDWARE OR RELATED SERVICES MAY NOT BE HACKED, COMPROMISED AND/OR CIRCUMVENTED. LENELS2 DOES NOT WARRANT THAT SOFTWARE, HARDWARE OR RELATED SERVICES WILL WORK PROPERLY IN ALL ENVIRONMENTS AND APPLICATIONS AND DOES NOT WARRANT ANY SOFTWARE, HARDWARE OR RELATED SERVICES AGAINST HARMFUL ELECTROMAGNETIC INTERFERENCE INDUCTION OR RADIATION (EMI, RFI, ETC.) EMITTED FROM EXTERNAL

SOURCES. THE ABILITY OF SOFTWARE, HARDWARE AND RELATED SERVICES TO WORK PROPERLY DEPENDS ON A NUMBER OF PRODUCTS AND SERVICES MADE AVAILABLE BY THIRD PARTIES OVER WHICH LENELS2 HAS NO CONTROL INCLUDING, BUT NOT LIMITED TO, INTERNET, CELLULAR AND LANDLINE CONNECTIVITY; MOBILE DEVICE AND RELATED OPERATING SYSTEM COMPATABILITY; OR PROPER INSTALLATION, CONFIGURATION AND MAINTENANCE OF AUTHORIZED HARDWARE AND OTHER SOFTWARE.

LENELS2 MAY MAKE CERTAIN BIOMETRIC CAPABILITIES (E.G., FINGERPRINT, VOICE PRINT, FACIAL RECOGNITION, ETC.), DATA RECORDING CAPABILITIES (E.G., VOICE RECORDING), AND/OR DATA/INFORMATION RECOGNITION AND TRANSLATION CAPABILITIES AVAILABLE IN PRODUCTS LENELS2 MANUFACTURES AND/OR RESELLS. LENELS2 DOES NOT CONTROL THE CONDITIONS AND METHODS OF USE OF PRODUCTS IT MANUFACTURES AND/OR RESELLS. THE END-USER AND/OR INSTALLER AND/OR RESELLER/DISTRIBUTOR ACT AS CONTROLLER OF THE DATA RESULTING FROM USE OF THESE PRODUCTS, INCLUDING ANY RESULTING PERSONALLY IDENTIFIABLE INFORMATION OR PRIVATE DATA, AND ARE SOLELY RESPONSIBLE TO ENSURE THAT ANY PARTICULAR INSTALLATION AND USE OF PRODUCTS COMPLY WITH ALL APPLICABLE PRIVACY AND OTHER LAWS, INCLUDING ANY REQUIREMENT TO OBTAIN CONSENT. THE CAPABILITY OR USE OF ANY PRODUCTS MANUFACTURED OR SOLD BY LENELS2 TO RECORD CONSENT SHALL NOT BE SUBSTITUTED FOR THE CONTROLLER'S OBLIGATION TO INDEPENDENTLY DETERMINE WHETHER CONSENT IS REQUIRED, NOR SHALL SUCH CAPABILITY OR USE SHIFT ANY OBLIGATION TO OBTAIN ANY REQUIRED CONSENT TO LENELS2.

For more information on warranty disclaimers and product safety information, please check <https://firesecurityproducts.com/en/policy/product-warning> or scan the following code:



Table of Contents

CHAPTER 1	Introduction	15
	Expectations and Behaviors of OpenAccess	16
	<i>Confirming the Installed Version of OnGuard</i>	16
	<i>Stopping and Restarting the Services</i>	16
	<i>Authorization</i>	17
	<i>User-Defined Fields</i>	17
	<i>OpenAccess and Brute Force Attack Protection</i>	17
	<i>Using OpenAccess to Issue Mobile Badges</i>	17
	<i>Authenticated Token and Inactivity Timeouts</i>	17
	OpenAccess Custom Configuration	19
	<i>Authentication Property</i>	19
	<i>Caching Properties</i>	20
	<i>Send Incoming Events Property</i>	21
	<i>Badge Printing Properties</i>	22
	<i>HTTP Request Properties</i>	22
	<i>Queuing Property</i>	24
	<i>Job Runner/Thread Pool Properties</i>	24
	<i>Timeout Property</i>	25
	<i>Event Context Provider Properties</i>	25
	Definitions, Acronyms, Abbreviations	25
	OpenAccess Architecture	26
	References and Applicable Documents	26
 CHAPTER 2	 Getting Started	 29
	License for OpenAccess	29
	<i>Application ID and Getting Started with Development</i>	29
	Starting OpenAccess	30
	Stopping and Restarting the Services	30
	LS OpenAccess Service	31

Authorization	32
Authentication	32
<i>Supported Authentication Modes</i>	32
Deploying the LS Event Context Provider Service	33
Enabling Verbose Logging	33
Starting the OpenAccess Tool	34
Sample Applications	34
<i>Sample Web Applications</i>	34
<i>Sample C# Applications</i>	35
<i>Sample Java Application</i>	37
Swagger Specification and Interactive Documentation	38
Using Response Headers to Develop Secure Web Applications	38

CHAPTER 3 *Using OpenAccess* 39

Searching for Objects	39
Date/Time Format	40
<i>Date/Time Format When Using OpenAccess API Calls</i>	40
<i>Date/Time Format When Using Events</i>	40
Binary Format	40
String Format	41
Features and Limitations	41
<i>Cardholders and Visitors</i>	41
<i>Badges</i>	41
<i>Directory Accounts</i>	41
<i>Visits</i>	41
<i>User-Defined Fields</i>	42
<i>User-Defined List Values</i>	42
<i>SegmentID</i>	42
Receiving Events	43
<i>Durable vs. Transient Event Subscribers</i>	43
<i>Using Event Filters with Subscriptions</i>	43
Cross-Origin Resource Sharing	48
OpenAccess Operations From Behind a Network Proxy	49
Version	49
OpenAccess and Brute Force Attack Protection	49
Preventing Malicious Code in OpenAccess Responses	49

CHAPTER 4 *REST API Reference* 53

Required Parameters for OpenAccess Requests	55
General OpenAccess API Calls	56
<i>get version</i>	56
<i>get keepalive</i>	56
<i>get feature_availability</i>	56
<i>get queue</i>	57
<i>get queue/{id}</i>	57
<i>delete queue/{id}</i>	58

<i>add partner_values</i>	58
<i>modify partner_values</i>	59
<i>get susp_expiration</i>	59
Login and Logout	60
<i>get directories</i>	60
<i>add authentication</i>	61
<i>delete authentication</i>	63
<i>get session</i>	64
<i>get identity_provider_url</i>	64
Receive Events	65
<i>get event_subscriptions</i>	65
<i>get event_subscriptions with id</i>	68
<i>add event_subscriptions</i>	69
<i>modify event_subscriptions with id</i>	71
<i>delete event_subscriptions with id</i>	72
Manage Instances	73
<i>get logged_events</i>	73
<i>get types</i>	76
<i>get type</i>	77
<i>get count</i>	79
<i>get instances</i>	80
<i>get badge print_request</i>	82
<i>add badge print_request</i>	83
<i>delete badge print_request</i>	85
<i>get badge mobile_devices</i>	85
<i>add badge issue_mobile_credential</i>	86
<i>get badge printers</i>	87
<i>add instances</i>	88
<i>modify instances</i>	89
<i>bulk modify instance property</i>	90
<i>delete instances</i>	91
<i>execute_method</i>	92
<i>get cardholders</i>	93
<i>get visitors</i>	99
<i>get video_recorders</i>	104
<i>get video_recorder auth_data</i>	106
<i>get video_recorder credentials</i>	107
<i>get map background</i>	107
<i>put access_level</i>	108
<i>post send_incoming_events</i>	109
Users	110
<i>get logged_in_user</i>	110
<i>get user managed_access_levels</i>	111
<i>add user managed_access_levels</i>	112
<i>delete user managed_access_levels</i>	113
<i>get user</i>	113
<i>modify user</i>	114
<i>put user password</i>	115
<i>get managers_of_access_level</i>	115
<i>get editable_segments</i>	116
<i>get user segments</i>	117
<i>add user segments</i>	118
<i>delete user segments</i>	119

<i>get user preferences</i>	119
<i>put user preferences</i>	120
<i>post user preferences</i>	121
<i>delete user preferences</i>	122
Cardholders	123
<i>get cardholder_from_directory</i>	123
<i>get directory_accounts</i>	123
<i>get directory_accounts_matching_cardholders</i>	124
<i>put update_cardholder_with_directory_account_property</i>	126
Console	127
<i>post console cards</i>	127
<i>delete console cards with id</i>	128
<i>get console layouts</i>	129
<i>put console layouts</i>	129
Settings	130
<i>get authorization warning settings</i>	130
<i>get cardholder settings</i>	132
<i>get enterprise settings</i>	133
<i>get password policy settings</i>	134
<i>put password policy settings</i>	136
<i>get segmentation settings</i>	139
<i>get visit settings</i>	140
<i>put visit settings</i>	140
<i>get video settings</i>	141
Maps	141
<i>get maps</i>	141
<i>get map devices</i>	144
<i>get map icon groups</i>	148
<i>get map icons</i>	152

CHAPTER 5 *Event API Reference* 155

Web Event Bridge Operations	155
<i>CreateSubscription</i>	155
<i>ModifySubscription</i>	157
<i>StopSubscription</i>	159
<i>StartManaging</i>	159
<i>StopManaging</i>	159
<i>ConnectionHeartbeat</i>	160
Web Event Bridge Client Event Handlers	160
<i>OnBusinessEventReceived</i>	160
<i>OnExceptionRaised</i>	161
<i>OnConnectionFromMessageBusLost</i>	161
<i>OnConnectionToMessageBusEstablished</i>	161
<i>OnManagementEvent</i>	161
Hardware Event Reference	162
<i>Access Granted Events</i>	165
<i>Access Denied Events</i>	166
<i>Area Control Events</i>	167
<i>Asset Events</i>	167
<i>Biometric Events</i>	168
<i>Intercom Events</i>	168

Intrusion Events	169
Transmitter Events	169
Video Events	169
Status Events	169
Alarm Acknowledgment Activity Event Reference	173
Software Event Reference	174
Person Directory Account Events	175
Badge Events	175
Cardholder Events	176
Visitor Events	178
Visit Events	179
VisitEvent Events	179

CHAPTER 6 *Data and Association Class Reference*181

Data Classes	181
Lnl_AccessGroup	181
Lnl_AccessLevel	182
Lnl_AccessLevelAssignment	183
Lnl_AccessLevelManaged	184
Lnl_AccessLevelReaderAssignment	185
Lnl_AccessRequest	185
Lnl_AccessLevelRequest	187
Lnl_Account	188
Lnl_AlarmAckHistory	189
Lnl_AlarmDefinition	190
Lnl_AlarmInput	192
Lnl_AlarmOutput	193
Lnl_AlarmPanel	194
Lnl_Area	195
Lnl_AuthenticationMode	196
Lnl_Badge	197
Lnl_BadgeFIPS201	200
Lnl_BadgeLastLocation	201
Lnl_BadgeStatus	202
Lnl_BadgeType	202
Lnl_Camera	204
Lnl_CameraDeviceLink	205
Lnl_CameraGroup	205
Lnl_CameraGroupCameraLink	206
Lnl_Cardholder	206
Lnl_DeviceGroup	208
Lnl_Directory	208
Lnl_Element	209
Lnl_ElevatorTerminal	209
Lnl_EventAlarmDefinitionLink	210
Lnl_EventParameter	211
Lnl_EventSubtypeDefinition	211
Lnl_EventSubtypeParameterLink	212
Lnl_EventType	212
Lnl_GuardTour	213
Lnl_Holiday	213
Lnl_HolidayType	214

<i>Lnl_HolidayTypeLink</i>	214
<i>Lnl_IncomingEvent</i>	215
<i>Lnl_Input</i>	217
<i>Lnl_IntrusionArea</i>	218
<i>Lnl_IntrusionDoor</i>	219
<i>Lnl_IntrusionOutput</i>	221
<i>Lnl_IntrusionZone</i>	221
<i>Lnl_LoggedEvent</i>	222
<i>Lnl_LogicalDevice</i>	225
<i>Lnl_LogicalSource</i>	225
<i>Lnl_LogicalSubDevice</i>	226
<i>Lnl_MonitoringZone</i>	226
<i>Lnl_MonitoringZoneCameraLink</i>	227
<i>Lnl_MonitoringZoneDeviceLink</i>	227
<i>Lnl_MonitoringZoneRecorderLink</i>	228
<i>Lnl_MultimediaObject</i>	229
<i>Lnl_OffBoardRelay</i>	231
<i>Lnl_OnBoardRelay</i>	232
<i>Lnl_Output</i>	233
<i>Lnl_Panel</i>	234
<i>Lnl_Person</i>	236
<i>Lnl_PersonSecondarySegments</i>	237
<i>Lnl_PrecisionAccessGroup</i>	237
<i>Lnl_PrecisionAccessGroupAssignment</i>	238
<i>Lnl_ProhibitedPassword</i>	238
<i>Lnl_PTZPreset</i>	239
<i>Lnl_Reader</i>	239
<i>Lnl_ReaderInput</i>	244
<i>Lnl_ReaderInput1</i>	245
<i>Lnl_ReaderInput2</i>	246
<i>Lnl_ReaderOutput</i>	247
<i>Lnl_ReaderOutput1</i>	248
<i>Lnl_ReaderOutput2</i>	249
<i>Lnl_ReaderRequest</i>	250
<i>Lnl_RequestableReader</i>	251
<i>Lnl_Segment</i>	252
<i>Lnl_SegmentGroup</i>	252
<i>Lnl_SegmentUnit</i>	253
<i>Lnl_Timezone</i>	253
<i>Lnl_TimezoneInterval</i>	253
<i>Lnl_User</i>	254
<i>Lnl_UserAccount</i>	256
<i>Lnl_UserPermissionGroup</i>	257
<i>Lnl_UserFieldPermissionGroup</i>	257
<i>Lnl_UserPermissionDeviceGroupLink</i>	258
<i>Lnl_UserReportPermissionGroup</i>	258
<i>Lnl_UserSecondarySegment</i>	259
<i>Lnl_VideoLayout</i>	259
<i>Lnl_VideoLayoutSource</i>	260
<i>Lnl_VideoTemplate</i>	260
<i>Lnl_Visit</i>	261
<i>Lnl_VisitEmailRecipient</i>	262
<i>Lnl_VisitEvent</i>	263
<i>Lnl_Visitor</i>	264

<i>Lnl_VisitDelegateAssignment</i>	265
<i>Lnl_VisitSelfServiceStation</i>	266
<i>Lnl_VisitSignInLocation</i>	266
<i>Lnl_Workstation</i>	267
<i>Lnl_WorldTimezone</i>	267
<i>User-Defined Value Lists</i>	270
Association Classes	271
<i>Lnl_AccessLevelGroupAssignment</i>	271
<i>Lnl_BadgeOwner</i>	271
<i>Lnl_CardholderAccount</i>	271
<i>Lnl_CardholderBadge</i>	272
<i>Lnl_CardholderMultimediaObject</i>	272
<i>Lnl_DirectoryAccount</i>	272
<i>Lnl_MultimediaObjectOwner</i>	273
<i>Lnl_PersonAccount</i>	273
<i>Lnl_ReaderEntersArea</i>	273
<i>Lnl_ReaderExitsArea</i>	274
<i>Lnl_SegmentGroupMember</i>	274
<i>Lnl_VisitorAccount</i>	274
<i>Lnl_VisitorBadge</i>	275
<i>Lnl_VisitorMultimediaObject</i>	275

CHAPTER 7	<i>Using OpenAccess to Send Alarms to OnGuard</i>	277
------------------	---	-----

CHAPTER 8	<i>Logical Sources Folder</i>	279
------------------	-------------------------------------	-----

Logical Sources Folder	279
Logical Source Downstream Devices	280
User Permissions Required	280
<i>Add, Modify, and Delete Logical Sources, Devices, and Sub-Devices</i>	280
<i>Trace Logical Sources, Devices, and Sub-Devices</i>	281
Logical Sources Form	281
Logical Sources Form Procedures	282
<i>Add a Logical Source</i>	282
<i>Modify a Logical Source</i>	282
<i>Delete a Logical Source</i>	283
Logical Devices Form	283
Logical Devices Form Procedures	284
<i>Add a Logical Device</i>	284
<i>Modify a Logical Device</i>	284
<i>Delete a Logical Device</i>	285
Logical Sub-Devices Form	285
Logical Sub-Devices Form Procedures	286
<i>Add a Logical Sub-Device</i>	286
<i>Modify a Logical Sub-Device</i>	286
<i>Delete a Logical Sub-Device</i>	286

CHAPTER 9	<i>Troubleshooting</i>	289
	Enabling Verbose Logging	289
	Testing if the LS OpenAccess Service is Online	289
	Error Messages	289
	Errors List	290
	Warning List	292
	Starting the OpenAccess Tool	292
	Using the OpenAccess Tool	292
	Creating Instances	292
	Modifying Instances	293
	Deleting Instances	293
	Authentication Expiration Warning for OpenAccess Tool	293
	Symptoms and Solutions	293
	Errors Connecting to the Message Broker	293
	SSL/TLS Secure Channel Errors	294
	CORS Errors When Accessing the OpenAccess API from a Web Application	294
	CORS Errors When Running the Cardholder Sample Web Application	294
	Errors After Updating the nginx.conf File	294
	Event Subscribers Do Not Receive Any Events	294
	Event Subscribers Do Not Receive Software Events	295
	Cannot Log Into OpenAccess Using Manual Single Sign-On	295
	Cannot Get Cardholders From Active Directory with Administrator Account	295
	Unsuccessful OpenAccess Operations From Behind a Network Proxy	295
	LS OpenAccess Service Does Not Start in a Cluster Environment	296
APPENDIX A	<i>Event Generator</i>	299
	Event Generator Main Window	299
	Edit Event (Simple) Window	300
	Edit Event (Advanced) Window	302
	Event Generator Menus	306
	File	306
	Edit	306
	Send Event	306
	Generate Events	307
	Required Event Generator Files	307
	Setting Up the Event Generator	307
	Registering the LnIEventGeneratoru.dll	308
	Adding an Event to the Event Generator	310
	Adding an Event Using the Simple User Interface	310
	Adding an Event Using the Advanced User Interface	310
	Generating Events	310
	Generating a Single Event	310
	Generating Multiple Events	310
	Saving an Event List	311
	Loading an Event List	311
	Closing the Event Generator	311

<i>APPENDIX B</i>	<i>Additional Copyright and Licensing Information</i>	313
	Entity Framework	313
	LinqToQuery	316
	Antlr	316
	Newtonsoft.Json	317
	SignalR	317
Index		319

This document provides information about the LS OpenAccess service that can be used to manage OnGuard and to integrate it with external systems such as IT systems. The LS OpenAccess service is the API into OnGuard, and provides access to ID management data, hardware events, software events, and access control events when changes are made to cardholders and their credentials.

The REST proxy that is part of the LS OpenAccess service allows you to create a client against a REST API to OnGuard through NGINX as the web service which abstracts the Advanced Message Queuing Protocol (AMQP) language. The LS Web Service is the service hosting NGINX. OpenAccess requires the LS Message Broker service, and Secure Socket Layer (SSL) must be enabled. The client uses the REST proxy to communicate with the LS OpenAccess service.

Note: If using OpenAccess or Enterprise in a cluster environment and using the default installed certificates, the certificates might need to be reissued on the machine running the LS Message Broker service. For instructions, refer to “Manually Issue an SSL Certificate” in the *NEC ExpressCluster X R3 Installation Guide* or the *Using Microsoft Cluster Services with OnGuard* guide. Also refer to the “OnGuard and the Use of Certificates” appendix in the *OnGuard Installation Guide*.

The OpenAccess Tool is also installed with the LS OpenAccess service for troubleshooting purposes, and is a client to the LS OpenAccess service. These services and the tool are applications that are installed on the servers.

The following are some common scenarios where OpenAccess can integrate OnGuard with IT systems:

Notes: OpenAccess is not intended to perform large batch processing tasks. If performing batch processing, you will achieve improved performance by using the DataExchange Server instead of OpenAccess.

There are some minor differences in behaviors between OpenAccess and legacy thick clients such as Alarm Monitoring and System Administration. For more information, refer to [Expectations and Behaviors of OpenAccess](#) on page 16.

- When a cardholder is created, the IT department creates a Windows account for that person. The Windows account name is derived from the OnGuard cardholder name. The account is linked to the cardholder in the OnGuard software.

- A single script creates an LDAP account, a cardholder, a badge for this cardholder (with a badge type, assigning default access levels), and a link between the account and this cardholder.
- A single script terminates a person's access to all company resources by disabling all of the person's badge(s) and LDAP accounts.
- When a cardholder is granted access to an area, that cardholder is granted access to use the computers in that area.
- A cardholder enters the building under duress. The cardholder's LDAP accounts are disabled to prevent potential unauthorized use.
- A cardholder's phone number changes in the OnGuard software. The new phone number is propagated to the associated Windows account in the company's Active Directory.

Administrators can also write scripts and applications that interact only with the OnGuard software. Examples include command line tools that automate frequent administrative tasks and web user interfaces that provide thin-client access to ID management data.

Expectations and Behaviors of OpenAccess

For applications that are built on the OpenAccess platform, there are minor differences in behavior between the web applications and existing client applications such as OnGuard Alarm Monitoring or OnGuard System Administration. The following sections describe these differences. Use this information in addition to [Troubleshooting](#) on page 289 to diagnose OpenAccess-related issues that may occur.

Confirming the Installed Version of OnGuard

Verify that OpenAccess and its dependent services are configured correctly by confirming that the following URL can be accessed to retrieve the installed OnGuard version:

```
https://<servername>:8080/api/openaccess/version?version=1.0
```

where <servername> is the name of the OnGuard server where Open Access is running.

The expected result should be:

```
{"product_name": "OnGuard 7.x Enterprise  
(Standard)", "product_version": "7.x.xxx.x"}
```

If this test fails, refer to [Chapter 9: Troubleshooting](#) on page 289.

Stopping and Restarting the Services

Stopping and restarting the services is generally unnecessary. The services are installed with their properties configured to start automatically. However, if there is an issue with a service, refer to [Stopping and Restarting the Services](#) on page 30 for more information.

Note: When enabling segmentation in an OnGuard system, or when enabling or disabling an OnGuard system's segmentation options, restart the OpenAccess service to prevent unexpected behavior. It is *not* necessary to restart the OpenAccess service when changing a user's segment access, as segment assignments are refreshed every 1 minute by default. This refresh interval is configured using the `permission_refresh_interval` property. For more information, refer to [Caching Properties](#) on page 20.

Authorization

All functionality available through OpenAccess is controlled by the same permissions that you are already using to manage data in the OnGuard software. For example, if you want to add a cardholder through OpenAccess, you must have the Add Cardholder user permission. If you want to view readers through OpenAccess, you must have the View Reader user permission.

OpenAccess caches user credentials and segments for 1 minute by default. This is done for performance reasons. Therefore, if a user is using an application built on the OpenAccess platform and that user's permissions or segments change, the user will continue to have his old permissions until the 1-minute timeout is reached.

The Event Context Provider service, which is responsible for sending events matching event subscriptions, caches user credentials and segments for 15 minutes by default. OnGuard Monitor requires the Event Context Provider service.

User-Defined Fields

The user-defined field schema is updated every 5 minutes. If a user changes, adds, or deletes a property using FormsDesigner, it will take up to 5 minutes for the change to appear in the LS OpenAccess service. For more information, refer to [User-Defined Fields](#) on page 42.

OpenAccess and Brute Force Attack Protection

OpenAccess protects users against Brute Force Attacks, where an attacker attempts to log into a user account repeatedly in an attempt to determine the password. The number of attempts and duration of lockout can be configured using the **put password policy settings** call. For more information, refer to [put password policy settings](#) on page 136.

For more information about brute force attacks, refer to [OpenAccess and Brute Force Attack Protection](#) on page 49.

Using OpenAccess to Issue Mobile Badges

If you are using an application built on the OpenAccess platform to issue mobile badges and are behind a network proxy, an error might occur when issuing or managing mobile credentials. To resolve this error, on the server where the LS OpenAccess service is running, change the logon account for the LS OpenAccess service from Local System to a user whose account has the correct proxy settings configured. For more information, refer to [get badge mobile_devices](#) on page 85.

Authenticated Token and Inactivity Timeouts

When using an application built on the OpenAccess platform, there are two properties that terminate authenticated sessions.

The **authenticated token timeout** property terminates an authenticated session after a predetermined, user-configurable time period. The default value for this time period is 8 hours. During this period, if there is no activity from the authenticated user within a predetermined, user-configurable time period (default of 15 minutes), the **inactivity_timeout_in_minutes** property of the **password policy settings** call terminates the authenticated session.

These properties are system-wide, which means every browser-based client of that OpenAccess server will have the same timeout settings applied. In an Enterprise system, these properties can be configured at each region to support local usage and regulation of the applications.

Notes: The authenticated token inactivity timeout property only applies if there are no cameras added to OnGuard Surveillance. If there is at least one camera added, OnGuard Surveillance polls OnGuard every 30 seconds, thus negating the no-activity period that this property monitors.

The inactivity timeout only affects browser-based clients that use OpenAccess. The inactivity timeout does not affect OnGuard thick clients.

The **authenticated token timeout** property can be configured in the **openaccess.ini** file. For more information about the **openaccess.ini** file, refer to [OpenAccess Custom Configuration](#) on page 19. For more information on the **inactivity_timeout_in_minutes** property, refer to [get password policy settings](#) on page 134 and [put password policy settings](#) on page 136.

OpenAccess Custom Configuration

OpenAccess can be configured by using an optional **openaccess.ini** file. This file is not provided upon installation of OpenAccess or the OnGuard software. Use a text editor to create an INI file in **C:\ProgramData\LnI**. Properties in the **openaccess.ini** file should remain unchanged. However, if a property is modified, restart the LS OpenAccess service in order for changes to take effect.

INI files typically organize properties into sections. For example, the following is an example of how the `authenticated_token_timeout` property should be set in the authentication section:

```
[authentication]
authenticated_token_timeout=12
```

Refer to the following sections for configurable properties.

Note: If the selected value cannot be parsed, the default value is used. If the property supports a range and the value specified is below the supported minimum value, the minimum value is used. Similarly, if the value specified is above the supported maximum value, the maximum value is used

Authentication Property

Property	Section	Default	Range	Description
authenticated_token_timeout	authentication	8	1 to 24	The authenticated token timeout, in hours.

Caching Properties

Note: Changing the caching properties to be more frequent than the default values will negatively affect performance. It is recommended to not modify the caching properties.

Property	Section	Default	Range	Description
hardware_configuration_refresh_interval	cache	300	1 to 3600	The hardware configuration refresh interval, in seconds.
hardware_status_thread_shutdown_interval	cache	15	1 to <i><value of panel_status_refresh_interval></i>	The amount of time after which a hardware status thread will shut down and stop waiting for an asynchronous status response from a particular panel, in seconds.
panel_status_refresh_interval	cache	60	5 to 3600	The time to check if cached panel's status is stale, in seconds. It is stale if the panel's status was read <i><panel_status_refresh_interval></i> seconds or more ago. When the UpdateHardwareStatus method is called, the OpenAccess service re-reads the panel's status if it is stale.
password_policy_setting_refresh_interval	cache	60	1 to 3600	The password policy setting refresh interval for an Enterprise system, in seconds.
permission_refresh_interval	cache	1	1 to 1440	The time in minutes to check if the following cached items are stale. It is stale if the cached item was read <i><permission_refresh_interval></i> minutes or more ago: • The logged-in user's permissions • The logged-in user's segment information • Cardholder options • System options

Property	Section	Default	Range	Description
udf_refresh_interval	cache	5	1 to 99999	The time in minutes to check if the following cached items are stale. It is stale if the cached item was read <i><udf_refresh_interval></i> minutes or more ago: - UDF - List Builder - The permission group's possible values provided in the <i>get Lnl_User</i> type call
user_cache_per_sid_count_threshold	cache	150	1 to 99999	The maximum number of cached connections per user. For each normal API, the OpenAccess service needs a connection to do actions like database query, user cache, and so on. The connection is locked at the begin of API and released at the end of API. When the OpenAccess service gets an API call from a user, it will do one of following: - Creates a new connection and caches it if all connections for the same user are in use and the number of connections is less than <i><user_cache_per_sid_count_threshold></i> - Use a cached connection if there is a cached connection for the same user and it is not in use - Waits for a cached connection to be released if all connections for the same user are in use and the maximum number of connections is reached

Send Incoming Events Property

Name	Section	Default	Range	Description
max_incoming_events_count	send-incoming-events	10	1 to 99999	The maximum count of incoming events to send in one call.

Badge Printing Properties

Use these properties to control how items are cleared from cache after making print requests. The expiration threshold is counted from the **submitted_at** property's value returned with the print request.

Property	Section	Default	Range	Description
poll_in_minutes	badgeprinting	15	1 to 1440	Determines how often the background thread polls for old badge print requests, in minutes.
expiration_threshold_in_minutes	badgeprinting	60	5 to 1440	Dictates how long the badge print requests will exist in the in-memory cache, in minutes.

Sample openaccess.ini content:

```
[badgeprinting]
poll_in_minutes=1
expiration_threshold_in_minutes=5
```

HTTP Request Properties

Use these properties to control how many HTTP requests OpenAccess can handle simultaneously.

Property	Section	Default	Range	Description
request_pool_size	http_request	32	1 to ({job_runner_name}_thread_number - 3)	Configures how many HTTP requests OpenAccess can handle simultaneously. OpenAccess creates threads for each of those simultaneous requests. So, its maximum value should be three fewer than the size of the thread pool ({job_runner_name}_thread_number - 3). Increasing or decreasing this value will impact system performance, since it will decrease or increase the number of other threads that handle these tasks: - Request queue task - Request timeout task - Handle a message from the message bus

Property	Section	Default	Range	Description
busy_request_pool_size	http_request	8	0 to ({job_runner_name}_thread_number - 3 - request_pool_size)	If this value is greater than 0, OpenAccess creates additional threads to handle the requests simultaneously. The client will receive "503 request pool full" error if the number of queued requests is greater than request_pool_size. It is recommended to modify this value to 0 to avoid this 503 error.

Sample openaccess.ini content:

```
[http_request]
request_pool_size = 100
busy_request_pool_size = 0
```

Queuing Property

Property	Section	Default	Range	Description
task_expiration	queue	60	1 to 1440	The time to expire a queued task, in minutes.

Job Runner/Thread Pool Properties

Property	Section	Default	Range	Description
names	job_runner	default	default	Lists the job runner names to be configured. Job runner names should match the service they are used by. The default job runner is named default . The OpenAccess job runner should be named openaccess . The REST proxy job runner should be named rest_proxy . Names should be comma separated. For example: names=default,rest_proxy,openaccess .
{job_runner_name}_thread_number	job_runner	256	1 to 65535	Configures the size of the thread pool for the given job runner.
{job_runner_name}_jobs_limit	job_runner	1024	1 to 65535	Configures the maximum number of queued jobs for the given job runner.

Sample openaccess.ini content:

```
[job_runner]
names=default
default_thread_number=30
default_jobs_limit=100
```


Timeout Property

Property	Section	Default	Range	Description
request_timeout	timeout	30	1 to 300	The OpenAccess timeout, in seconds. Requests taking longer than this value will result in an OpenAccess timeout error.

Event Context Provider Properties

Property	File > Section	Default	Description
HardwareCacheRefreshRateInHours	Lnl.OG. EventContext ProviderService. exe.config > appSettings	1	Hardware related cache refresh interval.
MinutesBetweenPrincipalCacheCleanups	application.config > appSettings	15	The permission cache refresh interval.

Definitions, Acronyms, Abbreviations

Class

A definition of a type of object. For example, the Lnl_Reader class is a definition for an access control reader.

Client

A script or application that uses OpenAccess.

JSON

JavaScript Object Notation.

Object/Instance

A representation of a particular class with actual data.

Person

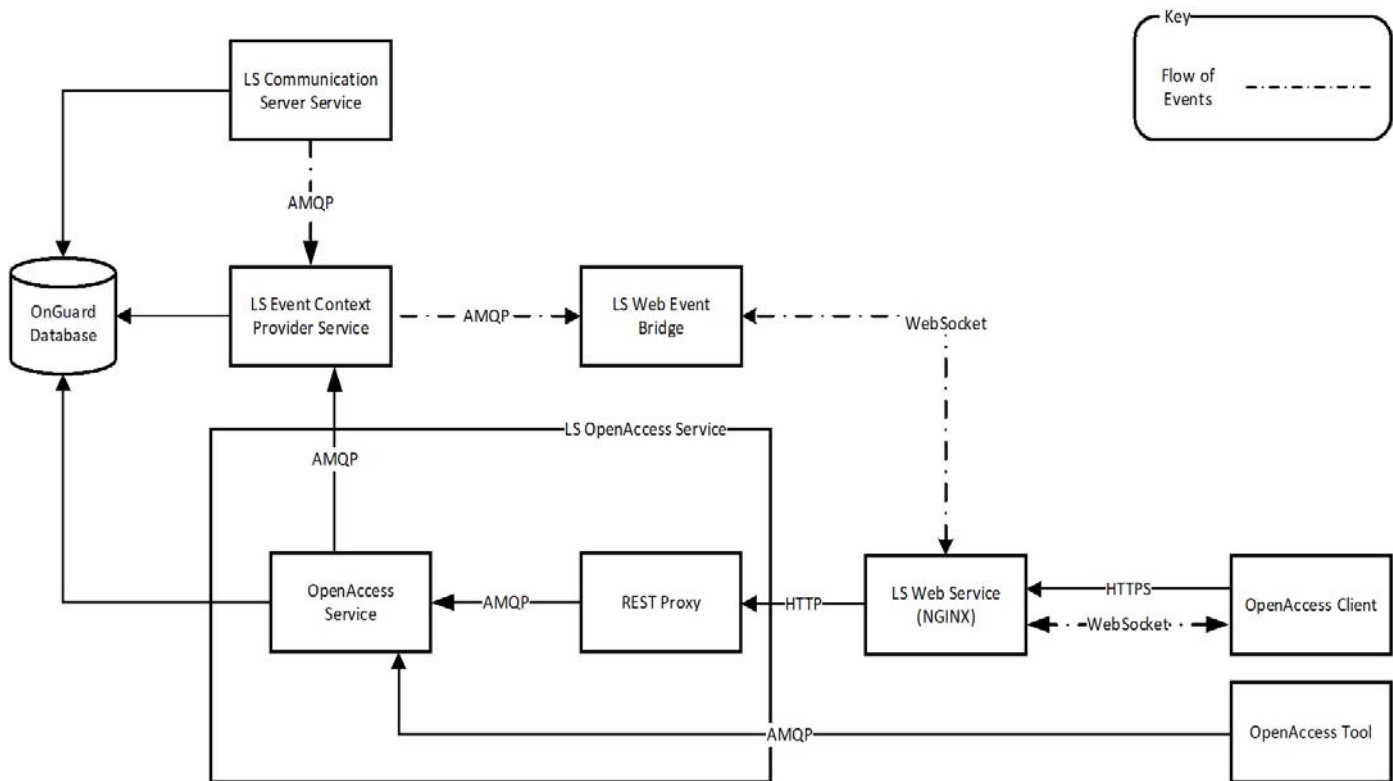
A cardholder or visitor.

SDK

Software Development Kit.

OpenAccess Architecture

OpenAccess Architecture



The LS Communication Server service publishes an event to the LS Event Context Provider service, which provides additional detail about the event. If the subscriber is using the LS Web Event Bridge, this service will begin publishing events to the client via WebSocket. For example, if the LS Communication Server service publishes an Access Granted event, the LS Event Context Provider service adds cardholder details. The event, with the added detail, is provided to the AMQP queue for each subscriber that has permission to receive information about the event. If the subscriber is using the LS Web Event Bridge, this service will publish events to the client via WebSocket.

The LS OpenAccess Service includes both the OpenAccess Service and REST Proxy. The LS Message Broker service provides the AMQP protocol. The LS Web Service (NGINX) exposes endpoints for each web service.

Note: Each subscriber has its own queue on the LS Message Broker service. This is done for security purposes, allowing subscribers to see only the event information they are authorized to see.

References and Applicable Documents

Note: Throughout this document, references to the <OnGuard installation directory> means the OnGuard installation directory. This is typically **C:\Program Files (x86)\OnGuard**, but may be different depending on system configuration and any custom path selected during OnGuard installation.

Microsoft Scripting Technologies documentation is located in the MSDN library at <http://msdn2.microsoft.com/en-us/library/ms950396.aspx>.

Information on JavaScript Object Notation (JSON) can be found at <http://www.json.org/>.

Information about NGINX can be found at <http://nginx.org/>.

This section provides details about procedures that must be performed before using the LS OpenAccess service, including:

- [License for OpenAccess](#) on page 29
- [Starting OpenAccess](#) on page 30
- [Stopping and Restarting the Services](#) on page 30
- [LS OpenAccess Service](#) on page 31
- [Authorization](#) on page 32
- [Authentication](#) on page 32
- [Deploying the LS Event Context Provider Service](#) on page 33
- [Enabling Verbose Logging](#) on page 33
- [Starting the OpenAccess Tool](#) on page 34
- [Sample Applications](#) on page 34

License for OpenAccess

OpenAccess is a licensed feature. For more information, refer to *Install Your OnGuard License* in the *Installation Guide*.

Application ID and Getting Started with Development

Each application or solution using OpenAccess must have a unique application ID and a specific license. You can obtain this development license along with additional license information by sending an email to openaccess@carrier.com with the subject *OA Dev Kit Request*. Your message should include the following:

- Contact information
- General description of the integration type you will develop using OpenAccess services

A company representative will contact you and help you obtain an OpenAccess license.

Starting OpenAccess

The LS OpenAccess service requires the LS Message Broker Service, and Secure Socket Layer (SSL) must be enabled. The LS Message Broker service is deployed with OnGuard servers automatically. For information on configuring the LS Message Broker Service, refer to the *System Options Folder* chapter in the *System Administration User Guide*.

1. Confirm that the LS Message Broker service is running on the workstation identified on the **System Administration > System Options** form.
2. Confirm that the LS OpenAccess service is running on the workstation identified on the **System Administration > System Options** form.

Note: Both the LS Message Broker service location and the LS OpenAccess service location configured on the **System Administration > System Options** form must match the deployed certificate name perfectly, or SSL/TLS errors will result. For more information, refer to [SSL/TLS Secure Channel Errors](#) on page 294.

3. Confirm that the LS Web Service is running.
4. Confirm that the LS Event Context Provider service is running.

Note: The LS Event Context Provider service must run on the same host as the LS OpenAccess service.

5. Confirm that the LS Web Event Bridge service is running.

Note: By default, the LS Web Event Bridge service is configured to locate LS OpenAccess on the same server. If you installed the LS Web Event Bridge service on a different server than the LS OpenAccess service, open the **Lnl.OG.WebEventBridgeService.exe.config** file and edit the proxy to the Fully Qualified Domain Name (FQDN) of the server running LS OpenAccess.

For more information, refer to [OpenAccess Architecture](#) on page 26.

LS OpenAccess can also be run as an application. For troubleshooting purposes, select **Start > All Programs > OnGuard > Service and Support > OpenAccess**.

Stopping and Restarting the Services

Stopping and restarting the services is generally unnecessary. The services are installed with their properties configured to start automatically.

In a few limited circumstances, however, you will need to stop and restart the LS OpenAccess service and the LS Event Context Provider service to allow it to retrieve new configuration information. You should stop and then restart these services after any of the following changes are made:

- You change the database connection information. For more information, refer to the *Configuration Editor* appendix in the *Installation Guide*.
- You install a new license.
- You make segmentation changes.

Note: When enabling segmentation in an OnGuard system, or when enabling or disabling an OnGuard system's segmentation options, restart the OpenAccess service to prevent unexpected behavior. It is *not* necessary to restart the OpenAccess service when changing a user's segment access, as segment assignments are refreshed every 1 minute.

by default. This refresh interval is configured using the `permission_refresh_interval` property. For more information, refer to [Caching Properties](#) on page 20.

- You make hardware changes, and you don't want to wait for the LS Event Context Provider to refresh its hardware cache. For more information, refer to [Deploying the LS Event Context Provider Service](#) on page 33.

If you change the location of the LS Message Broker service, you must also restart the following services:

- LS OpenAccess service
- LS Web Event Bridge
- LS Event Context Provider service

LS OpenAccess Service

REST service provider URL: <protocol>://<host>:8080/api/access/onguard/openaccess

The REST proxy that is part of the LS OpenAccess service interprets web requests intended for OpenAccess, and allows web clients to interface with the LS OpenAccess service. The LS OpenAccess service uses NGINX as the web service.

For information on how to format the “REST Request URL” proxy calls for each method, refer to [Chapter 4: REST API Reference](#) on page 53.

For some methods, “REST Request Body Contents” is also provided if a response is expected. The body is a JavaScript Object Notation (JSON) representation of the key-value pairs for each method.

Sample Request and Response With an Error

```
1  POST /api/access/onguard/openaccess/authentication?version=value
2
3  Header:
4  Application-Id: SUPPLIED_APPLICATION_ID
5  Session-Token: 12345-67890-12345-67890
6
7  Body:
8  {
9      "user_name": "admin",
10     "password": "badpass",
11     "directory_id": "directory",
12 }
13
14 HTTP/1.1 401
15 {
16     "error":
17     {
18         "code":"openaccess.general.invalidapplicationid",
19         "message":"You are not licensed for OpenAccess."
20     }
21 }
```

Authorization

All functionality available through OpenAccess is controlled by the same permissions that you are already using to manage data in ID CredentialCenter. For example, if you want to add a cardholder through OpenAccess, you must have the Add Cardholder user permission. If you want to view readers through OpenAccess, you must have the View Reader user permission.

Notes: OpenAccess caches user credentials and segments for 1 minute by default. This is done for performance reasons. Therefore, if a user is using OpenAccess and that user's permissions or segments change, the user will continue to have his old permissions until the 1-minute timeout is reached.

The Event Context Provider service, which is responsible for sending events matching event subscriptions, caches user credentials and segments for 15 minutes by default.

Authentication

Authentication to the LS OpenAccess service uses the OnGuard internal account or manual Single Sign-On (SSO) only. This differs from DataConduIT, which uses automatic SSO only. For more information, refer to the *Single Sign-On* section of the *Installation Guide*.

The following calls do not require authentication:

- get directories (See [get directories](#), on page 60 for details.)
- get version (See [get version](#), on page 56 for details.)

All other calls require authentication. Call **add authentication** to perform the authentication to the service. By default, an authenticated session expires 8 hours after it was created. For more information, refer to [add authentication](#) on page 61.

Supported Authentication Modes

OpenAccess supports the following authentication modes. Regardless of mode, the client begins with the **add authentication** call. Subsequent calls require an authentication token provided in one of the following ways.

Token-based Authentication Requests (all OnGuard versions)

For token-based authentication, the Session-Token parameter returned in the **add authentication** response must be provided in the request header for all subsequent calls.

Cookie-based Authentication Requests (OnGuard 7.6 and later)

The response to the **add authentication** call includes a Set-Cookie header resulting in a **OASessionID** session cookie. This cookie contains the session token and has the following attributes:

- HttpOnly
- Secure
- Samesite=Strict

To take advantage of cookie-based authentication, the application no longer needs to persist the session_token contained in the response body. Instead, the application must associate the cookie with a subsequent OpenAccess REST API request.

JavaScript API Request Example

Regardless of which JavaScript API you use, you must explicitly define or set the credentials for the HTTP request as shown below with **xhr.withCredentials=true**.

```
1 var xhr = new XMLHttpRequest();
2 xhr.open('GET', 'https://<URL>:8080/api/access/onguard/openaccess/
  type?type_name=lnl_reader&version=1.0', true);
3 xhr.withCredentials = true;
4 xhr.send(null);
```

Adding and Deleting Authentication

Every **add authentication** call should have a matching **delete authentication** call. Not calling **delete authentication** consumes resources unnecessarily over time.

It is also recommended that you keep a session token available (that is, do not **delete authentication**) until you no longer need to execute commands for at least several minutes. In other words, avoid rapidly adding and then deleting authentication within seconds. This can happen if you have a snippet of code that executes very frequently, and that code adds and then deletes authentication. Consider keeping the authentication token and executing the snippet frequently, and then deleting authentication only if the code won't execute again for another 1 or 2 minutes.

Deploying the LS Event Context Provider Service

The Communication Server publishes an event to the LS Event Context Provider service, which provides additional details about the event. For example, if the Communication Server publishes an Access Granted event, the LS Event Context Provider service adds cardholder information details. The event, with the added detail, is provided to the Direct Subscriber and Web Subscribers Event Queues where it can be shared with both Direct and Web Subscribers.

Note the following details about the LS Event Context Provider service:

- This service will only run on the workstation configured to run the LS OpenAccess service.
- This service logs all activity to the **EventContextProviderService.log** file located in the **C:\ProgramData\Ln\logs** directory.
- The LS Event Context Provider service refreshes its cached information every 1 hour. This includes badge/cardholder details as well as hardware information.

Enabling Verbose Logging

By default, the log file only shows error messages. Enable Verbose Logging when additional log details are required, such as when troubleshooting OpenAccess issues.

Note: The Event Generator is another useful troubleshooting tool. Use Event Generator to create “fake” events that can be received by event subscribers. For more information, refer to [Appendix A: Event Generator](#) on page 299.

To enable Verbose Logging:

1. Launch the Configuration Editor by selecting **Start > All Programs > OnGuard > Service and Support > Configuration Editor**.
2. Select **Show advanced settings**.

3. In the Verbose Logging section, select **LS OpenAccess**.
4. Click [Save Changes].

Note: You do not need to restart the LS OpenAccess service after enabling Verbose Logging.

By default, the OpenAccess.log file is located in **C:\ProgramData\Len\logs**. Disable Verbose Logging when finished troubleshooting to prevent the log file from growing too large.

Starting the OpenAccess Tool

The OpenAccess Tool is a sample client used for troubleshooting purposes. To start the tool, navigate to **Program Files (x86)\OnGuard**, and then double-click **OpenAccessTool.exe**. For more information, refer to [Chapter 9: Troubleshooting](#) on page 289.

Note: To run the OpenAccess Tool, you will be prompted to enter a valid Application ID. Contact LenelS2 OnGuard Technical Support if you do not have an Application ID.

Sample Applications

Sample applications that demonstrate how to use the OpenAccess API are located in **<OnGuard installation directory>\doc\en-US\OpenAccess Samples**.

Sample Web Applications

The following table lists the sample web applications:

Application	Description	APIs Used
Cardholder Search	Demonstrates how to authenticate, use pagination while searching, and provide some cardholder details such as the photo.	- get directories - add/delete authentication - get instances
Command and Control	Demonstrates how to list panels, readers, and panel status; search for panels by name; search for readers by name; paging; open doors; and change reader modes.	- get directories - add/delete authentication - get instances - execute method
Event Subscriber	Demonstrates how to create a subscription to receive events.	- get directories - add/delete authentication - add/modify/delete event_subscriptions - Web Event Bridge for receiving events using WebSocket

Configuring the Sample Web Applications

1. Load the sample web applications using one of the following methods:

- Temporarily add CORS support for sites accessed on a local drive by uncommenting the example configuration for the “null” origin in the **C:\ProgramData\Lnl\nginx\conf\cors.conf** file. For more information, refer to [Cross-Origin Resource Sharing](#) on page 48.
- Host the samples in NGINX to avoid CORS errors, by doing the following:
 - i. Rename **C:\ProgramData\Lnl\nginx\conf\modules\openaccess_samples.conf.disabled** to **openaccess_samples.conf**, removing the “.disabled” suffix. You can disable the samples again by adding the “.disabled” suffix again.
 - ii. Depending on where OnGuard is installed, you might need to update the value of **\$onguard_install_dir** in **C:\ProgramData\Lnl\nginx\conf\environment.conf**.
- 2. Regardless of which method you used to load the sample web applications, restart LS Web Service to pick up any NGINX configuration changes.
- 3. Each web application uses <https://localhost:8080/api/access/onguard/openaccess> as the default URL for the OpenAccess API. Each sample web application has a line in the **app.js** JavaScript file that looks similar to the following:


```
API_URL = 'https://localhost:8080/api/access/onguard/openaccess', // OpenAccess REST API endpoint
```

 Modify this line with the Fully Qualified Domain Name (FQDN) of your server.

Notes: If developing your own application, using WebSockets as the transport improves performance. To do this, target .NET Framework 4.6.1 or later instead of .NET Framework 4.0, as shown in this sample application. WebSockets functions correctly with any version of Windows supported by the OnGuard software.

When the LS Web Event Bridge service is restarted, it loses subscription details for all existing clients. Therefore, clients must re-subscribe to continue receiving events. New transient subscriptions must be created, but durable subscriptions can be re-established with the **ModifySubscription** call ([ModifySubscription](#) on page 157).

The sample clients do not listen for connection lost events. If the SignalR connection to the LS Web Event Bridge is lost, the client can modify or create a new subscription via the Web Event Bridge API to restore the SignalR connection and the flow of events. This limitation does not exist when using WebSockets. For more information, refer to [Chapter 5: Event API Reference](#) on page 155.

Running the Sample Web Applications

If loading the sample web applications from a local drive, use a web browser to load the web application’s **index.html** directly from the local drive.

If hosting the sample web applications in NGINX, open the URL of the sample in the web browser.

Sample C# Applications

The following table lists the sample C# applications:

Application	Description	APIs Used
Command and Control	Demonstrates how to list panels and readers, change reader mode, and open doors.	• get directories • add/delete authentication • get instances • execute method

Application	Description	APIs Used
Event Subscriber	Demonstrates how to create a subscription to receive hardware and software events.	• add/delete authentication • add/modify/delete event_subscriptions • Web Event Bridge for receiving events using WebSocket

Configuring the Sample C# Applications

For the Command and Control sample, the API URL is initially hardcoded to <https://localhost:8080/api/access/onguard/openaccess>. Modify the **API_URL** in the **RequestBuilder.cs** file to the Fully Qualified Domain Name (FQDN) of your server.

For the Event Subscriber sample:

- The API URLs, credentials, and subscription parameters are configured in the **App.config** file.
- The sample clients do not listen for connection lost events. If the SignalR connection to the LS Web Event Bridge is lost, the client can modify or create a new subscription via the Web Event Bridge API to restore the SignalR connection and the flow of events. For more information, refer to [Chapter 5: Event API Reference](#) on page 155.

Notes: If developing your own application, using WebSockets as the transport improves performance. To do this, target .NET Framework 4.6.1 or later instead of .NET Framework 4.0, as shown in this sample application. WebSockets functions correctly with any version of Windows supported by the OnGuard software.

When the LS Web Event Bridge service is restarted, it loses subscription details for all existing clients, and in some cases (such as not using WebSockets, stopping the LS Web Event Bridge process and then restarting it immediately) the client is not notified that the LS Web Event Bridge service has restarted. Therefore, clients must re-subscribe to continue receiving events. Clients can call **ConnectionHeartbeat** periodically (for example, make the call every 10 seconds) to monitor the connection and re-subscribe events. ConnectionHeartbeat might generate an exception for some cases, such as:

The LS Web Event Bridge service is stopped. Clients can call **Microsoft.AspNet.SignalR.Client.Connection.Start()** to re-establish the connection, and then call **ConnectionHeartbeat** or **ModifySubscription** to re-subscribe events.

The OpenAccess session token is expired. Clients can call **OpenAccess.add_authentication** to get new session token.

Building the Sample C# Applications

You can compile the C# applications with Visual Studio 2015 or later. These projects use NuGet for third party dependencies, so your workstation needs access to <https://www.nuget.org> for the NuGet packages to restore successfully.

Sample Java Application

The following table describes the sample Java application:

Application	Description	APIs Used
Event Subscriber	Demonstrates how to create a subscription to receive events. The sample Java application builds with Gradle (http://gradle.org).	- add/delete authentication - Web Event Bridge for receiving events using long polling

Configuring the Sample Java Application

The OpenAccess service URL, login credentials, and other parameters are defined in **src/main/java/Program.java**. Update these parameters to reflect your environment.

The sample clients do not listen for connection lost events. If the SignalR connection to the LS Web Event Bridge is lost, the client can modify or create a new subscription via the Web Event Bridge API to restore the SignalR connection and the flow of events. For more information, refer to [Chapter 5: Event API Reference](#) on page 155.

Building the Sample Java Application

1. Install the Java Development Kit (JDK).
2. Execute `gradlew build` at a command prompt. The first time you run this command, Gradle and the Java dependencies are downloaded. If you are behind a proxy, you might need update the **gradle.properties** file with the correct proxy information. Uncomment each line by removing the `#` and specify the proxy host and port. Update all four lines to set the proxy for both **HTTP** and **HTTPS** protocols.

Running the Sample Java Application

1. Make sure the root certificate of the SSL certificate is installed in the Java **cacerts** certificate store, making the SSL connection to OpenAccess trusted.
 - a. If using the default SSL certificate, export the root **Prism SOA Common Trusted Root** certificate from the **Trusted Root Certification Authorities** store of the local computer using Microsoft Management Console. Export the certificate with either DER or Base-64 encoding.
 - b. Run a command like the following, which adds the exported certificate to the Java certificate store. This will depend on the version of the Java Runtime Environment (JRE) you are using. You will need to enter a password, which is usually `changeit` or `changeme` by default, depending on the environment.


```
c:\Program Files\Java\jdk1.8.0_65\jre\bin\keytool.exe" -importcert
-alias prismsoaroot -file "F:\Certificates\PrismSOARoot.cer" -
keystore "C:\Program Files\Java\jdk1.8.0_65\jre\lib\security\
cacerts
```
2. Execute `gradlew run`, or extract one of the archives in **build\distributions** (created by `gradlew build`) and execute the appropriate startup script in the **bin** directory. If you run the sample with Gradle, the sample output will be contained within the Gradle output, which can be confusing if you are not familiar with it. For example, you will see something like `Building 75% > :run` on the last line of output while the sample is running. This indicates that the current Gradle task being executed is the `run` task. The sample is listening for events as soon as it prints `Connection to message bus established`. Press [Enter] to exit the sample.

Notes: The command ``gradlew run`` uses the JDK's private JRE (probably **C:\Program Files\Java\jdk1.8.0_65\jre**). Running the build output in **build\distributions** uses the public JRE in the path (probably **C:\Program Files\Java\jre1.8.0_65**), as expected.

For more information about certificates, refer to the “OnGuard and the Use of Certificates” appendix in the *OnGuard Installation Guide*.

Swagger Specification and Interactive Documentation

Many developers find the Swagger specification and interactive documentation useful for testing an API and discovering how to work with it. Swagger is supported by many tools, which might be useful when developing solutions that use the OpenAccess REST API.

A Swagger specification is available for the OpenAccess REST API at **<OnGuard installation directory>\doc\en-us\OpenAccess Swagger\swagger.yaml** or at **https://<server>:8080/api/access/onguard/openaccess/swagger.yaml**. Live documentation is also available at **https://<server>:8080/api/access/onguard/openaccess/swagger**.

For information about Swagger, refer to <http://swagger.io/>. For information about the Swagger documentation specification, refer to <http://swagger.io/specification/>.

Note: Depending on where OnGuard is installed, you might need to update the value of **\$onguard_install_dir** in **C:\ProgramData\Ln\nginx\conf\environment.conf**. Restart LS Web Service to pick up any NGINX configuration changes.

Using Response Headers to Develop Secure Web Applications

To mitigate attacks and security vulnerabilities in web applications, you should utilize response headers as shown in the **httpsecurity.conf** file, located by default in the **C:\ProgramData\Ln\nginx\conf** directory. You can either reference this **httpsecurity.conf** file, or you can specify the response headers you need directly in your web application code.

For more information about response headers and best practices for security, refer to:

- https://www.owasp.org/index.php/Main_Page
- https://www.owasp.org/index.php/List_of_useful_HTTP_headers#tab=Headers
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers#Security>

Searching for Objects

Filters are specified in OpenAccess syntax, which is a subset of the Structured Query Language (SQL) supported by most databases.

The expected format of a filter is:

`PROPERTY_NAME = VALUE`

To give you a feel for the OpenAccess syntax, here are some filters that you could use with OpenAccess. You could use these filters with the **get instances** call. For more information, refer to [get instances](#) on page 80.

Notes: You must use double-quotes around string delimiters when filtering. Single-quotes will result in a `system.parse` error.

You cannot search on some fields, such as encrypted text and password fields. If you search on an encrypted text or password field, an error is shown. Refer to the `display_attributes` response from [get type](#) on page 77 to determine if a field is searchable.

If the `\` or `"` characters are part of a name, those characters must be escaped in the search string. For example, if the name to search for is `Includes\Backslash`, it should be entered in the filter as `Includes\\Backslash`, and if the name is `Includes"Quote`, it should be entered as `Includes\"Quote`.

Find all cardholders whose last name is not “Lake”

```
LastName != "Lake"
```

Find all cardholders whose last name starts with “La”

```
LastName like "La%"
```

Find all cardholders with either the last name is “Lake” or the first name is “Lisa”

```
LastName = "Lake" OR FirstName = "Lisa"
```

For more information, refer to [Chapter 6: Data and Association Class Reference](#) on page 181.

Date/Time Format

Date/Time Format When Using OpenAccess API Calls

OpenAccess reports all times in the local time of the server, including the offset, unless configured to report times differently. OpenAccess formats date/time values using the ISO 8601 standard:

YYYY-MM-DDTHH:MM:SS+/-00:00

All date and time values are reported to the server as strings, and are returned as strings in this format. The following example shows the time that came from an OpenAccess server running in the Eastern Time Zone while daylight savings time is in effect:

2016-04-05T20:33:47-04:00

There are some instances where time is reported in UTC, as described in this guide. The client can convert the displayed time to local time, or modify the formatting of the date and time on the client, if desired.

Date/Time Format When Using Events

The OpenAccess format for date/time strings does not apply when receiving events through subscriptions. In those instances, the date and time is a 64-bit integer that identifies the number of milliseconds after January 1, 1970 in UTC time.

Binary Format

When doing a get instances call, the REST proxy that is part of the LS OpenAccess service returns binary properties (indicated as **binary** in [Data Classes](#) on page 181) as base64-encoded strings. When doing an add or modify instance call for a type with binary data, OpenAccess expects the data as a base64-encoded string (for example, iVBORw0KGgoAAAANSUhEUgAAAGIAAABUCAIAA...).

Binary data is returned to a client as a map with the following structure:

```
"content_type": "image/jpeg",  
"data": "[base64 encoded string]"
```

Notes: "image/jpeg" is an example of the content_type. The actual value is determined by the binary data.

When doing an add or modify call, the request does not include a map. Only the response on a get instance includes a map.

Binary data (indicated as **binary** in [Data Classes](#) on page 181) is returned as raw bytes in the OpenAccess Tool, not base64 encoded.

When sending data using the OpenAccess Tool, OpenAccess expects the data as a comma-separated string of bytes (for example, 137, 80, 78, 71, 13, 10, 26, ...).

String Format

All strings are expected in UTF-8 format.

Features and Limitations

The following features and limitations are specific to class.

Cardholders and Visitors

Each cardholder and visitor instance has all of its user-defined fields (UDFs) exposed through OpenAccess. This includes system fields such as first name (FIRSTNAME), last name (LASTNAME), social security number (SSNO), and internal ID (ID). All fields except for the internal ID and last changed timestamp are available for read/write access, subject to additional UDF validation and field/page viewing permissions.

If cardholders/visitors are segmented, an additional property named PRIMARYSEGMENTID will be made part of the Lnl_Cardholder/Lnl_Visitor class. If the client is a member of only one segment, this property will default to that segment ID. Otherwise, the client must specify the primary segment ID when a new cardholder/visitor is added.

Badges

Each badge instance has all of its UDFs exposed through OpenAccess. This includes system fields such as badge ID (ID), badge type (TYPE), badge status (STATUS), and the internal ID (BADGEKEY). All fields except for the internal ID, number of badge prints, last changed, and last printed timestamps are available for read/write access subject to the validation described above.

The PIN code is exposed in a manner similar to the way it is done in ID CredentialCenter. You can set the badge PIN code by setting the property during an add or modify operation. However, if you search up a badge and attempt the read the PIN code, the property will always contain a null value.

A client will be able to assign access levels to a new badge by giving it a badge type. The new badge will be assigned the default access levels for that badge type.

In a segmented system, the client cannot change the badge type if it controls a different set of segments than the previous badge type. This is because changing the badge type of a badge could possibly remove access levels from that badge without user confirmation.

Directory Accounts

Adding an instance of Lnl_Account is equivalent to linking a directory account to a cardholder or visitor in ID CredentialCenter. Similarly, deleting an instance is equivalent to unlinking the account. When adding an instance of Lnl_Account, all fields except for the ID are required. The AccountID property refers to the value of the LDAP attribute. For Microsoft Active Directory accounts, this defaults to the account security identifier, or SID. Other LDAP directories will probably use a different LDAP attribute.

Visits

Each visit instance has all of its UDFs exposed through OpenAccess. This includes system fields such as host id (CARDHOLDERID), type (TYPE), visitor id (VISITORID), and the internal ID (ID). All

fields except for the internal ID, last changed, time in, and time out are available for read/write access subject to the validation described above.

Once a visit has been signed in, scheduled time in cannot be changed, nor can the cardholder or visitor of the visit, same thing with signing out a visitor.

E-mail recipients configured through Lnl_Visit cannot be viewed through Lnl_Visit; Lnl_VisitEmailRecipient must be used for viewing.

User-Defined Fields

The user-defined field schema is updated every 5 minutes. If a user changes, adds, or deletes a property using FormsDesigner, it will take up to 5 minutes for the change to appear in the LS OpenAccess service.

Notes: OpenAccess generates property names based on the field names shown in FormsDesigner.

When provided via the object name of a User Defined Field (UDF) in FormsDesigner, the display_name attribute is the user-friendly name of the item. For more information, refer to [get type](#) on page 77. Also refer to the “Field Properties Folder – General Settings Form” section in the FormsDesigner User Guide.

User-Defined List Values

All user-defined list (populated via List Builder) are available for view/add/modify/delete. The only values that cannot be modified are:

- Active BadgeStatus (ID = 1)
- Supervisor Two Man Type
- Team Member Two Man Type

When doing a get type call, if the type is a UDF type such as **cardholder** or **badge**, and if the type contains list builder items, the list builder items themselves are returned as possible values for that property. The type definitions themselves have a 5-minute UDF refresh interval, but the values of the properties on the possible value list is refreshed each time you call a get type. You can also call get instances on the list builder type directly to get all possible values.

Therefore, if you perform a get type call for Lnl_Cardholder, the Title property returns a list of possible values associated with it. The schema for the Lnl_TITLE type and the Lnl_Cardholder type will refresh every 5 minutes, but the list of possible values for the Title property is not cached and is provided for convenience. These values are refreshed each time you call a get type on Lnl_Cardholder. You can also get this information by doing a get instances on Lnl_TITLE directly at any time to get current values for the type.

SegmentID

SEGMENTID only appears as a property in data classes that support segmentation when segmentation for that class is enabled. For more information, refer to [get segmentation settings](#) on page 139 and [Lnl_Segment](#) on page 252. Restarting the LS OpenAccess service is required when making segmentation changes.

Receiving Events

Durable vs. Transient Event Subscribers

An event subscriber can be *durable* or *transient*, which impacts how many events are received, as well as how often a **modify event_subscriptions** call must be sent in order to keep the subscriber active.

- *Durable event subscribers* receive events that occur while the subscriber is online (for a process) or logged in (for a user), as well as events that occur when the subscriber is offline/logged out. When the subscriber comes online/logs in again, the system sends the missed events to the subscriber. To continue receiving events and remain active, a durable subscriber must send a **modify event_subscriptions** call every seven days.

Note: Because a durable subscriber's events are stored while the subscriber is offline, you should minimize offline time and delete durable subscribers that are no longer needed, to avoid overwhelming the Message Broker.

- *Transient (non-durable) event subscribers* only receive events that occur while the subscriber is online (for a process) or logged in (for a user). Events that occur when the subscriber is offline/logged out are not sent. To continue receiving events and remain active, a transient subscriber must send a **modify event_subscriptions** call every 24 hours.

Note: If either the LS Message Broker service or the LS Event Context Provider service is not running, hardware and alarm acknowledgment events might not reach the client even if those events are reported within Alarm Monitoring and are using a durable event subscription.

If a subscriber fails to send a **modify event_subscriptions** call in the expected time frame (seven days for a durable subscription, 24 hours for a transient subscription), the system will delete the subscription and stop sending events. The LS Event Context Provider checks for and deletes expired subscriptions every 10 minutes.

To learn more about **event_subscriptions** calls:

- See add event_subscriptions on page 69.
- See modify event_subscriptions with id on page 71.
- See delete event_subscriptions with id on page 72.

Note: Deleted subscriptions cannot be reinstated. Create a new subscription using the **event_subscriptions** method.

Using Event Filters with Subscriptions

When an event filter is specified with a subscription, only the events that match the criteria specified in the filter are forwarded to the subscriber. The grammar of the filter supports a basic subset of the OData filter expression language. Visit <http://www.odata.org/documentation/odata-version-2-0/uri-conventions/#FilterSystemQueryOption> for details.

There are two formats for filtering event properties:

- `<property name> <operator> <property value>`
With this filter format, the *property name* is not case sensitive, but the *operator* and *property value* are case sensitive. All hardware and alarm acknowledgment events, as well as the common properties of software events, use this filter format. For more information about common properties of software events, refer to [Common Properties for All Software Events](#) on page 174.

For example: `business_event_class eq 'software_event'` is a valid filter, but `business_event_class Eq 'Software_Event'` is not a valid filter.

- `<new_/old_properties>/[<object property name>] <operator> <value>`

With this filter format, the *new/old properties* is not case sensitive, but the *object property name*, *operator*, and *value* are case sensitive. All software event object properties use this filter format. For more information, refer to [Software Event Reference](#) on page 174.

For example: `new_properties/[LASTNAME] eq 'Smith'` is a valid filter, but `new_proproties/[LastName] Eq 'smith'` is not a valid filter. Also with this format, the value for a property that is an int64 must have an 'L' appended. For example: `new_properties/[ID] eq 8` for filtering software events by badge ID will not work. That filter must be written as `new_properties/[ID] eq 8L`.

Notes: OpenAccess will not return an error if you filter on a field that does not exist.

Also, you cannot filter software or hardware events using `timestamp` or `object_id`.

If the \ or " characters are part of a name, those characters must be escaped in the search string. For example, if the name to search for is `Includes\Backslash`, it should be entered in the filter as `Includes\\Backslash`, and if the name is `Includes"Quote`, it should be entered as `Includes\"Quote`.

Here are some examples of event filters:

Example	Event Filter
Receive only hardware events with event ID equal to 214. (Set reader mode PIN or Card)	<code>business_event_class eq 'hardware_event' and event_id eq 214</code>
Receive only hardware events related to a specific cardholder.	<code>business_event_class eq 'hardware_event' and cardholder_last_name eq 'Smith'</code>
Receive software events.	<code>business_event_class eq 'software_event'</code>
Receive hardware events.	<code>business_event_class eq 'hardware_event'</code>
Receive only software events related to a specific badge.	<code>business_event_class eq 'software_event' and software_event_object_type eq 'Badge' and new_properties/[ID] eq 1L</code>

The following hardware and alarm acknowledgment event properties can only be specified in the definition of the filter parameter for subscription API calls:

Note: The following table is for hardware and alarm acknowledgment events only. All software events can be specified in the definition of the filter parameter for subscription API calls. For more information, refer to [Software Event Reference](#) on page 174.

Field Name	Field Description
access_granted_entry_made	Definition: See Properties for Access Granted Events on page 165. Type: Boolean Example: access_granted_entry_made eq true
alarm_id	Definition: See Properties for Controller-Based Events on page 164. Type: 32-bit signed integer Example: alarm_id eq 12
alarm_name	Definition: See Properties for Controller-Based Events on page 164. Type: String Example: alarm_name eq 'Access Granted Entry Made'
area_entering_id	Definition: See Properties for Access Granted Events on page 165. Type: 32-bit signed integer Example: area_entering_id eq 3
area_entering_name	Definition: See Properties for Access Granted Events on page 165. Type: String Example: area_entering_name eq 'Default Area'
area_exiting_id	Definition: See Properties for Access Granted Events on page 165. Type: 32-bit signed integer Example: area_exiting_id eq 3
area_exiting_name	Definition: See Properties for Access Granted Events on page 165. Type: String Example: area_exiting_name eq 'default area'
asset_id	Definition: See Properties for Asset Events on page 167. Type: string Example: asset_id eq '7'
associated_text	Definition: See Common Properties for All Hardware Events on page 162. Type: String Example: associated_text eq 'secured room'
badge_extended_id	Definition: The full Federal Agency Smart Credential Number (FASC-N) or full UUID from a Personal Identity Verification (PIV)-based card or other Federal Information Processing Standard (FIPS) 201-based card. Type: String; maximum length = 64 characters Example: badge_extended_id eq '1111222233333456666666666788889'
badge_issue_code	Definition: See Properties for Access Granted Events on page 165. Type: 32-bit unsigned integer Example: badge_issue_code eq 4

Field Name	Field Description
badge_key	Definition: See Properties for Access Granted Events on page 165. Type: 64-bit signed integer Example: badge_key eq 1326
badge_key_str	Definition: See Properties for Access Granted Events on page 165. Type: String Example: badge_key_str eq '1326'
badge_id	Definition: The ID encoded on a badge. Type: 64-bit signed integer Example: badge_id eq 123456789
badge_id_str	Definition: The ID encoded on a badge. Type: String Example: badge_id_str eq '123456789'
badge_status_name	Definition: See Properties for Access Granted Events on page 165. Type: String Example: badge_status_name eq 'Active'
badge_type_name	Definition: See Properties for Access Granted Events on page 165. Type: String Example: badge_type_name eq 'Employee'
biometric_score	Definition: See Properties for Biometric Events on page 168. Type: 32-bit unsigned integer Example: biometric_score eq 13
business_event_class	Definition: The type of event that occurred. Type: String Example: business_event_class eq 'hardware_event' Note: Valid values include Acknowledgment Event, generic_event, hardware_event, hardware_status, software_event, routing_event, shutdown_thread, or text_message.
cardholder_first_name	Definition: See Properties for Access Granted Events on page 165. Type: String Example: cardholder_first_name eq 'John'
cardholder_key	Definition: See Properties for Access Granted Events on page 165. Type: 32-bit integer Example: cardholder_key eq 636719
cardholder_last_name	Definition: See Properties for Access Granted Events on page 165. Type: String Example: cardholder_last_name eq 'Smith'
controller_id	Definition: See Properties for Controller-Based Events on page 164. Type: 16-bit unsigned integer Example: controller_id eq 5 Note: The ListEntityData service can be used to request a list of controllers in the system.

Field Name	Field Description
controller_name	Definition: See Properties for Controller-Based Events on page 164. Type: String Example: controller_name eq 'access panel 13' Note: The ListEntityData service can be used to request a list of controllers in the system.
controller_time_zone_id	Definition: See Properties for Controller-Based Events on page 164. Type: 16-bit unsigned integer Example: controller_time_zone_id eq 22 Note: The ListEntityData service can be used to request a list of controllers in the system.
device_id	Definition: See Properties for Controller-Based Events on page 164. Type: 16-bit unsigned integer Example: device_id eq 123456
device_name	Definition: See Common Properties for All Hardware Events on page 162. Type: String Example: device_name eq 'reader2'
device_type	Definition: See Common Properties for All Hardware Events on page 162. Type: 8-bit signed integer Example: device_type eq 1 Note: Valid values include 2 (IVAS CCTV camera), 1 CCTV camera, and 0 (all other device types)
event_parameter	Definition: See Common Properties for All Hardware Events on page 162. Type: 32-bit unsigned integer Example: event_parameter eq 12
event_parameter_description	Definition: See Properties for Controller-Based Events on page 164. Type: string Example: event_parameter_description eq 'channel number3'
event_source_name	Definition: See Properties for Controller-Based Events on page 164. Type: string Example: event_source_name eq 'access panel 13'
event_subtype	Definition: See Common Properties for All Hardware Events on page 162. Type: 16-bit unsigned integer Example: event_subtype eq 76
event_type	Definition: See Common Properties for All Hardware Events on page 162. Type: 8-bit unsigned integer Example: event_type eq 0
intrusion_area_id	Definition: See Properties for Intrusion Events on page 169. Type: 16-bit unsigned integer Example: intrusion_area_id eq 5

Field Name	Field Description
intrusion_user_id	Definition: See Properties for Intrusion Events on page 169. Type: string Example: intrusion_user_id eq '5'
receiver_area_id	Definition: See Properties for Intrusion Events on page 169. Type: 16-bit unsigned integer Example: receiver_area_id eq 3
receiver_controller_id	Definition: See Properties for Intrusion Events on page 169. Type: 16-bit unsigned integer Example: receiver_controller_id eq 6
receiver_line_number	Definition: See Properties for Intrusion Events on page 169. Type: 16-bit unsigned integer Example: receiver_line_number eq 4
source	Definition: See Common Properties for All Hardware Events on page 162. Type: string Example: source eq 'CommServer@DPSARRO1-VM2012'
segment_id	Definition: See Common Properties for All Hardware Events on page 162. Type: 32-bit unsigned integer Example: segment_id eq 3
subdevice_id	Definition: See Properties for Controller-Based Events on page 164. Type: 16-bit unsigned integer Example: subdevice_id eq 3
transmitter_id	Definition: See Properties for Transmitter Events on page 169. Type: 32-bit signed integer Example: transmitter_id eq 4
transmitter_input_id	Definition: See Properties for Transmitter Events on page 169. Type: 32-bit signed integer Example: transmitter_input_id eq 6
video_channel	Definition: See Common Properties for All Hardware Events on page 162. Type: 32-bit signed integer Example: video_channel eq 7

Cross-Origin Resource Sharing

If you have a web application or site that makes requests against the OpenAccess API but is hosted on a different server, you must enable Cross-Origin Resource Sharing (CORS):

1. Locate the **cors.conf** file and open it for editing. This file is located in **C:\ProgramData\LnI\nginx\conf**.
2. Find the section that begins with the following line:

```
map $http_origin $cors_http_origin {
```


3. Add an entry for each HTTP origin that accesses the OpenAccess API. There are several commented out examples in the config file (remove the "#" and then modify them as needed). There is support for simple strings as well as regular expressions. Refer to http://nginx.org/en/docs/http/nginx_http_map_module.html for more details about the NGINX map directive.
4. Save the file and restart the LS Web Service service.

OpenAccess Operations From Behind a Network Proxy

If you are using OpenAccess to perform OpenAccess operations from behind a network proxy (for example, issue mobile credentials, or using a third-party provider for OnGuard authentication, or performing any other operation that requires OpenAccess to reach an external network location) and are behind a network proxy, you must make the following configuration change on the server where the LS OpenAccess service is running. Change the logon account for the LS OpenAccess service from Local System to a user whose account has the correct proxy settings configured.

Version

Every OpenAccess API call must include a version, with versions starting at "1.0" and incrementing up from there. OpenAccess uses the version to maintain backward compatibility as the API is updated.

Versions are formatted `<major>.<minor>`. Each API call is versioned independently. For example, you can call `get_event_subscriptions (version = "1.0")` and then call `authenticate (version = "2.7")`. Versions with the same `<major>` components are compatible, but might offer different optional features. For example, calling `authenticate` version 1.3 might offer a `fast=true` property. This property might be ignored by version 1.0, but the basic authenticate functionality is the same. Versions with different `<major>` components are not forward compatible. For example, an API version 1.0 call that contains API version 2.0 parameters will result in an error.

OpenAccess and Brute Force Attack Protection

OpenAccess protects users against Brute Force Attacks, where an attacker attempts to log into a user account repeatedly in an attempt to determine the password.

For internal accounts, three failed log-in attempts to the same account will lock that account from OpenAccess for 5 minutes.

Note: This Brute Force Attack protection only applies to internal accounts. Directory accounts are protected according to directory policies.

Preventing Malicious Code in OpenAccess Responses

Developers of OpenAccess clients can prevent clients from receiving strings in plain text that might contain malicious code that would be executed by a browser.

For example, the names of directories returned in the `get directories` call are strings. If someone embedded JavaScript code in a directory name, then the response to that call would contain that

JavaScript code, and would be executed by the browser. For example, the **Name** property below contains JavaScript code:

```
{
  "property_value_map": {
    "ID": "id9",
    "Name": "<script>alert(1);</script>"
    "directory_type": 0
  }
}
```

To prevent malicious code from running on the client browser, the OpenAccess server encodes non-alphanumeric characters in text strings, eliminating raw HTML elements. For example, encoding the code sample shown above would result in the client receiving something like this instead:

```
{
  "property_value_map": {
    "ID": "id9",
    "Name": "&lt;script&gt;alert&#x28;1&#x29;&#x3B;&lt;&#x2F;script&gt;",
    "directory_type": 0
  }
}
```

This encoding would prevent the script from being executed by the browser.

Note: Only strings from free-format fields containing user-entered text are encoded. Other string properties are not encoded.

But this also means that the client would need to decode the encoded characters in order to display the string correctly. For example, a directory like **<internal>** would be encoded as **<internal>**; even though this is not an HTML tag, and it would be displayed that way unless the client decodes it.

The mechanisms being used to encode the characters are known as *HTML Entity Encoding* and *HTML Attribute Encoding*, described here and elsewhere online:

<https://www.owasp.org/index.php/>

[XSS_\(Cross_Site_Scripting\)_Prevention_Cheat_Sheet#Output_Encoding_Rules_Summary](#).

For OpenAccess clients to take advantage of this encoding, they would have to use the updated versions of the methods listed below, and they would have to implement the decoding logic for any returned string properties:

Method	Minimum version that supports string encoding
get directories (see get directories on page 60)	1.1
get instances (see get instances on page 80)	1.4
get event_subscriptions (see get event_subscriptions on page 65)	1.1

Method	Minimum version that supports string encoding
add event_subscriptions (see add event_subscriptions on page 69)	1.1
modify event_subscriptions with id (see modify event_subscriptions with id on page 71)	1.1
get logged_in_user (see get logged_in_user on page 110)	1.1
get managed_access_levels (see get user managed_access_levels on page 111)	1.1
get user (see get user on page 113)	1.1
get editable_segments (see get editable_segments on page 116)	1.1
get cardholders (see get cardholders on page 93)	1.1
get user_segments (see get user segments on page 117)	1.1
get authorization warning settings (see get authorization warning settings on page 130)	1.1
get visit settings (see get visit settings on page 140)	1.1
get badge_mobile_devices (see get badge mobile_devices on page 85)	1.1
add badge_issue_mobile_credential (see add badge issue_mobile_credential on page 86)	1.1
get print_request (see get badge print_request on page 82)	1.1
add print_request (see add badge print_request on page 83)	1.1
get directory_accounts (see get directory_accounts on page 123)	1.1
get directory_accounts_matching_cardholders (see get directory_accounts_matching_cardholders on page 124)	1.1
get user_preferences (see get user preferences on page 119)	1.1

Method	Minimum version that supports string encoding
put user_preferences (see put user preferences on page 120)	1.1
get video_recorders (see get video_recorders on page 104)	1.2
get auth_data (see get video_recorder auth_data on page 106)	1.1
get logged_events (see get logged_events on page 73)	1.1
get badge_printers (see get badge printers on page 87)	1.1
get console layouts (see get console layouts on page 129)	1.1
put console layouts (see put console layouts on page 129)	1.1

This section provides details about the LS OpenAccess service's Application Programming Interface (API).

The REST proxy that is part of the LS OpenAccess service allows you to create a client against a REST API to OnGuard through NGINX as the web service which abstracts the AMQP language. The LS Web Service is the service hosting NGINX. Use the REST Request URL and body contents described below for each API call.

Notes: The errors you might receive in the response header are very helpful when creating a client application that uses OpenAccess. Also, any request taking longer than 60 seconds to fulfill results in a timeout error. For more information, refer to [Error Messages](#) on page 289.

You will receive an HTTP 200 code whenever an API call executes successfully.

API calls are handled asynchronously. It is the responsibility of the client to handle synchronization as needed.

When creating Body content, this sample shows when to use quotation marks:

```
{
  "some_string": "I am a string",
  "some_number": 1000,
  "some_bool": false
}
```

Task queuing: dealing with long running requests

Some requests might take a long time, especially requests that access external systems, such as Active Directory. Standard OpenAccess requests will time out after 30 seconds if the HTTP request doesn't time out sooner, depending on the client. Any request that you expect to run long can be queued as a task by adding a `queue` property to the request, set to `true`. For example:

```
GET /directory_accounts_matching_cardholders?directory_id=id1
&cardholder_ids=[1,2,3,4,5,6,7,8,9,10]
&filter=displayname has 'firstname' and displayname has 'lastname'
&queue=true
```

`&version=1.0`

When a request is queued in this way, OpenAccess will queue a task for execution and return a 202 (Accepted) HTTP status code and a response identical to `GET /queue/{id}`. For example:

```
{
  "id": "5c4b7890-ee73-4199-b3d3-366003eb8ca1",
  "status": "pending",
  "version": "1.0"
}
```

The `id` property indicates the ID of the queued task, which can be used to check the status of the task:

`GET /queue/5c4b7890-ee73-4199-b3d3-366003eb8ca1?version=1.0`

When the task is complete, the response will include the response to the queued request:

```
{
  "id": "5c4b7890-ee73-4199-b3d3-366003eb8ca1",
  "response": {
    ...
  },
  "status": "complete",
  "version": "1.0"
}
```

The response can be retrieved any number of times until the task is deleted. A completed task can be deleted with `DELETE /queue/{id}` or it will be deleted automatically after 1 hour.

Even though you can queue any request, it is only recommended when a request is expected to run long, like `GET /directory_accounts` and `GET /directory_accounts_matching_cardholders`.

Required Parameters for OpenAccess Requests

Name	Type	Location	Required	Description
Session-Token	string	header	yes (for token-based authentication)	The authentication token for the current user session.
OASessionID	string	header	yes (for cookie-based authentication)	OR Client session cookie containing the authentication token. For more information, refer to Authentication on page 32.
Application-Id	string	header	yes	A unique Application-Id is provided by LeneIS2 OnGuard Technical Support. For more information, refer to License for OpenAccess on page 29.
do_not_reset_inactivity_timer	boolean	query	no	Default = false. If set to true, the OpenAccess call does not reset the inactivity timer to zero. If this parameter is not provided, it is assumed to be false and the call will reset the inactivity timer to zero. This parameter is available to all OpenAccess calls except the following: • get version • get directories • get identity_provider_url • get keepalive • post authentication • delete authentication
queue	boolean	query	no	Queues the request as a task, and returns a response identical to GET /queue/{id}. Defaults to false if not provided.
version	string	query	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

Note: The following methods are exceptions and do not require an authentication token:

- get version
- get directories
- add authentication

General OpenAccess API Calls

get version

Used to retrieve the OnGuard product name and version information.

REST Request URL: GET /api/access/onguard/openaccess/
version?version=value

get version response

Name	Type	Required	Description
product_name	string	yes	A string representing the product name and major version (stored in the Windows registry as "InstalledProductName"). For example: OnGuard #.#.
product_version	string	yes	A string representing the detailed version information (stored in the Windows registry as "ProductVersion"). For example: (#.#.###).
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get keepalive

Used to prevent idle session timeout.

REST Request URL: GET /api/access/onguard/openaccess/
keepalive?version=value

get feature_availability

Used to check if an OnGuard license feature is available.

REST Request URL: GET /api/access/onguard/openaccess/
feature_availability?version=value

get feature_availability response

Name	Type	Required	Description
is_available	boolean	yes	Indicates if this license feature is available.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get queue

Gets the queued tasks created by the user. This method is only intended to check the status of multiple tasks. Request a specific task to get the response. Users can only view their own queued tasks.

Note: This call does not return queues associated with **add authentication** requests.

REST Request URL: GET /api/access/onguard/openaccess/queue?version=value

get queue response

Name	Type	Required	Description
item_list	list	yes	A list of queued tasks. Each task in the list is provided with its unique ID and status.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get queue/{id}

Gets the queued task with the given ID, which includes the response when the task is complete. Users can only get their own queued tasks except for the **add authentication** queue task. This is because OpenAccess does not know which user created the **add authentication** queue tasks before that user has logged in.

REST Request URL: GET /api/access/onguard/openaccess/queue/{id}?version=value

get queue/{id}

Name	Type	Required	Description
id	string	yes	The ID of the task to return.

get queue/{id} response

Name	Type	Required	Description
id	string	yes	The ID of the task to return.
response	map	yes	The response of a queued task.
status	string	yes	The status of the queued task.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

delete queue/{id}

Deletes the queued task with the given ID. All queued tasks will be deleted automatically after 1 hour if not manually deleted. Only complete tasks can be deleted, and users can only delete their own queued tasks except for the **add authentication** queue task. This is because OpenAccess does not know which user created the **add authentication** queue tasks before that user has logged in.

REST Request URL: DELETE /api/access/onguard/openaccess/queue/{id}?version=value

delete queue/{id}

Name	Type	Required	Description
id	string	yes	The ID of the task to return.

delete queue/{id} response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

add partner_values

Used by OnGuard software partners.

REST Request URL: POST /api/access/onguard/openaccess/partner_values?version=value

add partner_values

Name	Type	Required	Description
partner_value_1	int32	no	First partner value.
partner_value_2	int32	no	Second partner value.
partner_value_3	int32	no	Third partner value.
partner_value_4	int32	no	Fourth partner value.
partner_value_5	int32	no	Fifth partner value.

add partner_values response

Name	Type	Required	Description
result	boolean	yes	Result of the operation.

add partner_values response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

modify partner_values

Used by OnGuard software partners.

REST Request URL: PUT /api/access/onguard/openaccess/partner_values?version=value

modify partner_values

Name	Type	Required	Description
partner_value_1	int32	no	First partner value.
partner_value_2	int32	no	Second partner value.
partner_value_3	int32	no	Third partner value.
partner_value_4	int32	no	Fourth partner value.
partner_value_5	int32	no	Fifth partner value.

modify partner_values response

Name	Type	Required	Description
result	boolean	yes	Result of the operation.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get susp_expiration

Retrieves the OnGuard SUSP expiration information.

REST Request URL: GET /api/access/onguard/openaccess/susp_expiration?version=value

get susp_expiration

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get susp_expiration response

Name	Type	Required	Description
susp_expiration	datetime (string)	no	The SUSP expiration date. This date is maintained by LeneIS2 and is made available to OnGuard during license activation.
message_status	string	yes	Provides the status of the SUSP expiration. Possible values are ACTIVE, EXPIRED, GRACE_PERIOD, INACTIVE, DISABLED, LONG_NOTICE, SHORT_NOTICE, MEDIUM_NOTICE. The values for the NOTICE statuses are available in the OnGuard license file.
SUSP_to_expire	int32	yes	Provides the number of days remaining before the SUSP expires. This number is calculated based on the formula: today_date – susp_expiration.
days_to_expire	int32	yes	Provides the number of days remaining before the SUSP expires. This number is calculated based on the formula: today_date – susp_expiration + grace_period. The grace_period value is provided in OnGuard license file.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

Login and Logout

get directories

Returns a list of directories configured within the OnGuard software. If using an internal account for authentication, you can call **add authentication** without specifying a directory ID. It is generally called prior to **add authentication** to get the user's directory ID.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/directories?version=value

get directories response

Name	Type	Required	Description
total_items	int32	yes	The total number of directories in the filter result.
item_list	list	no	A list of items returned if directories exist. If present, each item consists of a property_value_map.
property_value_map	map	yes	A map of directory attributes: - ID: Internal directory ID - Name: Name of the directory - directory_type: Directory type. Possible values: -1: Internal Directory 0: LDAP 1: Microsoft Active Directory 2: Microsoft Windows NT 4 Domain 3: Windows Local Accounts 4: OpenID Connect
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

add authentication

IMPORTANT: Version 2.0 of this call was introduced in OnGuard 7.5.

Authenticates a user with the LS OpenAccess service.

Notes: The add authentication call returns a token to be used in all subsequent authorized calls. For information about how OpenAccess protects against Brute Force Attacks, refer to [OpenAccess and Brute Force Attack Protection](#) on page 17.

The response to the **add authentication** call issues a Set Cookie response header that establishes the HTTPOnly cookie. For more information, refer to [Authentication](#) on page 32.

Every **add authentication** call should have a matching **delete authentication** call. Not calling **delete authentication** consumes resources unnecessarily over time.

It is also recommended that you keep a session token available (that is, do not **delete authentication**) until you no longer need to execute commands for at least several minutes. In other words, avoid rapidly adding and then deleting authentication within seconds. This can happen if you have a snippet of code that executes very frequently, and that code adds and then deletes authentication. Consider keeping the authentication token and executing the snippet frequently, and then deleting authentication only if the code won't execute again for another 1 or 2 minutes.

REST Request URL: POST /api/access/onguard/openaccess/authentication?version=value

REST Request Body Contents:

Note: The **oidc_token** name:value pair was introduced in Version 2.0 of the **add authentication** call.

```
{
  "user_name": "value",
  "password": "value",
  "directory_id": "value",
  "oidc_token": "value"
}
```

add authentication

Name	Type	Required	Version	Description
user_name	string	Required for Version 1.0. For Version 2.0 and later, not required if using oidc_token .	1.0 and later	The user's user name, in plain text.
password	string	Required for Version 1.0. For Version 2.0 and later, not required if using oidc_token .	1.0 and later	The user's password, in plain text.
directory_id	string	yes	1.0 and later	The user's directory ID, as a string. To get a list of available directory IDs, refer to get directories on page 60.
oidc_token	string	Not available for Version 1.0. For Version 2.0 and later, you must provide either the user_name and password or the oidc_token.	2.0 and later	An OpenID Connect access token. Introduced in Version 2.0 of the add authentication call.

add authentication response

Name	Type	Required	Version	Description
session_token	string	yes	1.0 and later	The authentication token, which is returned with a successful response.
password_expiration_time	datetime (string)	no	1.0 and later	This represents the time when the user password will expire, in UTC time. The client should use this information to change password as needed. For example: 2016-10-07T22:05:02+00:00. This only exists if the user logged in with internal account and the password expiration policy is enabled.
token_expiration_time	datetime (string)	yes	1.0 and later	This represents the time when the authenticated token will expire, in UTC time. The client should use this information to re-authenticate as needed. For example: 2016-10-07T22:05:02+00:00
version	string	yes	1.0 and later	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version": "1.0". For more information, refer to Version on page 49.
warning	string	no	1.0 and later	If present, contains additional information that might be useful to the user even though the authentication was successful. For example, password expiration information would be contained here. For more information, refer to Warning List on page 292.

delete authentication

Logs a user out of the LS OpenAccess service by invalidating the token and removing the user from its internal map.

Notes: Every **add authentication** call should have a matching **delete authentication** call. Not calling **delete authentication** consumes resources unnecessarily over time.

It is also recommended that you keep a session token available (that is, do not **delete authentication**) until you no longer need to execute commands for at least several minutes. In other words, avoid rapidly adding and then deleting authentication within seconds. This can happen if you have a snippet of code that executes very frequently,

and that code adds and then deletes authentication. Consider keeping the authentication token and executing the snippet frequently, and then deleting authentication only if the code won't execute again for another 1 or 2 minutes.

REST Request URL: DELETE /api/access/onguard/openaccess/authentication?version=value

get session

Retrieves session data for a session token.

REST Request URL: GET /api/access/onguard/openaccess/session?version=value

get session response

Name	Type	Required	Description
session_token	string	yes	The authentication token, which is returned with a successful response.
token_expiration_time	datetime (string)	yes	The time the token will expire, in UTC time. For example: 2016-10-07T22:05:02+00:00
token_start_time	datetime (string)	yes	The time the token was first issued, in UTC time. For example: 2016-10-07T22:05:02+00:00
user_id	string	yes	The user's ID, as a string.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get identity_provider_url

Gets the URL that users authenticating with a third-party OpenID Connect provider should be directed to in their browsers.

REST Request URL: GET /api/access/onguard/openaccess/identity_provider_url?version=value&directory_id=value&redirect_url=value&response_mode=value

get identity_provider_url

Name	Type	Required	Description
directory_id	string	yes	The directory ID of the selected identity provider. Must refer to an OpenId Connect directory.
redirect_url	string	yes	The URL to which the identity provider should send its response.

get identity_provider_url

Name	Type	Required	Description
response_mode	string	yes	The mode the identity provider should use to respond. Valid values are "form_post" and "fragment". "form_post" causes the identity provider to respond with an HTTP POST to the redirect_url, with the content in the message body. "fragment" will contain the response in the redirect URL.

get identity_provider_url response

Name	Type	Required	Description
url	string	yes	The URL to send the user to for authentication.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

Receive Events**get event_subscriptions**

Retrieves event subscriptions, and details about the subscriptions. Non-System Account (SA) users can only retrieve their own event subscriptions.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/event_subscriptions?version=value

get event_subscriptions

Name	Type	Required	Description
page_number	int32	no	The page number to be returned when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.

get event_subscriptions

Name	Type	Required	Description
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
order_by	string	no	A field or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by created_date. Fields must be valid properties of the requested object type. For more information, refer to Additional order_by Details on page 66.

Additional order_by Details

When using order_by to specify that a field is sorted in descending order, add a minus character (“-”) in front of the field name. Without the minus character, the field will be sorted in ascending order. Also, different fields can be sorted differently. For example, to sort created_date in descending order and message_broker_hostname in ascending order:

```
GET /api/access/onguard/openaccess/event_subscriptions?
page_number=1&page_size=20&
order_by=-created_date,message_broker_hostname&version=value
```

get event_subscriptions response

Name	Type	Required	Description
item_list	list	yes	A list of items returned, if instances exist. If a valid order_by parameter was provided in the request, then the list of items is sorted accordingly. If present, each item consists of the properties of the event subscription.
id	int32	yes	The ID of the event subscription to retrieve.
user_id	string	yes	The ID of the user who owns the subscription, as a string.
page_number	int32	no	The page number of the requested subset (page) of instances returned. Same as corresponding input parameter, or the default value if not provided as input.

get event_subscriptions response

Name	Type	Required	Description
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total existing number of instances of the object being requested.
description	string	yes	A description of the subscription.
filter	string	yes	This optional parameter filters the events that are received. If no filter is specified, all events are forwarded to the subscriber. For more information refer to Searching for Objects on page 39 and Using Event Filters with Subscriptions on page 43.
is_durable	boolean	yes	Indicates if this is a durable subscription. Default is "false". For more information, refer to Durable vs. Transient Event Subscribers on page 43.
message_broker_hostname	string	yes	The hostname of the message broker where the events are published.
message_broker_port	int32	yes	The port of the message broker where the events are published.
requires_secure_connection	boolean	yes	Indicates if an SSL connection should be opened with the message broker where the events are published.
exchange_name	string	yes	The exchange name on the message broker where events will be published.
binding_key	string	yes	The unique binding key with which events will be published on the exchange.
created_date	datetime (string)	yes	The date and time when the subscription was created.
last_updated_date	datetime (string)	yes	The date and time when the subscription was last updated.
count	int32	yes	The total number of records in the filter result.

get event_subscriptions response

Name	Type	Required	Description
queue_name	string	no	The name of the durable queue on the message broker where events will be published for durable subscriptions. Only included in the response when is_durable is true.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get event_subscriptions with id

Retrieves a specific event subscription. Non-System Account (SA) users can only retrieve their own event subscriptions.

REST Request URL: GET /api/access/onguard/openaccess/event_subscriptions/{id}?version=value

get event_subscriptions with id

Name	Type	Required	Description
id	int32	yes	The ID of the event subscription to retrieve.

get event_subscriptions with id response

Name	Type	Required	Description
id	int32	yes	The unique subscription ID.
user_id	string	yes	The ID of the user who owns the subscription, as a string.
description	string	yes	A description of the subscription.
filter	string	yes	This optional parameter filters the events that are received. If no filter is specified, all events are forwarded to the subscriber. For more information refer to Searching for Objects on page 39 and Using Event Filters with Subscriptions on page 43
is_durable	boolean	yes	Indicates if this is a durable subscription. Default is "false". For more information, refer to Durable vs. Transient Event Subscribers on page 43.
message_broker_hostname	string	yes	The hostname of the message broker where the events are published.
message_broker_port	int32	yes	The port of the message broker where the events are published.

get event_subscriptions with id response

Name	Type	Required	Description
requires_secure_connection	boolean	yes	Indicates if an SSL connection should be opened with the message broker where the events are published.
exchange_name	string	yes	The exchange name on the message broker where events will be published.
binding_key	string	yes	The unique binding key with which events will be published on the exchange.
created_date	datetime (string)	yes	The date and time when the subscription was created.
last_updated_date	datetime (string)	yes	The date and time when the subscription was last updated.
queue_name	string	no	The name of the durable queue on the message broker where events will be published for durable subscriptions. Only included in the response when is_durable is true.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

add event_subscriptions

Adds an event subscription.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: POST /api/access/onguard/openaccess/event_subscriptions?version=value

add event_subscriptions

Name	Type	Required	Description
description	string	no	A description of the subscription.
filter	string	no	This optional parameter filters the events that are received. If no filter is specified, all events are forwarded to the subscriber. For more information refer to Searching for Objects on page 39 and Using Event Filters with Subscriptions on page 43

add event_subscriptions

Name	Type	Required	Description
is_durable	boolean	no	Indicates if this is a durable subscription. Default is "false". For more information, refer to Durable vs. Transient Event Subscribers on page 43.

add event_subscriptions response

Name	Type	Required	Description
id	int32	yes	The unique subscription ID.
user_id	string	yes	The ID of the user who owns the subscription, as a string.
description	string	yes	A description of the subscription.
filter	string	yes	This optional parameter filters the events that are received. If no filter is specified, all events are forwarded to the subscriber. For more information refer to Searching for Objects on page 39 and Using Event Filters with Subscriptions on page 43
is_durable	boolean	yes	Indicates if this is a durable subscription. Default is "false". For more information, refer to Durable vs. Transient Event Subscribers on page 43.
message_broker_hostname	string	yes	The hostname of the message broker where the events are published.
message_broker_port	int32	yes	The port of the message broker where the events are published.
requires_secure_connection	boolean	yes	Indicates if an SSL connection should be opened with the message broker where the events are published.
exchange_name	string	yes	The exchange name on the message broker where events will be published.
binding_key	string	yes	The unique binding key with which events will be published on the exchange.
created_date	datetime (string)	yes	The date and time when the subscription was created.
last_updated_date	datetime (string)	yes	The date and time when the subscription was last updated.
queue_name	string	no	The name of the durable queue on the message broker where events will be published for durable subscriptions. Only included in the response when is_durable is true.

add event_subscriptions response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

modify event_subscriptions with id

Modifies an event subscription. Users other than the System Account (SA) user or SA delegate users can only modify their own event subscriptions. The SA user and SA delegate users can modify all event subscriptions.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: PUT /api/access/onguard/openaccess/event_subscriptions/{id}?version=value

modify event_subscriptions with id

Name	Type	Required	Description
id	int32	yes	The unique subscription ID.
description	string	no	A description of the subscription.
filter	string	no	This optional parameter filters the events that are received. If no filter is specified, all events are forwarded to the subscriber. For more information refer to Searching for Objects on page 39 and Using Event Filters with Subscriptions on page 43

modify event_subscriptions with id response

Name	Type	Required	Description
id	int32	yes	The unique subscription ID.
user_id	string	yes	The ID of the user who owns the subscription, as a string.
description	string	yes	A description of the subscription.
filter	string	yes	This optional parameter filters the events that are received. If no filter is specified, all events are forwarded to the subscriber. For more information refer to Searching for Objects on page 39 and Using Event Filters with Subscriptions on page 43

modify event_subscriptions with id response

Name	Type	Required	Description
is_durable	boolean	yes	Indicates if this is a durable subscription. Default is "false". For more information, refer to Durable vs. Transient Event Subscribers on page 43.
message_broker_hostname	string	yes	The hostname of the message broker where the events are published.
message_broker_port	int32	yes	The port of the message broker where the events are published.
requires_secure_connection	boolean	yes	Indicates if an SSL connection should be opened with the message broker where the events are published.
exchange_name	string	yes	The exchange name on the message broker where events will be published.
binding_key	string	yes	The unique binding key with which events will be published on the exchange.
created_date	datetime (string)	yes	The date and time when the subscription was created.
last_updated_date	datetime (string)	yes	The date and time when the subscription was last updated.
queue_name	string	no	The name of the durable queue on the message broker where events will be published for durable subscriptions. Only included in the response when is_durable is true.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

delete event_subscriptions with id

Deletes an event subscription. Users other than the System Account (SA) user and SA delegate users can only delete their own event subscriptions. The SA user and SA delegate users can delete all event subscriptions.

REST Request URL: DELETE /api/access/onguard/openaccess/event_subscriptions/{id}?version=value

delete event_subscriptions with id

Name	Type	Required	Description
id	int32	yes	The unique subscription ID.

Manage Instances

get logged_events

Retrieves a page of logged events from the OnGuard database.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/
logged_events?version=value

get logged_events

Name	Type	Required	Description
filter	string	yes	The clause text used to count only those instances that match a given attribute. For example, <code>firstname="Lisa"</code>. Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an <code>InvalidQuery</code> error. OpenAccess does not support filtering with the following properties: • EVENT_SOURCE_NAME • CARDHOLDER_FIRST_NAME • CARDHOLDER_LAST_NAME • DEVICE_NAME • SUBDEVICE_NAME • ACCESS_RESULT • CARDHOLDER_ENTERED • DURESS • ALARM_ACK_BLUE_CHANNEL • ALARM_ACK_GREEN_CHANNEL • ALARM_ACK_RED_CHANNEL • ALARM_BLUE_CHANNEL • ALARM_GREEN_CHANNEL • ALARM_RED_CHANNEL For more information refer to Searching for Objects on page 39.
page_number	int32	no	The page number to return when a subset (page) of instances is requested. Used in conjunction with <code>page_size</code>. Defaults to the first page (1) if not provided, and if provided, must be numeric.

get logged_events

Name	Type	Required	Description
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
order_by	string	no	A field or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by created_date. Fields must be valid properties of the requested object type. For more information, refer to Additional order_by Details on page 66.

get logged_events response

Name	Type	Required	Description
alarm_ack_blue_channel	int32	yes	The blue component of the RGB color for the alarm after it is acknowledged (0 to 255).
alarm_ack_green_channel	int32	yes	The green component of the RGB color for the alarm after it is acknowledged (0 to 255).
alarm_ack_red_channel	int32	yes	The red component of the RGB color for the alarm after it is acknowledged (0 to 255).
alarm_blue_channel	int32	yes	The blue component of the RGB color for the alarm (0 to 255).
alarm_green_channel	int32	yes	The green component of the RGB color for the alarm (0 to 255).
alarm_red_channel	int32	yes	The red component of the RGB color for the alarm (0 to 255).
alarm_priority	int32	yes	Alarm priority (0 to 255).
access_result	int32	yes	The level of access that was granted, resulting from reading the card. 0: Other 1: Unknown 2: Granted 3: Denied 4: Not Applicable
asset_id	int32	yes	Asset (where available) that caused the event.

get logged_events response

Name	Type	Required	Description
badge_extended_id	string	yes	Extended identifier of the card that caused the event.
badge_id	int64	yes	Card (where available) that caused the event.
badge_id_str	string	yes	A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the ID property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off. Note: This property is only returned when get instances is called with Version 1.2 or later.
badge_issue_code	int32	yes	Issue code of the card that caused the event.
cardholder_entered	boolean	yes	True if entry was made by the cardholder.
cardholder_first_name	string	yes	The first name of the cardholder.
cardholder_key	int32	yes	Internal identifier of the person who is assigned the badge at the time of the access event. See Lnl_Person.ID.
cardholder_last_name	string	yes	The last name of the cardholder.
controller_id	int32	yes	Controller at which the event occurred. Key field. Reference to Lnl_Panel ID.
controller_name	string	yes	The name of the controller at which the event occurred.
count	int32	yes	The number of logged events returned.
description	string	yes	Description of the event.
device_id	int32	yes	Device at which the event occurred (for example, Lnl_Reader, Lnl_AlarmPanel, etc.).
duress	boolean	yes	True if this card access indicates an under duress/emergency state.
event_type	int32	yes	Event type (for example, Duress, System, etc.). Corresponds to Lnl_EventSubtypeDefinition.TypeID and LnlEventType.ID.
event_source_name	string	yes	The name of the device at which the event occurred.

get logged_events response

Name	Type	Required	Description
event_subtype	int32	yes	Event sub-type (for example, Granted, Door Forced Open, etc.). Corresponds to Lnl_EventSubtypeDefinition.SubTypeID.
event_text	string	yes	Text associated with the event.
must_acknowledge	boolean	yes	If true, the alarm must be acknowledged before it is cleared.
must_mark_in_progress	boolean	yes	If true, the alarm must be marked in progress before it is cleared.
page_number	int32	no	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
serial_number	int32	yes	Serial number of the event. Key field.
segment_id	int32	yes	Segment where the event occurred.
subdevice_id	int32	yes	Secondary device at which the event occurred (for example, Lnl_Input).
timestamp	string	yes	Time when the event occurred.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total existing number of instances of the object being requested.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get types

Retrieves a list of types available via the LS OpenAccess service.

REST Request URL: GET /api/access/onguard/openaccess/types?version=value

get types response

Name	Type	Required	Description
types	map	yes	A map of type names to parent type names. All types ultimately derive from "LnI_Element", except for "LnI_Element" itself, which will have an empty string as its parent type name.
total_items	int32	yes	The total number of types that are exposed to the user and returned in the types map.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get type

Retrieves information for a specific type.

REST Request URL: GET /api/access/onguard/openaccess/type?type_name=value&version=value

get type

Name	Type	Required	Description
type_name	string	yes	The name of the type for which to retrieve information.

get type response

Name	Type	Required	Description
type_name	string	yes	The type name.
properties	list	yes	The properties of the type. See get type response: properties list on page 78.
access	string	yes	Indicates whether the type is view only, read only, or editable. Possible return values: View: Indicates the user cannot change the type. Read: Indicates the type can be added or deleted. Edit: Indicates the type can be added, modified, or deleted.

get type response

Name	Type	Required	Description
methods	list	yes	The methods available for this type. See get type response: methods map on page 79.
display_name	string	no	When provided via the object name of a User Defined Field (UDF) in FormsDesigner, the display_name attribute is the user-friendly name of the item. For more information, refer to Features and Limitations on page 41. Also refer to the "Field Properties Folder – General Settings Form" section in the <i>FormsDesigner User Guide</i> .
display_groups	list	no	Includes a list of user-defined and name attribute that follows the tab order specified in FormsDesigner.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get type response: properties list

Name	Type	Required	Description
name	string	yes	The name of the property.
type	string	yes	The type of the property.
access	string	yes	Indicates whether the property is view only, read only, or editable. Possible return values: View: Indicates the user cannot change the property. Read: Indicates the property value can be specified during add only. Edit: Indicates the property value can be changed at any time.
is_key	boolean	yes	Indicates if the property is a key property.
is_required	boolean	yes	Indicates if the property is required.
max_length	int32	only string properties and some binary properties	The maximum length of the string or binary property.
default_value	string	no	A default value of the property.
possible_values	map	no	A map of numerical keys to string values. For example: (0, "Zero"; 1, "One")

get type response: properties list

Name	Type	Required	Description
display_name	string	no	When provided via the object name of a User Defined Field (UDF) in FormsDesigner, the display_name attribute is the user-friendly name of the item. For more information, refer to Features and Limitations on page 41. Also refer to the “Field Properties Folder – General Settings Form” section in the <i>FormsDesigner User Guide</i> .
display_attributes	map	no	Displays the following attributes that describe the behavior of user-defined fields: is_password: If enabled, the password is masked as it is entered into a password field. is_searchable: If enabled, the user can search on this property. Note: You cannot search on encrypted text or password fields. permission: Indicates the field's permissions. For more information, refer to Data Classes on page 181. template: Specifies a template used to ensure the integrity of data entered into the field.

get type response: methods map

Name	Type	Required	Description
name	string	yes	The name of the method.
in_parameters	map	no	The parameters expected to be sent along with the execution request of the method. This can be empty. See get type response: method parameter map on page 79.
out_parameters	map	no	The parameters that represent the result of the method execution. This can be empty.

get type response: method parameter map

Name	Type	Required	Description
name	string	yes	The name of the parameter.
type	string	yes	The type of the parameter.

get count

Used to retrieve the number of existing instances of a given object type.

REST Request URL: GET /api/access/onguard/openaccess/count?type_name=value&filter=value&version=value

get count

Name	Type	Required	Description
type_name	string	yes	A string representing the name of the type for which instances will be counted. For example, Lnl_Cardholder.
filter	string	no	The clause text used to count only those instances that match a given attribute. For example, <code>firstname="Lisa"</code> . Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an InvalidQuery error. For more information refer to Searching for Objects on page 39.

get count response

Name	Type	Required	Description
total_items	int32	yes	The total number of instances of the object type being requested.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get instances

Retrieves instances of a particular type based on the client-supplied filter.

When using this call for types with binary properties (Lnl_MultimediaObject), the binary data is returned base64 encoded.

Notes: You must use Version 1.3 or later of this method if you need support for *BadgeID_str*, *badge_id_str*, *badge_key_str*, *CardNumber_str*, or *AssignedBadgeID_str*.

Version 1.4 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/instances?page_number=value&page_size=value&order_by=value&type_name=value&filter=value&version=value

Note: Page_number and page_size are optional. The default page_number = 1, and the default page_size = 20. Paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100. To preserve system

performance such as when using multimedia objects, you might need to choose a page size smaller than 100.

get instances

Name	Type	Required	Description
type_name	string	yes	The name of the type being added. For example, Lnl_Cardholder.
filter	string	no	The filter used to retrieve instances. For example, Lastname = "Smith" and Firstname = "Lisa". Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an InvalidQuery error. For more information refer to Searching for Objects on page 39.
page_number	int32	no	The page number to be returned when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
order_by	string	no	A field or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by key field(s). Fields must be valid properties of the requested object type. For more information, refer to Additional order_by Details on page 81.

Additional order_by Details

For Lnl_AlarmDefinition, you could pass **Priority**, **Description** (or **Priority , Description** because spaces are ignored). Results would be ordered by Priority (ALARM.ALPRIORITY) followed by Description (ALARM.ALDESCR). In general, filtering behaves the same as queries behave in System Administration.

Use care when selecting which values you specify with your order_by, as the request might take too long to fulfill. This is a problem if you order_by very large classes, such as Lnl_LoggedEvent ([Lnl_LoggedEvent](#) on page 222), which might result in a timeout error. For more information, refer to [Error Messages](#) on page 289.

In general, using the default order_by works well because key fields are optimized for performance through the use of an index. If you order_by fields that are not indexed and are large classes, performance might suffer.

When using order_by to specify that a field is sorted in descending order, add a minus character (“-”) in front of the field name. Without the minus character, the field will be sorted in ascending order.

Also, different fields can be sorted differently. For example, to sort lastname in descending order and firstname in ascending order:

```
GET /api/access/onguard/openaccess/
instances?page_number=1&page_size=20&
order_by=-lastname,firstname&type_name=Ln1_Cardholder&version=value
```

Note: The **order_by** parameter is not supported for association data classes.

get instances response

Name	Type	Required	Description
page_number	int32	no	The page number of the requested subset (page) of instances returned. Same as corresponding input parameter, or the default value if not provided as input.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total existing number of instances of the object being requested.
count	int32	yes	The total number of records in the filter result.
item_list	list	yes	A list of items returned if instances exist. If a valid order_by parameter was provided in the request, then the list of items is sorted accordingly. If present, each item consists of type_name and property_map.
type_name	string	yes	The name of the type being returned.
property_value_map	map	yes	This is a map where the key is property name and the value is the actual property value.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get badge print_request

Returns the status of the request to print a badge.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/badge/{badge_print_request_id}/print_request?version=value

get badge print_request

Name	Type	Required	Description
badge_print_request_id	string	yes	Represents a GUID that is system generated. Each print request has a unique id.

get badge print_request response

Name	Type	Required	Description
badgekey	int32	yes	The unique identifier of the badge assigned to a person. For more information, refer to LnI_Badge on page 197.
badge_print_request_id	string	yes	Represents a GUID that is system generated. Each print request has a unique id.
message	string	yes	Only applies to error messages returned from the badge printing service.
status	string	yes	Internal system codes indicating the status of the badge printing request as it is processed by the print service. Possible statuses: • Pending • Received • Waiting_for_printer_access • Printing • Completed • Completed_skipped_errors • Aborted_fatal_error • Canceled by user
submitted_at	datetime	yes	Represents when the request was sent to the print service.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version": "1.0". For more information, refer to Version on page 49.

add badge print_request

Submits a print request to print the badge.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: POST /api/access/onguard/openaccess/badge/{badgekey}/print_request?version=value

add badge print_request

Name	Type	Required	Description
badgekey	int32	yes	The unique identifier of the badge assigned to a person. For more information, refer to Lnl_Badge on page 197.
print-request	JSON	no	Message body, in JSON format.
workstation	string	no	The workstation corresponding to the printers returned from the GET /badge_printers API call. For more information, refer to get badge printers on page 87.

add badge print_request response

Name	Type	Required	Description
badgekey	int32	yes	The unique identifier of the badge assigned to a person. For more information, refer to Lnl_Badge on page 197.
badge_print_request_id	string	yes	Represents a GUID that is system generated. Each print request has a unique id.
message	string	yes	Only applies to error messages returned from the badge printing service.
status	string	yes	Internal system codes indicating the status of the badge printing request as it is processed by the print service. Possible statuses: • Pending • Received • Waiting_for_printer_access • Printing • Completed • Completed_skipped_errors • Aborted_fatal_error • Canceled by user
submitted_at	datetime	yes	Represents when the request was sent to the print service.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

delete badge print_request

Deletes a print request to print the badge that hasn't completed.

REST Request URL: DELETE /api/access/onguard/openaccess/badge/{badge_print_request_id}/print_request?version=value

delete badge print_request

Name	Type	Required	Description
badge_print_request_id	string	yes	Represents a GUID that is system generated. Each print request has a unique id.
request body	string	no	Pass an empty request body.

delete badge print_request response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get badge mobile_devices

This method retrieves a list of mobile devices for the person associated with a badge. The list is provided by the mobile credentialing services associated with the badge type of this badge.

Notes: If you are using OpenAccess to issue mobile badges and are behind a network proxy, an error might occur when issuing or managing mobile credentials. To resolve this error, on the server where the LS OpenAccess service is running, change the logon account for the LS OpenAccess service from Local System to a user whose account has the correct proxy settings configured.

Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/badge/{badgekey}/mobile_devices?version=value

get badge mobile_devices

Name	Type	Required	Description
badgekey	int32	yes	The badgekey of the mobile device assigned to a person. For more information, refer to LnI_Badge on page 197.

get badge mobile_devices response

Name	Type	Required	Description
total_items	int32	yes	The total existing number of instances.
mobile_device_list	list	yes	A list of mobile devices for the person associated with the badge. See get badge mobile_devices response: mobile_device_list properties on page 86.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get badge mobile_devices response: mobile_device_list properties

Name	Type	Required	Description
mobile_device_id	integer	yes	The mobile device's ID.
mobile_device_description	string	yes	The mobile device's descriptive name.
mobile_device_active	boolean	yes	Identifies whether or not the mobile device is active.

add badge issue_mobile_credential

This method issues a credential to a mobile device for the person with the given badge.

Notes: If you are using OpenAccess to issue mobile badges and are behind a network proxy, an error might occur when issuing or managing mobile credentials. To resolve this error, on the server where the LS OpenAccess service is running, change the logon account for the LS OpenAccess service from Local System to a user whose account has the correct proxy settings configured.

Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: POST /api/access/onguard/openaccess/badge/{badgekey}/issue_mobile_credential?version=value

add badge issue_mobile_credential

Name	Type	Required	Description
badgekey	int32	yes	The unique identifier of the badge for which a mobile credential should be issued. For more information, refer to LnI_Badge on page 197.
in_parameter_value_map	map	yes	A list of optional parameters to configure on the issued mobile credential. See add badge issue_mobile_credential: in_parameter_value_map properties on page 87.

add badge issue_mobile_credential: in_parameter_value_map properties

Name	Type	Required	Description
mobile_device_id	string	no	The mobile device's ID.
send_email	boolean	no	Set this value to False to prevent a welcome email from being sent to the cardholder upon issuance of the mobile credential. The default is to send an email.
mobile_issuance_method	string	no	Set this value to "regenerate" to resend the welcome email to a cardholder whose badge already had a mobile credential issued. Not specifying a value, or specifying any other value, causes a new mobile credential to be issued to the given badge.

add badge issue_mobile_credential response

Name	Type	Required	Description
mobile_device_activation_code	int32	yes	The activation code to use for issuing a credential to the mobile device.
mobile_issuance_message	string	yes	An optional message reported from the credentialing service to indicate additional issuance status information.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get badge printers

Retrieves a list of printers available for badge printing.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/badge_printers?version=value&badge_type_id=value

get badge_printers

Name	Type	Required	Description
badge_type_id	int32	no	When not passed into the request, the API returns all available printers for all badge types. Represents the badge type id found in the BadgeType table.

get badge_printers response

Name	Type	Required	Description
printers	array	yes	An array describing the available printers.
badge_type_id	int32	yes	The badge type ID.
printer_name	string	yes	The printer name, or the network path to the printer.
workstation	string	yes	The workstation associated with the printer. An asterisk (*) indicates the default network printer.
total_items	int32	yes	The number of items returned.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

Sample JSON Response

```

1      {
2      "printers": [
3      {
4      "badge_type_id": 1,
5      "printer_name": "\\PC-2016\\Printer Brand and Model 1",
6      "workstation": "*"
7      },
8      {
9      "badge_type_id": 1,
10     "printer_name": "ABC Card Printer",
11     "workstation": "PC-2016"
12     }
13     ],
14     "total_items": 2,
15     "version": "1.0"
16     }

```

add instances

Adds instances of a particular type.

Note: You must use Version 1.1 or later of this method if you need support for *BadgeID_str*, *badge_id_str*, *badge_key_str*, *CardNumber_str*, or *AssignedBadgeID_str*.

REST Request URL: POST /api/access/onguard/openaccess/instances?version=value

REST Request Body Contents:

```

{
  "type_name": "value",
  "property_value_map":
  {
    "property_name": value,
    ...
  }
}

```



```
}

```

add instances

Name	Type	Required	Description
type_name	string	yes	The name of the type being added. For example "Lnl_Cardholder".
property_value_map	map	yes	The property name to property value map that represents the instance data to add.

add instances response

Name	Type	Required	Description
type_name	string	yes	The name of the type being added. For example "Lnl_Cardholder".
property_value_map	map	yes	The property name to property value map that represents the instance data of the added object. Only key properties are returned for add instances calls.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

modify instances

Modifies existing instances of a particular type.

Note: You must use Version 1.1 or later of this method if you need support for *BadgeID_str*, *badge_id_str*, *badge_key_str*, *CardNumber_str*, or *AssignedBadgeID_str*.

REST Request URL: PUT /api/access/onguard/openaccess/instances?version=value

REST Request Body Contents:

```
{
  "type_name": "value",
  "property_value_map":
  {
    "property_name": value,
    ...
  }
}
```

modify instances

Name	Type	Required	Description
type_name	string	yes	The name of the type being modified. For example, "Lnl_Cardholder".

modify instances

Name	Type	Required	Description
property_value_map	map	yes	The property name to property value map that represents the instance data to be modified. Note: Key properties must be specified here to resolve the object that will be modified properly.

modify instances response

Name	Type	Required	Description
type_name	string	yes	The name of the type to modify. For example, "Lnl_Cardholder".
property_value_map	map	yes	The property name to property value map that represents the instance data of the modified object. Only key properties are returned for modify instances calls.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

bulk modify instance property

Bulk modifies the value of an instance's property.

REST Request URL: PUT /api/access/onguard/openaccess/property_bulk_update?version=value

REST Request Body Contents:

```
{
  "property_name": "value",
  "property_value": "value"
}
```

bulk modify instance property

Name	Type	Required	Description
type_name	string	yes	The name of the type. Currently only "Lnl_User" is supported.
property_name	string	yes	The name of the property. Currently only "PasswordChangeRequired" is supported.
property_value	string	yes	The new property value. For example, input "true" or "false" for property "Lnl_User.PasswordChangeRequired".

bulk modify instance property

Name	Type	Required	Description
id_list	list	no	List of instance IDs in the format [1,2,3,...]. If no list is provided, all instances are modified. For example, if the property is "LnI_User.PasswordChangeRequired" and no list is provided, all users with internal accounts are modified.

bulk modify instance property response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

delete instances

Deletes existing instances of a particular type.

REST Request URL: DELETE /api/access/onguard/openaccess/instances?version=value

REST Request Body Contents:

```
{
  "type_name": "value",
  "property_value_map":
  {
    "property_name": value,
    ...
  }
}
```

delete instances

Name	Type	Required	Description
type_name	string	yes	The name of the type being deleted. For example "LnI_Cardholder".
property_value_map	map	yes	The key property name to key property value map that represents the instance data to be deleted. Note: Key properties must be specified here in order to properly resolve the object to be deleted.

execute_method

Executes a supported method against an existing instance of a particular type. For an example, refer to [Chapter 7: Using OpenAccess to Send Alarms to OnGuard](#) on page 277.

Note: You must use Version 1.1 or later of this method if you need support for *BadgeID_str*, *badge_id_str*, *badge_key_str*, *CardNumber_str*, or *AssignedBadgeID_str*.

REST Request URL: POST /api/access/onguard/openaccess/execute_method?version=value

REST Request Body Contents:

```
{
  "method_name": "value",
  "type_name": "value",
  "property_value_map": {
    "property_name": value,
    ...
  },
  "in_parameter_value_map": {
    "property_name": value,
    ...
  }
}
```

execute method

Name	Type	Required	Description
type_name	string	yes	The name of the type being operated upon. For example "LnI_IncomingEvent".
property_value_map	map	yes	The key property name to key property value map that represents the instance data to be operated on. Note: Key properties must be specified here to properly resolve the object on which to execute the method.
method_name	string	yes	The name of the method to be executed. Supported methods are returned in the get type response. For example, "SendIncomingEvent".
in_parameter_value_map	map	no	The name/value map of any input parameters to the method.

execute method response

Name	Type	Required	Description
out_parameter_value_map	map	no	The name/value map of any output of the method.

execute method response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

get cardholders

Performs an advanced cardholder search, optionally searching on badge fields. Returns instances that match the search criteria. For more information, refer to [Lnl_Cardholder](#) on page 206.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/cardholders?auto_load_badge=value&auto_load_multimedia_object=value&auto_load_access_level=value&auto_load_reader=value&auto_load_timezone=value&auto_load_timezone_interval=value&version=value&page_number=value&page_size=value&order_by=value&cardholder_filter=value&badge_filter=value&has_badges=value&has_photo=value&has_signature=value&has_any_directory_account=value&access_level_filter=value&access_level_list=[value1,value2,...,valueN]&access_level_search_type=value&reader_filter=value&reader_name_list=[value1,value2,...,valueN]&reader_name_list_search_type=value&timezone_name_list=[value1,value2,...,valueN]&timezone_name_list_search_type=value

get cardholders

Name	Type	Required	Description
access_level_list	list	no	A list of access level IDs for which to search cardholders. For example: [1,2,3]. This parameter must be used with the access_level_search_type property.
access_level_search_type	string	no	The type of access level search to apply. This parameter describes how to interpret access_level_list: • any_of - Finds cardholders with any of the access levels in access_level_list (at least one). • none_of - Finds cardholders with none of the access levels in access_level_list. • all_of - Finds cardholders with all of the access levels in access_level_list. • exactly - Finds cardholders with exactly the access levels in access_level_list (all of the access levels and no others).

get cardholders

Name	Type	Required	Description
cardholder_filter	string	no	The filter, based on the cardholder properties. For more information refer to Searching for Objects on page 39 and Lnl_Cardholder on page 206.
badge_filter	string	no	The filter, based on the badge properties. For more information refer to Searching for Objects on page 39 and Lnl_Badge on page 197.
has_badges	boolean	no	Boolean search for confirming that the cardholder has a badge. - If has_badges = false, cardholders that have no badges are returned as specified by cardholder_filter. - If has_badges = true, cardholders that have at least one badge are returned as specified by cardholder_filter. - If has_badges is not specified in the request, cardholders are returned as specified by cardholder_filter. - If specifying has_badges = false, it cannot be combined with badge_filter. InvalidRequest error is returned if you specify both.
has_photo	boolean	no	Boolean search for confirming that the cardholder has a photo.
has_signature	boolean	no	Boolean search for confirming that the cardholder has a signature.
access_level_list	list	no	A list of access level IDs for which to search cardholders. For example: [1,2,3]. This parameter must be used with the access_level_search_type property.
access_level_search_type	string	no	The type of access level search to apply. This parameter describes how to interpret access_level_list: any_of - Finds cardholders with any of the access levels in access_level_list (at least one). none_of - Finds cardholders with none of the access levels in access_level_list. all_of - Finds cardholders with all of the access levels in access_level_list. exactly - Finds cardholders with exactly the access levels in access_level_list (all of the access levels and no others).

get cardholders

Name	Type	Required	Description
auto_load_badge	boolean	no	A flag indicating whether to load the badges assigned to cardholders in the response. This parameter requires Version 1.2 or later of the get cardholders method.
auto_load_multimedia_object	boolean	no	A flag indicating whether to load the multimedia objects (such as cardholder photos and signatures) assigned to cardholders in the response. This parameter requires Version 1.2 or later of the get cardholders method.
auto_load_access_level	boolean	no	A flag indicating whether to load the access levels assigned to cardholders in the response. This parameter requires Version 1.2 or later of the get cardholders method.
auto_load_reader	boolean	no	A flag indicating whether to load the readers assigned to access levels in the response. This parameter requires Version 1.2 or later of the get cardholders method.
auto_load_timezone	boolean	no	A flag indicating whether to load the timezones assigned to access levels in the response. This parameter requires Version 1.2 or later of the get cardholders method.
auto_load_timezone_interval	boolean	no	A flag indicating whether to load the time-zone intervals assigned to the timezone. This parameter requires Version 1.2 or later of the get cardholders method.
access_level_filter	string	no	The filter, based on Access Level properties (for example, Name="name1" or AccessMode=1). This parameter requires Version 1.2 or later of the get cardholders method.
reader_filter	string	no	The filter, based on Reader properties. This parameter requires Version 1.2 or later of the get cardholders method.
reader_name_list	list	no	A list of reader names for which to search cardholders. For example: [name1,name2,name3]. This parameter must be used with the reader_name_list_search_type. This parameter requires Version 1.2 or later of the get cardholders method.

get cardholders

Name	Type	Required	Description
reader_name_list_search_type	string	no	The type of reader name list search to apply. This parameter describes how to interpret reader_name_list. • any_of - Finds cardholders with any of the readers in reader_name_list (at least one). • none_of - Finds cardholders with none of the readers in reader_name_list. • all_of - Finds cardholders with all of the readers in reader_name_list. • exactly - Finds cardholders with exactly the readers in reader_name_list (all of the readers and no others). This parameter requires Version 1.2 or later of the get cardholders method.
timezone_name_list	list	no	A list of timezone names for which to search cardholders. For example: [name1,name2,name3]. This parameter must be used with the timezone_name_list_search_type. This parameter requires Version 1.2 or later of the get cardholders method.
timezone_name_list_search_type	string	no	The type of timezone name list search to apply. This parameter describes how to interpret timezone_name_list. • any_of - Finds cardholders with any of the timezones in timezone_name_list (at least one). • none_of - Finds cardholders with none of the timezones in timezone_name_list. • all_of - Finds cardholders with all of the timezones in timezone_name_list. • exactly - Finds cardholders with exactly the timezones in timezone_name_list (all of the timezones and no others). This parameter requires Version 1.2 or later of the get cardholders method.
page_number	int32	no	The page number of the requested subset (page) of instances returned. Same as corresponding input parameter, or the default value if not provided as input.

get cardholders

Name	Type	Required	Description
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
order_by	string	no	A field or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by key field(s). Fields must be valid properties of the requested object type. For more information, refer to Additional order_by Details on page 81.

get cardholders response

Name	Type	Required	Description
page_number	int32	no	The page number of the requested subset (page) of instances returned. Same as corresponding input parameter, or the default value if not provided as input.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total existing number of instances of the object being requested.
count	int32	yes	The total number of records in the filter result.
item_list	list	yes	A list of Lnl_Cardholder items returned, if instances exist. If a valid order_by parameter was provided in the request, then the list of items is sorted accordingly. If present, each item consists of a property_value_map for the cardholder being returned (see footnote below).

get cardholders response

Name	Type	Required	Description
property_value_map*	map	yes	This is a map where the key is property name and the value is the actual property value. Each property_value_map entry in the item_list represents a single cardholder. For more information, refer to Lnl_Cardholder on page 206. Each cardholder can also contain nested sub-objects, as described in the footnote below. For more information about those objects, refer to Lnl_Badge on page 197, Lnl_MultimediaObject on page 229, Lnl_AccessLevel on page 182, Lnl_Reader on page 239, Lnl_Timezone on page 253, and Lnl_TimezoneInterval on page 253.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

* The property_value_map in **get cardholders response** can optionally contain badges and multimedia objects, depending on how the following boolean values were configured in the request:

- auto_load_badge
- auto_load_multimedia_object
- auto_load_access_level
- auto_load_reader
- auto_load_timezone
- auto_load_timezone_interval

It's important to understand the following object hierarchy:

Cardholders				
	• Multimedia objects			
	• Badges			
		• Access levels		
			• Reader assignment	
				• Readers
				• Timezones
				• Timezone intervals

In the **get cardholders** request, enabling a boolean value for an object type without having enabled the boolean value for all higher object types will generate an error. For example, if you enable auto_load_access_level without having also enabled auto_load_badge, then an error is generated.

get visitors

Performs an advanced visitor search, optionally searching on badge fields. Returns instances that match the search criteria. For more information, refer to [Lnl_Visitor](#) on page 264.

REST Request URL: GET /api/access/onguard/openaccess/visitors?auto_load_badge=value&auto_load_multimedia_object=value&auto_load_access_level=value&auto_load_reader=value&auto_load_timezone=value&auto_load_timezone_interval=value&version=value&page_number=value&page_size=value&order_by=value&visitor_filter=value&badge_filter=value&has_badges=value&has_photo=value&has_signature=value&has_any_directory_account=value&access_level_filter=value&access_level_list=[value1,value2,...,valueN]&access_level_search_type=value&reader_filter=value&reader_name_list=[value1,value2,...,valueN]&reader_name_list_search_type=value&timezone_name_list=[value1,value2,...,valueN]&timezone_name_list_search_type=value

get visitors

Name	Type	Required	Description
visitor_filter	string	no	The filter, based on the visitor properties. For more information refer to Searching for Objects on page 39 and Lnl_Visitor on page 264.
badge_filter	string	no	The filter, based on the badge properties. For more information refer to Searching for Objects on page 39 and Lnl_Badge on page 197.
has_badges	boolean	no	Boolean search for confirming that the visitor has a badge.
has_photo	boolean	no	Boolean search for confirming that the visitor has a photo.
has_signature	boolean	no	Boolean search for confirming that the visitor has a signature.
access_level_filter	string	no	The filter, based on Access Level properties.
access_level_list	list	no	A list of access level IDs for which to search visitors. For example: [1,2,3]. This parameter must be used with the access_level_search_type property.

get visitors

Name	Type	Required	Description
access_level_search_type	string	no	The type of access level search to apply. This parameter describes how to interpret access_level_list: • any_of - Finds visitors with any of the access levels in access_level_list (at least one). • none_of - Finds visitors with none of the access levels in access_level_list. • all_of - Finds visitors with all of the access levels in access_level_list. • exactly - Finds visitors with exactly the access levels in access_level_list (all of the access levels and no others).
auto_load_badge	boolean	no	A flag indicating whether to load the badges assigned to visitors in the response.
auto_load_multimedia_object	boolean	no	A flag indicating whether to load the multimedia objects (such as photos and signatures) assigned to visitors in the response.
auto_load_access_level	boolean	no	A flag indicating whether to load the access levels assigned to visitors in the response.
auto_load_reader	boolean	no	A flag indicating whether to load the readers assigned to access levels in the response.
auto_load_timezone	boolean	no	A flag indicating whether to load the timezones assigned to access levels in the response.
auto_load_timezone_interval	boolean	no	A flag indicating whether to load the time-zone intervals assigned to the timezone.
reader_filter	string	no	The filter, based on Reader properties.
reader_name_list	list	no	A list of reader names for which to search visitors. For example: [name1,name2,name3]. This parameter must be used with the reader_name_list_search_type.

get visitors

Name	Type	Required	Description
reader_name_list_search_type	string	no	The type of reader name list search to apply. This parameter describes how to interpret reader_name_list. • any_of - Finds visitors with any of the readers in reader_name_list (at least one). • none_of - Finds visitors with none of the readers in reader_name_list. • all_of - Finds visitors with all of the readers in reader_name_list. • exactly - Finds visitors with exactly the readers in reader_name_list (all of the readers and no others).
timezone_name_list	list	no	A list of timezone names for which to search visitors. For example: [name1,name2,name3]. This parameter must be used with the timezone_name_list_search_type.
timezone_name_list_search_type	string	no	The type of timezone name list search to apply. This parameter describes how to interpret timezone_name_list. • any_of - Finds visitors with any of the timezones in timezone_name_list (at least one). • none_of - Finds visitors with none of the timezones in timezone_name_list. • all_of - Finds visitors with all of the timezones in timezone_name_list. • exactly - Finds visitors with exactly the timezones in timezone_name_list (all of the timezones and no others).
page_number	int32	no	The page number of the requested subset (page) of instances returned. Same as corresponding input parameter, or the default value if not provided as input.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.

get visitors

Name	Type	Required	Description
order_by	string	no	A field or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by key field(s). Fields must be valid properties of the requested object type. For more information, refer to Additional order_by Details on page 81.

get visitors response

Name	Type	Required	Description
page_number	int32	no	The page number of the requested subset (page) of instances returned. Same as corresponding input parameter, or the default value if not provided as input.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total existing number of instances of the object being requested.
count	int32	yes	The total number of records in the filter result.
item_list	list	yes	A list of Lnl_Visitor items returned, if instances exist. If a valid order_by parameter was provided in the request, then the list of items is sorted accordingly. If present, each item consists of a property_value_map for the visitor being returned (see footnote below).

get visitors response

Name	Type	Required	Description
property_value_map*	map	yes	This is a map where the key is property name and the value is the actual property value. Each property_value_map entry in the item_list represents a single visitor. For more information, refer to LnI_Visitor on page 264. Each visitor can also contain nested sub-objects, as described in the footnote below. For more information about those objects, refer to LnI_Badge on page 197, LnI_MultimediaObject on page 229, LnI_AccessLevel on page 182, LnI_Reader on page 239, LnI_Timezone on page 253, and LnI_TimezoneInterval on page 253.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

* The property_value_map in **get visitors response** can optionally contain badges and multimedia objects, depending on how the following boolean values were configured in the request:

- auto_load_badge
- auto_load_multimedia_object
- auto_load_access_level
- auto_load_reader
- auto_load_timezone
- auto_load_timezone_interval

It's important to understand the following object hierarchy:

Visitors				
	• Multimedia_ objects			
	• Badges			
		• Access levels		
			• Reader assignment	
				• Readers
				• Timezones
				• Timezone intervals

In the **get visitors** request, enabling a boolean value for an object type without having enabled the boolean value for all higher object types will generate an error. For example, if you enable `auto_load_access_level` without having also enabled `auto_load_badge`, then an error is generated.

get video_recorders

This method retrieves one page of the list of all video recorders configured in the OnGuard system.

Notes: This method replaces the previously existing `get instances` call for the type `Lnl_VideoRecorder`, which retrieved only Lenel NVR video recorders. This method retrieves all recorders, regardless of type.

Version 1.1 and later of this method supports filtering as discussed in [get instances](#) on page 80.

Version 1.2 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

Version 1.3 of this method supports retrieving gateway information linked to the recorder, and recorder information needed by the gateway.

REST Request URL: GET `/api/access/onguard/openaccess/video_recorders?version=value`

get video_recorders

Name	Type	Required	Description
<code>order_by</code>	string	no	The fields to use when sorting the results.
<code>filter</code>	string	no	The filter used to retrieve instances. For example, <code>Lastname = "Smith"</code> and <code>Firstname = "Lisa"</code>. Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an <code>InvalidQuery</code> error. For more information refer to Searching for Objects on page 39.
<code>page_number</code>	int32	no	The page number to be returned when a subset (page) of instances is requested. Used in conjunction with <code>page_size</code> . Defaults to the first page (1) if not provided, and if provided, must be numeric.
<code>page_size</code>	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with <code>page_number</code> . Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (<code>page_size</code>) that can be retrieved with a single request is 100.

get video_recorders response

Name	Type	Required	Description
computer_name	string	yes	The computer name of the recorder.
count	int32	yes	The number of recorders returned in the response.
database_id	int32	yes	The database ID that identifies the server containing this recorder. Only returned for Enterprise systems.
gateway_http_port	int32	yes	The HTTP port configured for the Lenel NVR web service's Gateway.
gateway_https_port	int32	yes	The HTTPS port configured for the Lenel NVR web service's Gateway.
gateway_id	int32	yes	The internal database ID of the gateway in the access panel table. Key field.
gateway_name	string	yes	The display name of the gateway.
gateway_network_address	string	yes	The fully qualified name of the gateway.
http_port	int32	yes	The HTTP port configured for the Lenel NVR web service.
https_port	int32	yes	The HTTPS port configured for the Lenel NVR web service.
id	int32	yes	The internal database ID of the recorder in the access panel table. Key field.
is_daylight_saving	boolean	yes	Whether or not this recorder observes Daylight Saving Time.
is_online	boolean	yes	Whether or not the recorder is online.
mediasource_classid	string	yes	Unique ID of the media source component used by the gateway to communicate with the respective recorder.
name	string	yes	The display name of the recorder
page_number	int32	no	The page number of the requested subset (page) of instances returned. Same as corresponding input parameter, or the default value if not provided as input.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.

get video_recorders response

Name	Type	Required	Description
panel_type_id	int32	yes	The internal database ID of the type of recorder in the panel type table.
panel_type_name	string	yes	The name of the panel type.
port	int32	yes	The recorder port configured on the Video Recorder form (Connection sub-tab) in System Administration. Used to establish a connection to the recorder.
primary_ip_address	int32	yes	The primary IP address to use when connecting to a server with network access.
segment_id	int32	yes	The segment to which this recorder belongs. Only returned for segmented systems.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total existing number of instances of the object being requested.
workstation	int32	yes	The recorder workstation name.
world_timezone_id	int32	yes	The time zone of the recorder (reference to Lnl_WorldTimezone.ID)
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get video_recorder auth_data

This method retrieves the authentication token for a Lenel NVR. This token is used for authentication and authorization against Lenel NVR Services. This method replaces the GetAuthenticationData method of the Lnl_VideoRecorder type.

Notes: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

In OnGuard 7.4, this method is supported for video recorders of type Lenel NVR only.

REST Request URL: GET /api/access/onguard/openaccess/video_recorder/{id}/auth_data?version=value

get video_recorder auth_data

Name	Type	Required	Description
id	int32	yes	The panel ID of the recorder for which the authentication data is being requested.

get video_recorder auth_data response

Name	Type	Required	Description
authentication_data	string	yes	The authentication token for the specified recorder, or both the recorder and gateway.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

get video_recorder credentials

Retrieves user credentials for the video recorder with the provided panel ID and realm.

REST Request URL: GET /api/access/onguard/openaccess/video_recorder/{id}/credentials?realm_value=value&version=value

get video_recorder credentials

Name	Type	Required	Description
id	int32	yes	The panel ID of the recorder for which credentials data is being requested.
realm_value	string	yes	String parameter provided by the client, to be used when calculating the hash.

get video_recorder credentials response

Name	Type	Required	Description
video_recorder_hashed_data	string	yes	The hashed string of the user's credentials.
video_recorder_user_name	string	yes	The user's name, in plain text.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

get map background

This method retrieves the background for a specific map ID. The response is an array of image data without encoding, in .png or .jpg format.

Note: The maximum image size is 55 MB.

REST Request URL: GET /api/access/onguard/openaccess/map/{id}/background?version=value

get map background

Name	Type	Required	Description
id	int32	yes	The map ID.

put access_level

Use this method to update access levels without using System Administration. Currently, this method only allows the assignment of readers to an access level, or the removal of readers from an access level.

REST Request URL: PUT /api/access/onguard/openaccess/access_level/{id}

put access_level

Name	Type	Required	Description
readers	JSON body	yes	JSON, in the format: [{ "reader_id": 5, "panel_id": 7, "timezone_id": 1 }, { "reader_id": 6, "panel_id": 2, "timezone_id": 1 }]

put access_level response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

post send_incoming_events

Use this method to send a list of events to the OnGuard software. For more information, refer to [Chapter 7: Using OpenAccess to Send Alarms to OnGuard](#) on page 277.

REST Request URL: POST /api/access/onguard/openaccess/send_incoming_events?version=value

post send_incoming_events

Name	Type	Required	Description
incoming_event_list	list	yes	A list of incoming events to send. Each event has the following attributes: • event_type: The event type. • event_id: The event ID. • source_id: The logical source ID. • device_id: The logical device ID. • subdevice_id: The logical sub-device ID. • event_text: The event text. • badge_id: The badge ID. • badge_id_str: A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the badge_id property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off. You cannot provide both badge_id and badge_id_str in the same call. • extended_id: The badge extended ID. • time: The event time. Optional. If not provided, the current time is used when sending the event.

post send_incoming_events response

Name	Type	Required	Description
failure_list	map	yes	A map of failed event attributes: - total_items: The total count of failed events. - item_list: The failed event list. Each failed event has the following attributes: - event_type: The event type. - event_id: The event ID. - source_id: The logical source ID. - device_id: The logical device ID. - subdevice_id: The logical sub-device ID. - event_text: The event text. - badge_id: The badge ID. - badge_id_str: A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the badge_id property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off. - extended_id: The badge extended ID. - time: The event time.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

Users**get logged_in_user**

Returns information pertaining to the authenticated user.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/
logged_in_user?version=value

get logged_in_user response

Name	Type	Required	Description
user_id	string	yes	The user's ID, as a string.
user_name	string	yes	The user's user name, in plain text.
first_name	string	yes	The user's first name.
last_name	string	yes	The user's last name.
password_expiration_time	datetime (string)	yes	The date and time that the password will expire. This only exists if the user logged in with the password expiration policy enabled.
permission_map	map	yes	A subset of user permissions configured in System Administration. For each entry in the map, the value is true if the user's assigned permission group has this permission, or false if the user's permission group does not have this permission. For more information, refer to "Administration: Users Folder: Permission Groups Tree: User Permissions" in the <i>System Administration User's Guide</i> .
ptz_priority	int32	yes	The PTZ priority level of the user. Since only one person can control a PTZ camera at a time, a user with higher priority can take over PTZ control of a camera from someone who has lower priority. The SA user and SA delegate users have a PTZ priority of 1000. Other users are assigned values between 1 (low priority) and 255 (high priority). For more information, refer to "Monitor Permission Groups: Permissions Sub-tab Procedures" in the <i>System Administration User's Guide</i> .
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get user managed_access_levels

Returns a list of access levels a user can manage, and indicates if the user has Area Access Manager view-only access.

Notes: If an **sa** user or SA delegate user calls get managed_access_levels after authenticating with OpenAccess as "sa", OpenAccess returns no results. The **sa** user and SA delegate users can manage all access levels in the system.

Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/user/{id}/managed_access_levels?version=value

get user managed_access_levels

Name	Type	Required	Description
id	string	yes	ID of the user for whom you want the managed access levels, as a string.

get user managed_access_levels response

Name	Type	Required	Description
access_level_list	list	yes	The list of access levels a user can manage. Each item in the list contains the id , which is the ID of the access level associated with the user, and the name , which is the name of the access level. The access level filter and badge filter are combined, so that the access level search is applied only to those badges that match the badge filter.
total_items	int32	yes	A count of the items in the access_level_list.
has_aam_view_only_access	boolean	yes	Describes if the user has view-only access to levels in Area Access Manager. If false, the user can control all assigned access levels in Area Access Manager. For a list of access levels the user can control, refer to get user managed_access_levels on page 111.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

add user managed_access_levels

Adds to the existing list of access levels a user can manage.

Notes: If adding any of the requested access levels fail, an error code is provided and none of the requested access levels are added.

Access level management cannot be added to the SA user or SA delegate users.

REST Request URL: POST /api/access/onguard/openaccess/user/{id}/managed_access_levels?version=value

REST Request Body Contents:

```
{
  "access_level_list":
  [
```



```

        access_level_id,
        ...
    ]
}

```

add user managed_access_levels

Name	Type	Required	Description
id	string	yes	ID of the user to which access level management will be added, as a string.
access_level_list	list	yes	A list of access level IDs the user can manage.

delete user managed_access_levels

Deletes specific access levels from the access levels a user can manage.

REST Request URL: DELETE /api/access/onguard/openaccess/user/{id}/managed_access_levels?version=value

REST Request Body Contents:

```

{
  "access_level_list":
  [
    access_level_id,
    ...
  ]
}

```

delete user managed_access_levels

Name	Type	Required	Description
id	string	yes	ID of user from which to remove access level management, as a string.
access_level_list	list	yes	A list of access level IDs the user cannot manage.

get user

Gets the OnGuard-specific properties for a user.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/user/{id}?version=value

get user

Name	Type	Required	Description
id	string	yes	ID of the user for whom you want the monitoring zone ID and monitoring zone name, as a string.

get user response

Name	Type	Required	Description
database_id	int32	yes	The database identifier in an Enterprise system that identifies the server containing the user. For more information, refer to get enterprise settings on page 133.
monitoring_zone_id	int32	yes	The ID of the user's monitoring zone. For more information, refer to LnI_Monitoring-Zone on page 226.
monitoring_zone_name	string	yes	The name of the user's monitoring zone. If the user is not associated with a monitoring zone, then this property is returned as empty.
has_aam_view_only_access	boolean	yes	Describes if the user has view-only access to levels in Area Access Manager. If false, the user can control all assigned access levels in Area Access Manager. For a list of access levels the user can control, refer to get user managed_access_levels on page 111.
is_user_account_locked	boolean	yes	A flag to indicate if the user's account is locked because of too many incorrect password attempts.
last_successful_login_time	datetime	yes	The date and time of the user's last successful login.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

modify user

Modifies the OnGuard-specific properties for a user.

REST Request URL: PUT /api/access/onguard/openaccess/user/{id}?version=value&database_id=value

modify user

Name	Type	Required	Description
database_id	int32	no	The database identifier in an Enterprise system that identifies the server containing the user. If changing this value with a modify user call, the existing value must be -1 or the local DatabaseID, or an insufficient privileges error is returned. For more information, refer to get enterprise settings on page 133.
id	string	yes	ID of the user for whom you want to assign the monitoring zone ID, as a string.
monitoring_zone_id	int32	no	ID of the monitoring zone you want to assign to the user.
has_aam_view_only_access	boolean	no	Describes if the user has view-only access to levels in Area Access Manager. If false, the user can control all assigned access levels in Area Access Manager. For a list of access levels the user can control, refer to get user managed_access_levels on page 111. Note: You can only modify this value if the user has at least one access level to manage.
unlock_account	boolean	no	If true, unlock the account of the user with a locked account because of too many incorrect password attempts.

put user password

Update the current user's password.

REST Request URL: PUT /api/access/onguard/openaccess/user_password?version=value

put user password

Name	Type	Required	Description
user_name	string	yes	The user's name.
current_password	string	yes	The current password.
new_password	string	yes	The new password.

get managers_of_access_level

Gets a list of user IDs for users who can manage the access level.

Note: Users assigned “view-only” permission to an access level are not included in the list returned from this call.

REST Request URL: GET /api/access/onguard/openaccess/managers_of_access_level?access_level_id=value&version=value

get managers_of_access_level

Name	Type	Required	Description
access_level_id	int32	yes	ID of the access level for which to retrieve users who can manage that access level.

get managers_of_access_level response

Name	Type	Required	Description
total_items	int32	yes	A count of users who can manage the access level.
user_id_list	list	yes	List of user IDs for users who can manage the access level.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

get editable_segments

Gets a list of segments and segment groups for which the logged-in user has editable permission. For more information, refer to [Lnl_Segment](#) on page 252.

Notes: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

This call is not supported on non-segmented systems. Use the **get segmentation** call to determine if your system supports segmentation (refer to [get segmentation settings](#) on page 139).

REST Request URL: GET /api/access/onguard/openaccess/editable_segments?version=value

get editable_segments response

Name	Type	Required	Description
total_items	int32	yes	A count of segments and segment groups for which the logged-in user has editable permission.

get editable_segments response

Name	Type	Required	Description
segment_list	list	yes	The list of segments assigned to a user. Each item in the list contains the segment_id , which is the ID of the segment assigned to the user, the segment_name , which is the name of the segment, and type , which is either segment_unit , or segment_group . For Enterprise systems, also returns database_id for each item in the segment_list , and type can also be dynamic_segment . For more information, refer to LnI_Segment on page 252.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

get user segments

Returns a list of segments assigned to a user.

Notes: This call is not supported on non-segmented systems. Use the **get segmentation** call to determine if your system supports segmentation. For more information, refer to [get segmentation settings](#) on page 139.

Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/user/{id}/segments?version=value

get user segments

Name	Type	Required	Description
id	string	yes	ID of the user for whom you want to retrieve segments, as a string.

get user segments response

Name	Type	Required	Description
segment_list	list	yes	The list of segments assigned to a user. Each item in the list contains the segment_id , which is the ID of the segment assigned to the user, the segment_name , which is the name of the segment, and type , which is either segment_unit , or segment_group . For Enterprise systems, also returns database_id for each item in the segment_list , and type can also be dynamic_segment . For more information, refer to LnI_Segment on page 252.
total_items	int32	yes	A count of the segments in the segment_list.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

add user segments

Adds to the existing list of segments assigned to a user. Use the **get editable_segments** call to determine which segments can be assigned to a user. For more information, refer to [get editable_segments](#) on page 116.

Note: This call is not supported on non-segmented systems. Use the **get segmentation** call to determine if your system supports segmentation. For more information, refer to [get segmentation settings](#) on page 139.

REST Request URL: POST /api/access/onguard/openaccess/user/{id}/segments?version=value

REST Request Body Contents:

```
{
  "segment_list":
  [
    segment_id,
    ...
  ]
}
```

add user segments

Name	Type	Required	Description
id	string	yes	ID of the user to which segment assignment will be added, as a string.

Name	Type	Required	Description
segment_list	list	yes	A list of segment IDs that indicate which segments to assign to the user. For more information, refer to Lnl_Segment on page 252.

delete user segments

Deletes specific segments from the segments assigned to a user. Use the **get editable_segments** call to determine which segments can be deleted from a user. For more information, refer to [get editable_segments](#) on page 116.

Note: This call is not supported on non-segmented systems. Use the **get segmentation** call to determine if your system supports segmentation. For more information, refer to [get segmentation settings](#) on page 139.

REST Request URL: DELETE /api/access/onguard/openaccess/user/{id}/segments?version=value

REST Request Body Contents:

```
{
  "segment_list":
  [
    segment_id,
    ...
  ]
}
```

delete user segments

Name	Type	Required	Description
id	string	yes	ID of user from which to remove segment assignment, as a string.
segment_list	list	yes	A list of segment IDs that indicate which segments to remove from the user. For more information, refer to Lnl_Segment on page 252.

get user preferences

Gets the user preferences of the logged in user.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/user_preferences?version=value&setting_type=value&preference_id=value&is_global=value

get user preferences

Name	Type	Required	Description
setting_type	string	yes	The setting type refers to the category of settings to which the client wants to refer. For example, setting_type="UI".
preference_id	int32	no	The unique ID of the preference.
is_global	boolean	no	Optional parameter. Get call returns all the preferences of the logged-in user, as well as global preferences. If TRUE, only the global preferences are returned. If FALSE, returns the preferences of that logged-in user only.
client_name	string	yes	The name of the client application making use of the user preferences (for example, Credentials, CSS, Access Manager, Monitor).

get user preferences response

Name	Type	Required	Description
preference_list	string	yes	Refers to the list of preferences, in JSON format.
total_list	int32	yes	The total number of user preferences retrieved.
client_name	string	yes	The name of the client application making use of the user preferences (for example, Credentials, CSS, Access Manager, Monitor).
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

put user preferences

Update the existing user preferences of the logged in user.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: PUT /api/access/onguard/openaccess/user_preferences?version=value

put user preferences

Name	Type	Required	Description
preference_id	int32	yes	The unique identifier of the user preference.
preference_settings	string	no	The preference settings refers to the data the user wants to save, in json format. For example: preference_settings: { "Address": { "Operator": "LIKE", "value": "NYC" } }
setting_type	string	yes	The setting type refers to the category of settings to which the client wants to refer. For example, setting_type="UI".

put user preferences response

Name	Type	Required	Description
preference_id	int32	yes	The unique identifier of the user preference.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

post user preferences

Save the user preferences of the logged in user.

REST Request URL: POST /api/access/onguard/openaccess/user_preferences?version=value

post user preferences

Name	Type	Required	Description
preference_settings	string	no	The preference settings refers to the data the user wants to save, in json format. For example: preference_settings: { "Address": { "Operator": "LIKE", "value": "NYC" } }
setting_type	string	yes	The setting type refers to the category of settings to which the client wants to refer. For example, setting_type="UI".
is_global	boolean	no	If TRUE, the preference is visible to other users. If FALSE, the preference is visible only to the logged-in user.

Name	Type	Required	Description
client_name	string	yes	The name of the client application making use of the user preferences (for example, Credentials, CSS, Access Manager, Monitor).

post user preferences response

Name	Type	Required	Description
preference_id	int32	yes	The unique identifier of the user preference.
preference_settings	json	yes	The data the user wants to save in json format. For example: preference_settings : { "Address": { "Operator": "LIKE", "value": "NYC" } }
setting_type	string	yes	The category of settings to which the client refers. For example: setting_type="UI"
is_global	boolean	yes	If "is global" is TRUE, the preference is visible to other users. If "is_global" is FALSE, the preference is visible to only the logged in user.
user_id	int32	yes	The owner of the preference. In case of global preference, the value of the user_id is id0.
client_name	string	yes	The name of the client application making use of the user preferences (for example, Credentials, CSS, Access Manager, Monitor).
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

delete user preferences

Delete the existing user preferences of the logged in user, and current application type.

REST Request URL: DELETE /api/access/onguard/openaccess/user_preferences?version=value

delete user preferences

Name	Type	Required	Description
preference_id	int32	yes	The unique identifier of the user preferences to be removed.

delete user preferences response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

Cardholders**get cardholder_from_directory**

This is an authenticated method that returns the internal ID, equivalent to Lnl_cardholder.ID, of a cardholder in the system who has a linked directory account with the directory credentials that are passed in as parameters. For more information, refer to [Lnl_Cardholder](#) on page 206.

get cardholder_from_directory

Name	Type	Required	Description
user_name	string	yes	The user's user name, in plain text.
password	string	yes	The user's password, in plain text.
directory_id	string	yes	The cardholder's directory ID, as a string. To get a list of available directory IDs, use the get directories call. For more information, refer to get directories on page 60.

get cardholder_from_directory response

Name	Type	Required	Description
cardholder_id	int32	yes	The ID of the cardholder.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

get directory_accounts

Gets directory accounts matching the provided filter.

Notes: Depending on the Active Directory server configuration, number of users in the directory, and uniqueness of the search criteria, this method might time out. Consider using the queue parameter, which allows for an asynchronous response. For more

information, refer to [Task queuing: dealing with long running requests](#) on page 53, and also refer to [get queue](#) on page 57.

Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/directory_accounts

get directory_accounts

Name	Type	Required	Description
directory_id	string	yes	Directory ID of the directory containing the active directory accounts you want to find, as a string. To get a list of available directory IDs, use the get directories call. For more information, refer to get directories on page 60.
filter	string	yes	Filter, in the format <adattr> <condition> '<value>'. For example, displayname has 'smith' • Support Conditions: eq, has. One specific case is <adattr> <eq> "", which means AD attribute's value is empty. For example, displayname eq '' • Support negative conditions: not(<adattr.> <has> '<value>') means AD attribute's value does not contain the input value. For example, not(samaccountname has 'smith') not(<adattr.> <eq> "") means AD attribute's value is not empty.

get directory_accounts_matching_cardholders

Gets directory accounts matching the given cardholders, based on the property pairs specified by the filter.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/directory_accounts_matching_cardholders

get directory_accounts_matching_cardholders

Name	Type	Required	Description
directory_id	string	yes	Directory ID of the directory containing the active directory accounts you want to find, as a string. To get a list of available directory IDs, use the get directories call. For more information, refer to get directories on page 60.
cardholder_ids	int32 array	yes	List of cardholder IDs in the format [1,2,3,...].
filter	string	yes	OData-formatted filter. Compares a directory account's attribute value with cardholder record attribute value.

Additional Filter Details

Filter format: <adattr> <condition> '<cardholderattr>'. For example, displayname has 'firstname'

Filter supports these comparison types: eq, has

Filter supports the negative condition: Therefore, not(<adattr.> <has> '<cardholderattr>') means the Active Directory attribute's value does not contain the Cardholder attribute's value. For example, not(displayname has 'lastname').

get directory_accounts_matching_cardholders response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

The returned parameters are a list of matching cardholders or non-matching cardholders. For example:

name : type : required : description

version : string : yes : used by openaccess to maintain back... etc.

successful_list : object : contains a list of successfully matched cardholders' details

successful_list.total_items : int32 : count of successfully matched cardholders

successful_list.item_list: object array : list of successfully matched cardholders' details

`successful_list.item_list.cardholder_id`: int32 : cardholder id
`successful_list.item_list.directory_account` : object : contains details about the cardholder
`successful_list.item_list.directory_account.SID` : string : SID of the matched directory user
`successful_list.item_list.directory_account.email` : string : email of the matched directory user
`successful_list.item_list.directory_account.user_name` : string : username of the matched directory user
`failure_list` : contains a list of cardholders that could not be matched to directory accounts
`failure_list.total_items` : int32 : count of failed matches
`failure_list.item_list` : object : list of failed matched cardholders
`failure_list.item_list.cardholder_id` : int32 : id of an unmatched cardholder
`failure_list.item_list.error_message` : string : reason why the match failed for this cardholder

put update_cardholder_with_directory_account_property

Updates the given cardholder with the given directory account property.

REST Request URL: PUT /api/access/onguard/openaccess/update_cardholder_with_directory_account_property

put update_cardholder_with_directory_account_property

Name	Type	Required	Description
cardholder_id	integer	yes	The ID of the cardholder to update with a directory account property.
parameter_name	JSON body	yes	JSON, in the format: <pre>{ "directory_account_property": "string", "cardholder_property": "string", "can_overwrite": true }</pre>

put update_cardholder_with_directory_account_property response

Name	Type	Required	Description
updated	boolean	yes	Indicates if the cardholder has been updated with the directory account property.

put update_cardholder_with_directory_account_property response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

Console**post console cards**

Adds a console card to all layouts, or modifies a console card in the system console layout.

REST Request URL: POST /api/access/onguard/openaccess/console/cards?version=value

post console cards

Name	Type	Required	Description
id	string	no	The ID of the console card.
group_id	string	no	The group ID to which the console card belongs.
license	string	yes	The feature license ID.
display_name	string	yes	The console card display name.
color	string	yes	The color, in HEX.
icon	string	yes	The icon content, in base64. Should start with 'data:/*;base64,'.
application_type	string	yes	Options are 'web' or 'native'.
url	string	yes	The card URL.
extended_properties	string	no	Currently empty, but in the future could contain a JSON-formatted text string to be used by the Lenel Console web application to define and store new properties to associate with a console card.
type	string	yes	The type of card. Options are 'system_default' or 'user'.

post console cards response

Name	Type	Required	Description
Session-Token	string	yes	The authentication token for the current user session.

post console cards response

Name	Type	Required	Description
Application-Id	string	yes	A unique Application-Id, provided by LenelS2 OnGuard Technical Support.
id	string	yes	The ID of the console card.
group_id	string	yes	The group ID to which the console card belongs.
license	string	yes	The feature license ID.
display_name	string	yes	The console card display name.
color	string	yes	The color, in HEX.
icon	string	yes	The icon content, in base64. Should start with 'data:*/*;base64,'.
application_type	string	yes	Options are 'web' or 'native'.
url	string	yes	The card URL.
extended_properties	string	no	Currently empty, but in the future could contain a JSON-formatted text string to be used by the Lenel Console web application to define and store new properties to associate with a console card.
type	string	yes	The type of card. Options are 'system_default' or 'user'.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

delete console cards with id

Deletes the specified console card from all layouts.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: DELETE /api/access/onguard/openaccess/console/cards?card_id=value&version=value

delete console cards with id

Name	Type	Required	Description
card_id	string	yes	The ID of the console card.

delete console cards with id response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get console layouts

Returns the specific system console layout.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being implemented by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/console/layouts?layout_id=value&version=value

get console layouts

Name	Type	Required	Description
layout_id	string	yes	The ID of the console layout.

get console layouts response

Name	Type	Required	Description
id	string	yes	The ID of the console layout.
display_name	string	yes	The console layout display name.
groups	string	yes	List of console card groups, in JSON format.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

put console layouts

Modify the existing system console layout, or add the console layout if it does not exist already.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: PUT /api/access/onguard/openaccess/console/layouts?version=value

put console layouts

Name	Type	Required	Description
id	string	no	The ID of the console layout. Add a new console layout if it is not provided.
display_name	string	yes	The console layout display name.
groups	string	yes	List of console card groups, in JSON format.

put console layouts response

Name	Type	Required	Description
console_layout_id	string	yes	The unique ID of the console layout.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

Settings

get authorization warning settings

Returns the settings for an authorization warning, as configured in System Administration.

Notes: You do not need to be logged in to make this call. A session-token and application-id are not required.

Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

If **Logon authorization warning** in System Administration is set to **None**, then the response to **get authorization_warning display_authorization_warning** is set to **false** and **authorization_warning_options** is not available.

Authorization_warning_options is a map which contains the values described in the Response table below. One property in the map is **font_properties**, which is a map of properties specific to the display font.

Some of the font properties are not directly selectable in the font dialog when setting up the font for the authorization warning in System Administration. For example, **escapement** cannot be set directly. Its value is based on other factors of the font selection. **height** is related to the font size selected, but does not map to it exactly; it often comes back negative. **weight** changes based on whether **bold** is selected or not. **face_name** is the name of the font selected. These properties come directly from the MFC LOGFONT structure. The purpose is to give a web client application all of the font information,

and then let the client figure out how to convert this information to the appropriate HTML for the client to show.

REST Request URL: GET /api/access/onguard/openaccess/settings/authorization_warning?version=value

get authorization warning settings response

Name	Type	Required	Description
display_authorization_warning	boolean	yes	Indicates if the client should display the authorization warning.
authorization_warning_options	map	no	Will not be present if display_authorization_warning is false. Contains information about how to display the warning.
authorization_warning_text	string	yes	Member of authorization_warning_options. The authorization warning text to display. Can include HTML hyperlinks.
yes_button_text	string	yes	Member of authorization_warning_options. The text to display on the Yes button.
no_button_text	string	yes	Member of authorization_warning_options. The text to display on the No button.
yes_is_default_button	boolean	yes	Member of authorization_warning_options. If true, the Yes button is the default button in the authorization warning dialog.
font_properties	map	yes	Member of authorization_warning_options. Describes the display font for the authorization warning. - height (int32) - width (int32) - escapement (int32) - orientation (int32) - weight (int32) - italic (boolean) - underline (boolean) - strikeout (boolean) - character_set (string) - out_precision (string) - clip_precision (string) - quality (string) - pitch (string) - family (string) - face_name (string)
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get cardholder settings

Returns cardholder- and badge-related settings for the system as configured in System Administration.

REST Request URL: GET /api/access/onguard/openaccess/settings/cardholder?segment_id=value&version=value

get cardholder settings

Name	Type	Required	Description
segment_id	int32	yes	Identifies the segment from which to retrieve cardholder options, and is required only if the system is segmented. For more information, refer to get segmentation settings on page 139.

get cardholder settings response

Name	Type	Required	Description
activate_deactivate_dates_use_time	boolean	no	Indicates whether or not both date and time are specified for badge activation/deactivation.
badge_pin_properties	map	no	• can_edit_pin_code (boolean): If true, a user with the appropriate permissions can change PIN values. • copy_pin_code (boolean): If true, the Copy PIN check box on the Access Level and PIN Assignment dialog is selected by default. If false, the Copy PIN check box is not selected by default. For more information, refer to <i>Add or Replace a Badge Record</i> in the System Administration User Guide. • digits (int32): Indicates the number of digits the PIN contains. • enforce_unique_pin_code (boolean): If true, indicates that the cardholder badge record must have a unique PIN code. If false, duplicate PIN codes are allowed. • generate_pin_code (boolean): If true, indicates whether a PIN is randomly generated when a badge is created. If false, a PIN must be manually entered.
create_photo_thumbnails	boolean	no	Indicates whether or not thumbnail versions for all existing cardholder photos are saved in the database.

get cardholder settings response

Name	Type	Required	Description
max_accesslevels_per_badge_standard	int32	no	Indicates the maximum number of standard access levels that can be assigned to a badge at one time. For Lenel access panels, the maximum number is 128. Dependent on the segment_id property, if segmentation is enabled.
max_accesslevels_per_badge_temporary	int32	no	Indicates the maximum number of temporary access levels that can be assigned to a badge at one time. For Lenel access panels, the maximum number is 128. Dependent on the segment_id property, if segmentation is enabled.
max_accesslevels_per_badge_total	int32	no	Indicates the maximum number of access levels that can be assigned to a badge at one time. This includes both standard and temporary access levels. For Lenel access panels, the maximum number is 128. Dependent on the segment_id property, if segmentation is enabled.
max_active_badges	int32	no	Indicates the maximum number of active badges that are allowed for each cardholder.
max_badge_id_length	int32	no	Indicates the maximum number of digits in a badge number. For Lenel access panels, the maximum length is 18 digits. Dependent on the segment_id property, if segmentation is enabled.
max_extended_id_length	int32	no	Indicates the maximum extended ID length if extended identifiers are used (64 bits long). For Lenel access panels, the maximum length is 32 bytes. Dependent on the segment_id property, if segmentation is enabled.
temporary_accesslevel_granularity	int32	no	Indicates how frequently the Linkage Server examines and updates temporary access levels for date and time badge activation and deactivation purposes.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get enterprise settings

Returns enterprise-related settings for the system as configured in System Administration, if Enterprise support is enabled.

REST Request URL: GET /api/access/onguard/openaccess/settings/enterprise?version=value

get enterprise settings response

Name	Type	Required	Description
default_cardholder_replication	int32	yes	The value in this property indicates where the cardholder record gets replicated. This property is not available on a Master server. Returns a value that matches one of the items in the server_list property as the database_id.
default_user_replication	int32	yes	The value in this property indicates where a user record gets replicated. Returns a value that matches one of the items in the server_list property as the database_id.
default_visitor_replication	int32	yes	The value in this property indicates where the visitor record gets replicated. This property is not available on a Master server. Returns a value that matches one of the items in the server_list property as the database_id.
is_enterprise_system	boolean	yes	Identifies whether or not this is an OnGuard Enterprise system.
is_master_server	boolean	yes	Identifies whether or not this machine is the Master server in an OnGuard Enterprise system.
local_database_id	int32	yes	Identifies the id of this Enterprise server.
server_list	list	yes	All Enterprise servers of the Enterprise system. A list that will return database_id, display_name, and server_type of each server.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get password policy settings

Returns the password policy settings for the system.

REST Request URL: GET /api/access/onguard/openaccess/settings/password_policy?version=value

get password policy settings response

Name	Type	Required	Description
is_lockout_policy_enabled	boolean	yes	A flag indicating whether the lockout policy is enabled.
login_attempt_threshold	int32	yes	The number of invalid login attempts that will lock an internal account.
login_attempt_reset_interval_in_minutes	int32	yes	The number of minutes to wait before resetting the record of invalid logins.
lockout_interval_in_minutes	int32	yes	The number of minutes to lock an internal account after exceeding the invalid login attempt threshold.
disable_lockout_for_sa	boolean	yes	Supports disabling the lockout policy for the SA user and SA delegate users.
is_expiration_policy_enabled	boolean	yes	A flag indicating whether the expiration policy is enabled.
expiration_days	int32	yes	The number of days the password will be expired.
is_expiration_reminders_enabled	boolean	yes	A flag indicating whether to remind the user if the password is almost expired.
expiration_first_reminder_days	int32	yes	The first day to remind the user that the password is almost expired.
expiration_reminder_days	int32	yes	The day to start reminding the user with each login that the password is almost expired.
is_minimum_length_required	boolean	yes	A flag indicating whether a minimum password length is required.
minimum_length	int32	yes	The minimum password length.
is_numeric_characters_required	boolean	yes	A flag indicating whether the password must contain a numeric character.
is_special_characters_required	boolean	yes	A flag indicating whether the password must contain a non-alphanumeric character.
is_upper_and_lower_case_required	boolean	yes	A flag indicating whether the password must contain an uppercase alphabetic and a lowercase alphabetic character.
is_history_policy_enabled	boolean	yes	A flag indicating whether the password history policy is enabled.

get password policy settings response

Name	Type	Required	Description
history_password_count	int32	yes	The number of previous passwords that will be prohibited when resetting the password.
minimum_password_age	int32	yes	Determines how long users must keep a password before they can change it.
is_prohibited_password_policy_enabled	boolean	yes	A flag indicating whether the prohibited password policy is enabled.
is_inactivity_timeout_policy_enabled	boolean	yes	A flag indicating whether the inactivity timeout policy is enabled.
inactivity_timeout_in_minutes	int32	yes	The authenticated token inactivity timeout, in minutes.
can_be_same_as_user_name	boolean	yes	A flag indicating whether the password can be the same as the user name.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

put password policy settings

Updates the password policy settings for the system.

REST Request URL: PUT /api/access/onguard/openaccess/settings/password_policy?version=value

put password policy settings

Name	Type	Required	Description
can_be_same_as_user_name	boolean	no	A flag indicating whether the password can be the same as the user name. Default = FALSE
disable_lockout_for_sa	boolean	no	Supports disabling the lockout policy for the SA user and SA delegate users. Default = FALSE
expiration_days	int32	no	The number of days the password will be expired. Default = 90 Minimum = 0 Maximum = 730

put password policy settings

Name	Type	Required	Description
expiration_first_reminder_days	int32	no	The first day to remind the user that the password is almost expired. Default = 15 Minimum = expiration_reminder_days Maximum = expiration_days
expiration_reminder_days	int32	no	The day to start reminding the user with each login that the password is almost expired. Default = 7 Minimum = 0 Maximum = expiration_days
history_password_count	int32	no	The number of previous passwords that will be prohibited when resetting the password. Default = 3 Minimum = 0 Maximum = 24
inactivity_timeout_in_minutes	int32	no	The authenticated token inactivity timeout, in minutes. Default = 15 Minimum = 1 Maximum = authenticated_token_timeout configured in openaccess.ini
is_expiration_policy_enabled	boolean	no	A flag indicating whether the expiration policy is enabled. Default = TRUE
is_expiration_reminders_enabled	boolean	no	A flag indicating whether to remind the user if the password is almost expired. Default = FALSE
is_history_policy_enabled	boolean	no	A flag indicating whether the password history policy is enabled. Default = TRUE
is_inactivity_timeout_policy_enabled	boolean	no	A flag indicating whether the inactivity timeout policy is enabled. Default = TRUE
is_lockout_policy_enabled	boolean	no	A flag indicating whether the lockout policy is enabled. Default = TRUE
is_minimum_length_required	boolean	no	A flag indicating whether a minimum password length is required. Default = TRUE
is_numeric_characters_required	boolean	no	A flag indicating whether the password must contain a numeric character. Default = TRUE

put password policy settings

Name	Type	Required	Description
is_prohibited_password_policy_enabled	boolean	no	A flag indicating whether the prohibited password policy is enabled. Default = TRUE
is_special_characters_required	boolean	no	A flag indicating whether the password must contain a non-alphanumeric character. Default = TRUE
is_upper_and_lower_case_required	boolean	no	A flag indicating whether the password must contain an uppercase alphabetic and a lowercase alphabetic character. Default = TRUE
lockout_interval_in_minutes	int32	no	The number of minutes to lock an internal account after exceeding the invalid login attempt threshold. Default = 5 Minimum = 1 Maximum = 99999
login_attempt_threshold	int32	no	The number of invalid login attempts that will lock an internal account. Default = 3 Minimum = 1 Maximum = 999
login_attempt_reset_interval_in_minutes	int32	no	The number of minutes to wait before resetting the record of invalid logins. Default = 60 Minimum = 1 Maximum = 99999
minimum_length	int32	no	The minimum password length. Default = 8 Minimum = 1 Maximum = 127
minimum_password_age	int32	no	Determines how many days a users must keep a password before they can change it. Default = 0 Minimum = 0 Maximum = 7

put password policy settings response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get segmentation settings

Returns the segmentation-related settings of the system as configured in System Administration. The information returned in the response of this call identifies which SEGMENTID properties or classes are shown in OpenAccess. For more information, refer to [Chapter 6: Data and Association Class Reference](#) on page 181.

Note: For more information about segmentation settings, refer to “Segment Options Form” in the *System Administration User Guide*.

REST Request URL: GET /api/access/onguard/openaccess/settings/segmentation?version=value

get segmentation settings response

Name	Type	Required	Description
allow_access_levels_to_be_configured_as_assignable_by_other_segments	boolean	yes	Identifies if users in other segments can configure this segment's access levels.
allow_segment_to_belong_to_multiple_groups	boolean	yes	Identifies if this segment can belong to more than one segment group.
segment_badge_types	boolean	yes	Identifies if badge type segmentation is enabled.
segment_card_formats	boolean	yes	Identifies if card format segmentation is enabled.
segment_cardholders	boolean	yes	Identifies if cardholders are segmented.
segment_non_system_list_builder_lists	boolean	yes	Identifies if non-system List Builder entries are segmented.
segment_visitors	boolean	yes	Identifies if visitors are segmented.
segmentation_enabled	boolean	yes	Identifies if segmentation is enabled.

get segmentation settings response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

For more information, refer to “Segment Options Form” in the *System Administration User Guide*.

get visit settings

Gets the visit settings of the system.

Note: Version 1.1 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser. For more information, refer to [Preventing Malicious Code in OpenAccess Responses](#) on page 49.

REST Request URL: GET /api/access/onguard/openaccess/settings/visit?version=value

get visit settings response

Name	Type	Required	Description
default_visitor_badge_type_id	string	yes	The unique identifier of the default visitor badge type.
default_visitor_badge_type_name	string	yes	The name of the default visitor badge type.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : "1.0". For more information, refer to Version on page 49.

put visit settings

Modifies the visit settings of the system.

REST Request URL: PUT /api/access/onguard/openaccess/settings/visit?version=value

put visit settings

Name	Type	Required	Description
VisitSettings	JSON body	yes	The visit settings, in JSON format.
default_visitor_badge_type_id	int32	yes	The internal database ID of the default visitor badge type.

put visit settings response

Name	Type	Required	Description
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get video settings

Retrieves video settings for the Video Tile.

REST Request URL: GET /api/access/onguard/openaccess/settings/video?version=value

get video settings response

Name	Type	Required	Description
instant_rewind_time_in_seconds	int32	yes	Determines the amount of time from the current time, in seconds, to rewind the video for playback. This value is configured on the System Administration > Administration > System Options > Video Options tab. Default value = 30.
time_limited_playback_restriction_in_minutes	int32	yes	Determines the amount of time from the current time, in minutes, that the user is allowed to play back recorded video. For more information, refer to "Limit Video Viewing to Time-limited Playback Only" in the <i>System Administration User Guide</i> . Default value = 480.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

Maps

get maps

Retrieves a page of maps from the OnGuard database.

Note: Version 1.0 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser.

REST Request URL: GET /api/access/onguard/openaccess/maps?version=value

get maps

Name	Type	Required	Description
map_filter	string	no	The filter based on map properties. The clause text used to count only those instances that match a given attribute. For example, name like "%MAP%". Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an InvalidQuery error. This parameter supports filtering with the following properties: - map_id - name - width - height
map_device_filter	string	no	The filter based on map device properties. The clause text used to count only those instances that match a given attribute. For example, device_id=1 and panel_id=1. Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an InvalidQuery error. This parameter supports filtering with the following properties: - map_item_id - panel_id - device_id - input_device_id - pos_x - pos_y - pos_z - icon_id - icon_group_id - custom_text
page_number	int32	no	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.

get maps

Name	Type	Required	Description
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
order_by	string	no	A field- or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by map_id. Fields must be valid properties of the requested object type. This parameter supports sorting with the following properties: - map_id - name - width - height

get maps response

Name	Type	Required	Description
item_list	list	yes	A list of maps. Each map in the list has following properties: - map_id - name - width - height - device_count
map_id	int32	yes	The internal database ID of the map. Key field.
name	string	yes	The display name of the map.
width	int32	yes	The width of the map.
height	int32	yes	The height of the map.
device_count	int32	yes	The device count in the map.
count	int32	yes	The number of maps returned.
page_number	int32	yes	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.

get maps response

Name	Type	Required	Description
page_size	int32	yes	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total number of instances of the object being requested.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

get map devices

Retrieves a page of map devices from the OnGuard database.

Note: Version 1.0 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser.

REST Request URL: GET /api/access/onguard/openaccess/maps/{id}/devices?version=value

get map devices

Name	Type	Required	Description
id	int32	yes	The ID of the map to retrieve its devices.

get map devices

Name	Type	Required	Description
filter	string	no	The filter based on map device properties. The clause text used to count only those instances that match a given attribute. For example, <code>device_id=1</code> and <code>panel_id=1</code>. Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an <code>InvalidQuery</code> error. This parameter supports filtering with the following properties: • <code>map_item_id</code> • <code>panel_id</code> • <code>device_id</code> • <code>input_device_id</code> • <code>pos_x</code> • <code>pos_y</code> • <code>pos_z</code> • <code>icon_id</code> • <code>icon_group_id</code> • <code>custom_text</code>
page_number	int32	no	The page number to return when a subset (page) of instances is requested. Used in conjunction with <code>page_size</code>. Defaults to the first page (1) if not provided, and if provided, must be numeric.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with <code>page_number</code>. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (<code>page_size</code>) that can be retrieved with a single request is 100.

get map devices

Name	Type	Required	Description
order_by	string	no	A field- or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by map_item_id. Fields must be valid properties of the requested object type. This parameter supports sorting with the following properties: - map_item_id - panel_id - device_id - input_device_id - pos_x - pos_y - pos_z - icon_id - icon_group_id - custom_text

get map devices response

Name	Type	Required	Description
item_list	list	yes	A list of map devices. Each map device in the list has following properties: - map_item_id - panel_id - device_id - input_device_id - pos_x - pos_y - pos_z - icon_id - icon_group_id - custom_text - label_color - font_size - font_weight - font_face - is_italic - is_strikeout - is_underline - label_text - label_width - label_height - label_alignment
map_item_id	int32	yes	The internal database ID of the map device. Key field.

get map devices response

Name	Type	Required	Description
panel_id	int32	yes	The ID of the panel.
device_id	int32	yes	The ID of the device.
input_device_id	int32	yes	The ID of the input device.
pos_x	int32	yes	The icon's X position.
pos_y	int32	yes	The icon's Y position.
pos_z	int32	yes	The icon's Z position.
map_item_type	int32	yes	The type of map item.
icon_id	int32	yes	The ID of the icon.
icon_group_id	int32	yes	The ID of the icon group.
custom_text	string	yes	The custom text.
label_color	string	yes	The label color, in HEX format.
font_size	int32	yes	The font size.
font_weight	int32	yes	The font weight.
font_face	string	yes	The font face name.
is_italic	boolean	yes	Indicates if the label font style is italic.
is_strikeout	boolean	yes	Indicates if the label is shown with strikeout marks.
is_underline	boolean	yes	Indicates if the label is shown underlined.
label_text	string	no	The label text. Only available when this map item is text.
label_width	int32	no	The label width. Only available when this map item is text.
label_height	int32	no	The label height. Only available when this map item is text.
label_alignment	int32	no	The label alignment. Only available when this map item is text.
count	int32	yes	The number of map devices returned.
page_number	int32	yes	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.

get map devices response

Name	Type	Required	Description
page_size	int32	yes	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total number of instances of the object being requested.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format " version " : " 1.0 ". For more information, refer to Version on page 49.

get map icon groups

Retrieves a page of map icon groups from the OnGuard database.

Note: Version 1.0 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser.

REST Request URL: GET /api/access/onguard/openaccess/map_icon_groups?version=value

get map icon groups

Name	Type	Required	Description
filter	string	no	The filter based on map icon group properties. The clause text used to count only those instances that match a given attribute. For example, name like "%AAA%". Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an InvalidQuery error. This parameter supports filtering with the following properties: - icon_group_id - name - map_item_type

get map icon groups

Name	Type	Required	Description
page_number	int32	no	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
order_by	string	no	A field- or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by icon_group_id. Fields must be valid properties of the requested object type. This parameter supports sorting with the following properties: - icon_group_id - name - map_item_type

get map icon groups response

Name	Type	Required	Description
item_list	list	yes	A list of map icon groups. Each map icon group in the list has following properties: - icon_group_id - name - map_item_type - map_item_group_members
icon_group_id	int32	yes	The internal database ID of the map icon group. Key field.
name	string	yes	The display name of the map icon group.

get map icon groups response

Name	Type	Required	Description
map_item_type	int32	yes	The type of map item. Possible values: • Panel = 1 • Reader = 2 • Alarm panel = 3 • Input = 4 • Output = 5 • Map = 6 • Text = 7 • Alarm mask group = 8 • Function list = 9 • Reader auxiliary input = 10 • Reader auxiliary output = 11 • Reader group = 12 • Input group = 13 • Output group = 14 • CCTV panel = 15 • Fire panel = 16 • CCTV camera = 17 • CCTV monitor = 18 • Fire device = 19 • Intercom = 20 • Fire input/output = 21 • Intercom panel = 22 • Personal safety panel = 23 • Camera group = 24 • Off-line lock panel = 25 • Switcher panel = 26 • PC panel = 27 • Receiver = 28 • Receiver account = 29 • Account zone = 30 • Muster zone = 31 • Safe location = 32 • Normal anti-passback area = 33 • Intrusion panel = 34 • Intrusion zone = 35 • Intrusion on relay = 36 • Intrusion off relay = 37 • Intrusion door = 38 • Intrusion area = 39 • Intercom station input = 40 • Intercom station output = 41 • Intercom station door = 42 • Action group = 43 • SNMP manager = 44 • SNMP agent = 45

get map icon groups response

Name	Type	Required	Description
map_item_type (continued)	int32	yes	• Open Platform Communications (OPC) connection = 46 • OpenIT source= 47 • OPC source = 48 • Point of Sale (POS) panel = 52 • POS register = 53 • CCTV IVS = 54 • CCTV IVS application = 55 • CCTV camera input = 56 • CCTV camera output = 57 • CCTV video monitor = 58 • Video monitor group = 59 • Elevator panel = 60 • Elevator keypad terminal = 61 • OpenIT device = 62 • OpenIT sub-device = 63 • Power supply = 64; • Intrusion reader = 67 • Intrusion RAS = 68 • Intrusion DGP = 69
map_item_group_members	list	yes	A list of map icon group members. Each map icon group member in the list has following properties: • map_item_state • icon_id See below for those properties.
map_item_state	int32	yes	The map item state.
icon_id	int32	yes	The map icon ID.
count	int32	yes	The number of map icon groups returned.
page_number	int32	yes	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.
page_size	int32	yes	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.

get map icon groups response

Name	Type	Required	Description
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total number of instances of the object being requested.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

get map icons

Retrieves a page of map icons from the OnGuard database.

Note: Version 1.0 and later of this method supports string encoding, which is helpful in preventing malicious code from being run by the client browser.

REST Request URL: GET /api/access/onguard/openaccess/map_icons?version=value

get map icons

Name	Type	Required	Description
filter	string	no	The filter based on map icon properties. The clause text used to count only those instances that match a given attribute. For example, name like "%AAA%". Note: You must use double-quotes around string delimiters when filtering. Single-quotes will result in an InvalidQuery error. This parameter supports filtering with the following properties: - icon_id - name
page_number	int32	no	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.

get map icons

Name	Type	Required	Description
page_size	int32	no	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
order_by	string	no	A field- or comma-separated list of fields to use for sorting the instances when performing paging. If not provided, results are ordered by icon_id. Fields must be valid properties of the requested object type. This parameter supports sorting with the following properties: - icon_id - name

get map icons response

Name	Type	Required	Description
item_list	list	yes	A list of map icons. Each map icon in the list has following properties: - icon_id - name - blob
icon_id	int32	yes	The internal database ID of the map icon. Key field.
name	string	yes	The display name of the map icon.
blob	string	yes	The data of the map icon. Converted to base64-encoded string.
count	int32	yes	The number of map icons returned.
page_number	int32	yes	The page number to return when a subset (page) of instances is requested. Used in conjunction with page_size. Defaults to the first page (1) if not provided, and if provided, must be numeric.

get map icons response

Name	Type	Required	Description
page_size	int32	yes	The page size, or number of instances per page, to be returned when a subset (page) of instances is requested. Used in conjunction with page_number. Defaults to 20 if not provided, and if provided, must be numeric. For performance reasons, paging is always performed, and the maximum number of instances (page_size) that can be retrieved with a single request is 100.
total_pages	int32	yes	The total number of pages, given the existing number of instances (total_items) and the page_size being used.
total_items	int32	yes	The total number of instances of the object being requested.
version	string	yes	Used by OpenAccess to maintain backward compatibility as the API is updated. Required string, in the format "version" : "1.0". For more information, refer to Version on page 49.

Events can be received using the Web Event Bridge. The Web Event Bridge is a SignalR server running at **/api/access/onguard/openaccess/eventbridge**, which provides a hub named “Outbound”. Because the Web Event Bridge is a SignalR server, it is easiest to use one of the SignalR client APIs. There are SignalR client APIs for C# and JavaScript, and there are sample event subscriber applications provided for both. For help writing SignalR clients, refer to <http://www.asp.net/signalr/overview/guide-to-the-api/hubs-api-guide-net-client> and <http://www.asp.net/signalr/overview/guide-to-the-api/hubs-api-guide-javascript-client>.

Note: The latest version of the SignalR core server component and client component is version 2.4.1. The server component is currently using version 2.4.0. It is suggested that the client use either version 2.4.0 or 2.4.1.

Web Event Bridge Operations

CreateSubscription

Creates a subscription and starts receiving events using the **OnBusinessEventReceived** event handler.

Method Signature

CreateSubscription(security : object, eventSubscription : object) : object

For a list of inputs and outputs, refer to [add event_subscriptions](#) on page 69.

Parameters

Name	Type	Required	Description
security	object	yes	An object containing the session token and application ID properties for the OpenAccess API.
security.SessionToken	string	yes	An authenticated OpenAccess session token. To authenticate with a session cookie ('OASession'), pass the SessionToken field as an empty string, and the session cookie must be submitted with the CreateSubscription request. For more information, refer to notes beneath this table. To authenticate with a Session-Token header, simply pass the session identifier in the SessionToken field. For more information, refer to notes beneath this table. Note: Depending upon the JavaScript API library that you use, the credentials might be included automatically. If not, you must explicitly include the cookie credentials with the OpenAccess API request.
security.ApplicationId	string	yes	An OpenAccess application ID.
eventSubscription	object	yes	An object containing the event subscription parameters.
eventSubscription.description	string	no	An optional description for the event subscription.
eventSubscription.filter	string	no	An optional filter for the event subscription.

Notes: The message body associated with the CreateSubscription request takes the following form depending upon the authentication mechanism:

Cookie

```
{"H":"outbound","M":"CreateSubscription","A":[{"ApplicationId":
"<APPLICATION-ID>","SessionToken":"",""}], "I":0}
```

SessionToken header

```
{"H":"outbound","M":"CreateSubscription","A":[{"ApplicationId":
"<APPLICATION-ID>","SessionToken":"<SESSION-IDENTIFIER>"}, {""}], "I":0}
```

Return Value

The created event subscription.

Name	Type	Required	Description
Id	int32	yes	The unique subscription id.
user_id	string	yes	The ID of the user who owns the subscription.
name	string	yes	The unique name of the subscription.
description	string	yes	A description of the subscription.
filter	string	yes	This optional parameter filters the events that will be received. If no filter is specified, all events will be forwarded to the subscriber. For more information, refer to Using Event Filters with Subscriptions on page 43.
is_durable	boolean	yes	Indicates if this is a durable subscription.
message_broker_hostname	string	yes	The hostname of the message broker where the events will be published.
message_broker_port	int32	yes	The port of the message broker where the events will be published.
requires_secure_connection	boolean	yes	Indicates if an SSL connection should be opened by the message broker where the events will be published.
exchange_name	string	yes	The exchange name on the message broker where the events will be published.
binding_key	string	yes	The unique binding key with which the events will be published on the exchange.
queue_name	string	yes	The unique queue name where the events will be published if the subscription is durable.
created_date	datetime (string)	yes	The time when the subscription was created.
last_updated_date	datetime (string)	yes	The time when the subscription was last updated.

ModifySubscription

Modifies a subscription and starts receiving events using the **OnBusinessEventReceived** event handler.

Method Signature

ModifySubscription(security : object, eventSubscription : object) : object

Parameters

Name	Type	Required	Description
security	object	yes	An object containing the session token and application ID properties for the OpenAccess API.
security.SessionToken	string	yes	An authenticated OpenAccess session token.
security.ApplicationId	string	yes	An OpenAccess application ID.
eventSubscription	object	yes	An object containing the event subscription parameters.
eventSubscription.description	string	no	An optional description for the event subscription.
eventSubscription.filter	string	no	An optional filter for the event subscription.

Return Value

The modified event subscription.

Name	Type	Required	Description
Id	int32	yes	The unique subscription id.
user_id	string	yes	The ID of the user who owns the subscription.
name	string	yes	The unique name of the subscription.
description	string	yes	A description of the subscription.
filter	string	yes	This optional parameter filters the events that will be received. If no filter is specified, all events will be forwarded to the subscriber. For more information, refer to Using Event Filters with Subscriptions on page 43.
is_durable	boolean	yes	Indicates if this is a durable subscription.
message_broker_hostname	string	yes	The hostname of the message broker where the events will be published.
message_broker_port	int32	yes	The port of the message broker where the events will be published.

Name	Type	Required	Description
requires_secure_connection	boolean	yes	Indicates if an SSL connection should be opened by the message broker where the events will be published.
exchange_name	string	yes	The exchange name on the message broker where the events will be published.
binding_key	string	yes	The unique binding key with which the events will be published on the exchange.
queue_name	string	yes	The unique queue name where the events will be published if the subscription is durable.
created_date	datetime (string)	yes	The time when the subscription was created.
last_updated_date	datetime (string)	yes	The time when the subscription was last updated.

StopSubscription

Stops receiving events using the **OnBusinessEventReceived** event handler. Also deletes the subscription if it is transient.

Method Signature

StopSubscription()

StartManaging

Starts receiving management messages using the **OnManagementEvent** event handler.

Method Signature

StartManaging(agentName : string)

Parameters

Name	Type	Required	Description
agentName	string	yes	A name to use for the management agent.

StopManaging

Stops receiving management messages using the **OnManagementEvent** event handler.

Method Signature

StopManaging()

ConnectionHeartbeat

Tests the connection, and re-subscribes events if LS Web Event Bridge service is restarted.

Method Signature

ConnectionHeartbeat(security : object, eventSubscription : object) : object

Parameters

Name	Type	Required	Description
security	object	yes	An object containing the session token and application ID properties for the OpenAccess API.
security.SessionToken	string	yes	An authenticated OpenAccess session token.
security.ApplicationId	string	yes	An OpenAccess application ID.
eventSubscription	object	yes	An object containing the event subscription parameters.
eventSubscription.id	int	yes	The unique subscription ID.
eventSubscription.description	string	no	An optional description for the event subscription.
eventSubscription.filter	string	no	An optional filter for the event subscription.

Web Event Bridge Client Event Handlers

Notes: If developing your own application, using WebSockets as the transport improves performance. To do this, target .NET Framework 4.6.1 or later instead of .NET Framework 4.0, as shown in this sample application. WebSockets functions correctly with any version of Windows supported by the OnGuard software.

When the LS Web Event Bridge service is restarted, it loses subscription details for all existing clients. Therefore, clients must re-subscribe to continue receiving events. New transient subscriptions must be created, but durable subscriptions can be re-established with the **ModifySubscription** call ([ModifySubscription](#) on page 157).

If not using WebSockets, there is a limitation within SignalR where the client will not be notified that the LS Web Event Bridge service has restarted. In this case, the client will not know to re-subscribe. This limitation does not exist when using WebSockets.

OnBusinessEventReceived

Called when an event is received.

Event Handler Signature

OnBusinessEventReceived(businessEvent : object)

Parameters

Name	Type	Required	Description
businessEvent	object	yes	The business event, with the properties specific to the given event type. For more information, refer to Hardware Event Reference on page 162, Alarm Acknowledgment Activity Event Reference on page 173, and Software Event Reference on page 174.

OnExceptionRaised

Called when an exception is raised.

Event Handler Signature

OnExceptionRaised(message : string)

Parameters

Name	Type	Required	Description
message	string	yes	The error message describing the exception.

OnConnectionFromMessageBusLost

Called when the connection to the message bus is lost.

Event Handler Signature

OnConnectionFromMessageBusLost()

OnConnectionToMessageBusEstablished

Called when the connection to the message bus is established.

Event Handler Signature

OnConnectionToMessageBusEstablished()

OnManagementEvent

Called when a management event is received.

Event Handler Signature

OnManagementEvent(message : string)

Parameters

Name	Type	Required	Description
message	string	yes	The management message. For example: "Updated Transient subscription 123. Client Id 7ffb8f0a-c38e-41c4-aaad-6e7eaa7f4d24".

Hardware Event Reference

In OnGuard, events generally originate in the access control hardware and are displayed in Alarm Monitoring. An example is when a reader grants access to a cardholder.

This chapter includes the different categories of events, as well as properties that are common to all events, as included in the following table.

Notes: If an event contains an ID for an item that does not exist in the database, the fields relating to that item are not included in the event. For example, if an access denied event is received with a badge ID of 4, but there is no badge with an ID of 4 in the database, there will be no badge or cardholder properties included in that event.

For a complete list of event types and subtypes, perform a **get instances** call on `Lnl_EventType` and `Lnl_EventSubtypeDefinition`. For more information, refer to [get instances](#) on page 80, [Lnl_EventType](#) on page 212, and [Lnl_EventSubtypeDefinition](#) on page 211.

Common Properties for All Hardware Events

Property	Type	Description
alarm_ack_blue_channel	int16	The blue component of the RGB color for the alarm after it is acknowledged (0 to 255).
alarm_ack_green_channel	int16	The green component of the RGB color for the alarm after it is acknowledged (0 to 255).
alarm_ack_red_channel	int16	The red component of the RGB color for the alarm after it is acknowledged (0 to 255).
alarm_active_alarm	boolean	True if this alarm is configured as active, meaning that Alarm Monitoring clients should highlight alarms of this type when they occur.
alarm_aggregate_alarm	boolean	True if this alarm is to be aggregated, meaning that Alarm Monitoring clients should combine all alarms of this type into a single alarm for display purposes.
alarm_blue_channel	int16	The blue component of the RGB color for the alarm (0 to 255).
alarm_change_response	boolean	True if the operator is allowed to change the information provided when acknowledging this alarm type.
alarm_display_alarm	boolean	True if this alarm should be displayed.

Common Properties for All Hardware Events (Continued)

Property	Type	Description
alarm_display_map	boolean	True if a map containing the location of this alarm should be displayed automatically.
alarm_do_not_delete_on_acknowledge	boolean	True if this alarm should not be deleted from the client view after it is acknowledged.
alarm_green_channel	int16	The green component of the RGB color for the alarm (0 to 255).
alarm_login_required_for_acknowledge	boolean	True if the operator is required to log in when acknowledging this type of alarm.
alarm_must_acknowledge	boolean	True if this alarm must be acknowledged before it can be deleted.
alarm_must_mark_in_progress	boolean	True if this alarm must be marked In Progress before it can be deleted.
alarm_print_alarm	boolean	True if this alarm should be printed.
alarm_priority	int16	Alarm priority (0 to 255).
alarm_red_channel	int16	The red component of the RGB color for the alarm (0 to 255).
alarm_response_required	boolean	True if notes are required when acknowledging this alarm.
alarm_show_cardholder	boolean	True if the cardholder view should be shown for this type of alarm.
alarm_video_verify	boolean	True if the video verification view should be shown for this type of alarm.
alarm_visual_notification	boolean	True if the occurrence of this alarm type should be highlighted by, for example, bringing the main alarm monitor window to the foreground.
associated_text	string	Optional text that provides additional information about an event.
business_event_class	string	Type of event. Will always be hardware_event.
device_name	string	Name of the device that is the source of the event.
domain	string	The source domain of an event.
event_parameter	uint32	A parameter that provides additional information about an event.
event_subtype	uint16	A subtype of a class of events defined in the system.
event_type	uint8	A class of events defined in the system and reported by the API that can be further broken down into subtypes. For example, 0 indicates an access granted event and 1 indicates an access denied event.
initiating_event_id	int32	The ID of a previous event that caused the event.

Common Properties for All Hardware Events (Continued)

Property	Type	Description
segment_id	uint32	The segment ID of the source of an event, if segmentation is enabled in the system. Otherwise, the value is null.
source	string	The source of the event encoded in a domain-specific manner as a URI string. For example, a source defined as a UUID should be encoded as <i>urn:uuid:7673868d-231e-490d-9c4f-19288e7e668d</i> . For more examples, visit: http://example.org/absolute/URI/with/absolute/path/to/resource.txt
timestamp	int64	The time when the event occurred at its source, following the AMQP standard of milliseconds since January 1, 1970 in UTC time.
version	string	The version of this specific event message type. This is a period-delimited string in the format <i><major>.<minor></i> . - A <i>minor</i> version change is one in which only fields were added, and a parser that ignores unrecognized fields can still process the message. - A <i>major</i> version change is one in which the message structure has changed in a manner that is not backwards compatible with the previous structure. Version is managed on a per event type basis, not the version of the application that sent the message. A specific event type is uniquely identified using the ordered list of domain, event type, and version.

The following properties are delivered for controller-based events, which are events for devices that are either controllers or have a root parent device that is a controller:

Properties for Controller-Based Events

Property	Type	Description
alarm_id	int32	ID for the alarm.
alarm_name	string	Name of the alarm.
controller_id	uint16	The ID of the controller for the device that is the source of an event.
controller_name	string	Name of the controller to which the device or subdevice is connected. May also refer to the controller itself.
device_id	uint16	The ID of the device that is the source of an event. A value of 0 indicates that the source of the event is a controller.
device_type	int8	The type of device that generated an event.
event_parameter_description	string	The description of the event parameter. Note: This value may be included for events that convey additional information.

Properties for Controller-Based Events

Property	Type	Description
event_source_name	string	The name of the device that generated the event.
controller_time_zone_id	uint16	The time zone where the controller is located.
serial_number	int32	The serial number of the event, as specified by the controller.
subdevice_id	uint16	The ID of the subdevice of a device that is the source of the event. A value of 0 indicates that the source is a device or a controller.
timestamp_processed	int64	The time when the event was processed by the Communication Server, following the AMQP standard of milliseconds since January 1, 1970 in UTC time.

Access Granted Events

When an Access Granted event occurs, subscribers with proper authorization receive the following properties and their values:

Properties for Access Granted Events

Property	Type	Description
access_granted_entry_made	boolean	Indicates if entry was made through the door. Value Range: True, False
area_entering_id	int32	The ID of the area that a cardholder entered, if the corresponding reader is defined to detect when an area is entered.
area_entering_name	string	The name of the area that a cardholder entered.
area_exiting_id	int32	The ID of the area that a cardholder exited, if the corresponding reader is defined to detect when an area is exited.
area_exiting_name	string	The name of the area that a cardholder exited.
badge_extended_id	string	The full Federal Agency Smart Credential Number (FASC-N) or full UUID from a Personal Identity Verification (PIV)-based card or other Federal Information Processing Standard (FIPS) 201-based card.
badge_id	int64	The ID encoded on a badge.
badge_id_str	string	A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the ID property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off.
badge_issue_code	uint32	The issue code of the badge.
badge_key	int64	The database record ID of the badge.

Properties for Access Granted Events

Property	Type	Description
badge_key_str	string	A string representation of the badge key. To accurately display badge key, web clients should use this property instead of the badge_key property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off.
badge_status_name	string	The status of the badge, which must be “Active” if access was granted.
badge_type_name	string	The cardholder's badge type, as configured in System Administration.
cardholder_first_name	string	The cardholder's first name, as configured in System Administration.
cardholder_key	int32	The database record ID, which is not displayed in System Administration, but which can be useful when developing custom scripts.
cardholder_last_name	string	The cardholder's last name, as configured in System Administration.
controller_segment_id	int32	The ID of the controller segment.
event_parameter	int32	A parameter that provides additional information about an event.
event_parameter_description	string	The description of the event parameter. Note: This value may be included for events that convey additional information.

Access Denied Events

When an Access Denied event occurs, subscribers with proper authorization receive the following properties and their values:

Properties for Access Denied Events

Property	Type	Description
badge_id	int64	The ID encoded on a badge.
badge_id_str	string	A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the ID property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off.
badge_issue_code	uint32	The issue code of the badge.
badge_key	int64	The database record ID of the badge.
badge_key_str	string	A string representation of the badge key. To accurately display badge key, web clients should use this property instead of the badge_key property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off.

Properties for Access Denied Events

Property	Type	Description
badge_status_name	string	The status of the badge.
badge_type_name	string	The cardholder's badge type, as configured in System Administration.
cardholder_first_name	string	The cardholder's first name, as configured in System Administration.
cardholder_key	int32	The database record ID, which is not displayed in System Administration, but which can be useful when developing custom scripts.
cardholder_last_name	string	The cardholder's last name, as configured in System Administration.

Area Control Events

When an Area Control event occurs, subscribers with proper authorization receive the following properties and their values:

Property for Area Control Events

Property	Type	Description
area_apb_id	int32	The name of an APB area where an event occurred.

Asset Events

When an Asset event occurs, subscribers with proper authorization receive the following properties and their values:

Properties for Asset Events

Property	Type	Description
asset_id	string	The ID of the asset that caused the event.
asset_event_type	int32	The event type of the event associated with the asset event.
asset_event_subtype	int32	The event subtype of the event associated with the asset event.
badge_key	int64	The database record ID of the badge.
badge_key_str	string	A string representation of the badge key. To accurately display badge key, web clients should use this property instead of the badge_key property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off.
badge_status_name	string	The status of the badge.

Properties for Asset Events

Property	Type	Description
badge_type_name	string	The cardholder's badge type, as configured in System Administration.
cardholder_first_name	string	The cardholder's first name, as configured in System Administration.
cardholder_key	int32	The database ID, which is not displayed in System Administration, but which can be useful when developing custom scripts.
cardholder_last_name	string	The cardholder's last name, as configured in System Administration.

Biometric Events

Properties for Biometric Events

Property	Type	Description
badge_id	int64	The ID encoded on a badge.
badge_id_str	string	A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the ID property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off.
badge_issue_code	uint32	Issue code associated with the card.
biometric_score	uint32	The biometric score for a biometric card event.

Intercom Events

When an Intercom event occurs, subscribers with proper authorization receive the following properties and their values:

Properties for Intercom Events

Property	Type	Description
intercom_data	uint32	Special intercom data associated with the event.
intercom_line_number	int32	The line number used by special intercom events.

Intrusion Events

When an Intrusion event occurs, subscribers with proper authorization receive the following properties and their values:

Properties for Intrusion Events

Property	Type	Description
intrusion_area_id	uint16	The ID of the area where an intrusion was detected.
intrusion_user_id	string	The ID of the user who will receive information about an intrusion event.
receiver_area_id	uint16	The ID of the area where the receiver is located.
receiver_controller_id	uint16	The ID of the receiver that generated the event.
receiver_line_number	uint16	The line number used by the receiver that generated the event.

Transmitter Events

When a Transmitter event occurs, subscribers with proper authorization receive the following properties and their values:

Properties for Transmitter Events

Property	Type	Description
transmitter_id	int32	The ID of the device transmitting the event.
transmitter_input_id	int32	The ID of the input on the transmitter associated with the event.

Video Events

Properties for Video Events

Property	Type	Description
video_channel	int32	The physical channel to which the camera is connected.
video_start_time	uint32	The start time of the video associated with an event.
video_end_time	uint32	The end time of the video associated with an event.

Status Events

All events are examined, regardless of their message type, to determine if the information indicates a status change. If that is the case, additional information specifying the status change is appended to the event before it is distributed to subscribing clients. The appended information follows the same key/value pair methodology but uses specific keys to indicate that the data specifies status information.

The presence of the key **status_count** indicates that status information is contained in the event and the value is an integer count of the number of status change items that have been appended. In most cases, the count value will be one, but there are cases where the count value can be higher indicating that the source event contained information indicating that multiple state changes have occurred.

For each status change item, there are four key/value pairs that convey the information about that particular status change, as summarized below.

Status Information Key/Value Pairs

Key structure	Type	Value description
status_<n>_name	string	The name of the status item that changed, where <n> is an integer index specifying which status item the data is for, with 0 for the first status item, 1 for the second, etc.
status_<n>_name_text	string	The language translated display text for the name.
status_<n>_value	string	The new value for the status item.
status_<n>_value_text	string	The language translated display text for the value of the status item.
status_count	int32	An integer specifying the number of status change items appended to the event.

Here is an example of status change information that can be appended to an event:

```
status_0_name      ReaderMode
status_0_name_text Reader Mode
status_0_value     ReaderModePinOrCard
status_0_value_text Pin or Card
status_count       1
```

Here is an example of status change information where the status item conveys a value and the range of values is not fixed or predefined. For these status items, both the value and value_text elements contain the data.

```
status_0_name      PanelCardCapacity
status_0_name_text  Panel Card Capacity
status_0_value     500
status_0_value_text 500
status_count       1
```

Here is an example of status change information containing multiple status items that can be appended to an event:

```
status_0_name = ReaderAuxInputLineStatus
status_0_name_text = Reader Auxiliary Input Line Status
```

```

status_0_value = Alarm
status_0_value_text = Alarm
status_1_name = ReaderAuxInputMasking
status_1_name_text = Reader Auxiliary Input Masking
status_1_value = Unmasked
status_1_value_text = Unmasked
status_count = 2

```

The table below identifies the status change items currently supported through the OpenAccess API.

Status Change Items

Name	Description
Device-independent status items	
OnlineStatus	The communication status of the device. Values: Online, Offline
FirmwareRevision	The firmware revision of the device. Value: A text string
SerialNumber	The serial number of the device. Value: An integer
Panel status items	
PanelPowerInputStatus	The power input status for a panel. Values: Secure, Alarm
PanelCabinetStatus	The cabinet status for a panel. Values: Secure, Alarm
PanelFirmwareDownloadStatus	The firmware download status for a panel. Values: Completed, In Progress
PanelDownloadStatus	The download status for a panel. Values: Completed, In Progress
PanelEventPollingStatus	The event polling status for a panel. Values: Normal, Stopped
PanelCardCapacity	The maximum number of cards supported by the panel. Value: An integer
PanelCardCount	The current number of cards downloaded to the panel. Value: An integer
Reader status items	
ReaderAuxInputMasking	The masking state of a reader auxiliary input. Values: Masked, Unmasked
ReaderAuxOutputActivation	The activation state of a reader auxiliary output. Values: Activated, Deactivated

Status Change Items (Continued)

Name	Description
ReaderMode	The mode of a reader. Values: Facility Code Only, Card Only, Pin Only, First Card Unlock, Card Unlocked, Locked, Unlocked, Pin or Card, Card and Pin, Cipher or Card, Dual Custody, Escort, Blocked, Secured, Unsecured, Normal
ReaderAuxInputLineStatus	The reader auxiliary input physical line status. Values: Secure, Alarm, Shorted, Open, Grounded, Error
ReaderPowerfailStatus	The power status for a reader. Values: Active, Inactive
ReaderCabinetTamperStatus	The cabinet tamper status for a reader. Values: Active, Inactive
ReaderExternalTamperStatus	The external tamper status for a reader. Values: Active, Inactive
ReaderExtraPowerfailStatus	The extra powerfail status for a reader. Values: Active, Inactive

Example Access Denied Event

```

1  badge_id: 1
2  controller_id: 1
3  device_id: 1
4  device_type: 0
5  domain: access
6  event_subtype: 65
7  event_type: 1
8  initiating_event_id: 0
9  intelligent_video: 0
10 segment_id: 0
11 serial_number: 1460010837
12 source: CommServer@TEST105-248
13 subdevice_id: 0
14 timestamp: 1460011160000
15 timestamp_processed: 1460011160684
16 transmitter_id: 0
17 transmitter_input_id: 0
18 version: 1.0
19 controller_name: Panel-3300
20 controller_segment_id: 0
21 controller_time_zone_id: 16
22 event_source_name: Reader-AAA
23 alarm_id: 4100
24 alarm_name: Denied Access
25 badge_key: 1
26 badge_extended_id:
27 badge_type_name: Employee
28 badge_status_name: Active
29 cardholder_first_name: Lisa
30 cardholder_last_name: Lake
31 cardholder_key: 1
32 business_event_class: hardware_event

```

Alarm Acknowledgment Activity Event Reference

The Alarm Acknowledgment Activity event is published when an alarm is acknowledged by a user. Subscribers with proper authorization receive the following properties and their values:

Properties for Alarm Acknowledgment Activity Events

Property	Type	Description
controller_id	int16	The ID of the access panel that generated the alarm.
serial_number	int32	The serial number of the alarm.
user_id	string	The ID of the user that submitted the acknowledgment.
acknowledge_notes	string	Optional notes submitted with the acknowledgment.
acknowledge_status	int32	The status of the acknowledgment that can be one of the following: 0 Update 1 Acknowledged without notes 2 Acknowledged with notes 3 In Progress
device_id	uint16	The ID of the device that is the source of an event. A value of 0 indicates that the source of the event is a controller.
subdevice_id	uint16	The ID of the subdevice of a device that is the source of the event. A value of 0 indicates that the source is a device or a controller.
event_type	uint8	A class of events defined in the system and reported by the API that can be further broken down into subtypes. For example, 0 indicates an access granted event and 1 indicates an access denied event.
event_id	int32	The ID of the event.
domain	string	The source domain of an event.
source	string	The source of the event encoded in a domain-specific manner as a URI string. For example, a source defined as a UUID should be encoded as <code>urn:uuid:7673868d-231e-490d-9c4f-19288e7e668d</code> . For more examples, visit: http://example.org/absolute/URI/with/absolute/path/to/resource.txt
timestamp	int64	The time when the event occurred at its source, following the AMQP standard of milliseconds since January 1, 1970 in UTC time.

Properties for Alarm Acknowledgment Activity Events

Property	Type	Description
version	string	The version of this specific event message type. This is a period-delimited string in the format <code><major>.<minor></code>. - A <i>minor</i> version change is one in which only fields were added, and a parser that ignores unrecognized fields can still process the message. - A <i>major</i> version change is one in which the message structure has changed in a manner that is not backwards compatible with the previous structure. Version is managed on a per event type basis, not the version of the application that sent the message. A specific event type is uniquely identified using the ordered list of domain, event type, and version.
business_event_class	string	Type of event. Will always be Acknowledgment Event.

Software Event Reference

A software event is an event that occurs when an object in OnGuard is added, modified, or deleted. Examples of such objects include cardholders, visitors, and badges.

Users with *all segments* and *view all permissions* can register to receive software events that they have permission to receive. In general, users can view a software event for an object if they could view that object normally. For example, if users do not have permission to view visitors, then they cannot receive software events indicating that a visitor was created, modified, or deleted. Furthermore, if users do not have view permissions for each property of a class, then they can't receive software events for instances of that class. For example, if users can't view the visitor address field (set through the field/page permission groups in System Administration), then they can't view visitor software events.

Note: For all **Add** events, each object property name is prefixed with **new_**. For all **Delete** events, each object property name is prefixed with **old_**. All **Modify** events include both the **new_** and **old_** prefixes.

Common Properties for All Software Events

Property	Type	Description
business_event_class	string	Type of event. Will always be software_event.
object_id	int32	The unique identifier of the software event.
software_event_object_type	string	The software event's object type, such as Cardholder, Visitor, Badge, Visit, VisitEvent, or Account.
software_event_operation_type	string	The software event's operation type, such as Add, Modify, or Delete.
timestamp	int64	The time when the event occurred at its source, following the AMQP standard of milliseconds since January 1, 1970 in UTC time.

Person Directory Account Events

When a Person Directory Account event occurs, subscribers with proper authorization receive the following properties and their values. For more information, refer to [LnL_Account](#) on page 188.

Properties for Person Directory Account Events

Property	Type	Description
AccountID	string	ID of the entry in the external directory.
DirectoryID	string	Internal ID of the directory to which this account belongs.
ID	int32	ID that uniquely identifies this directory account.
PersonID	int32	Internal ID of the person who owns this account.

Badge Events

When a Badge event occurs, subscribers with proper authorization receive the following properties and their values. For more information, refer to [LnL_Badge](#) on page 197.

Properties for Badge Events

Property	Type	Description
ACTIVATE	datetime (string)	Badge activate date. The default is the current date and time.
APBEXEMPT	boolean	Whether the badge is APB exempt.
BADGEKEY	int32	ID that uniquely identifies the badge.
DEACTIVATE	datetime (string)	Badge deactivate date.
DEADBOLT_OVERRIDE	boolean	If true, the selected cardholder will have deadbolt override privileges, which allows the cardholder to access a door with a deadbolt function mortise lock even when the deadbolt is thrown.
DEFAULT_DOOR	int32	Indicates which elevator door (front or rear) is opened at the Default floor when the badge is presented to a reader associated with the DEC (elevator terminal).
DEFAULT_FLOOR	int32	Indicates the floor number that is called by default when the badge is presented to a reader associated with the DEC (elevator terminal). Configure the Default floor from -128 to 127.
DESCRIPTOR_FLAG	int32	Custom objects that are sent to an elevator dispatch system.
DEST_EXEMPT	boolean	When selected, the badge will not be included in the destination assurance processing and no alarms will be generated if the cardholder violates any of the destination assurance settings.

Properties for Badge Events

Property	Type	Description
EMBOSSSED	int32	Any numbers or characters that are embossed on the card. Typically this applies to Proximity cards, which are embossed by the manufacturer prior to delivery.
EXTEND_STRIKE_HELD	boolean	Use extended strike/held times.
EXTENDED_ID	string	Extended length string identifier that refers to a PIV-based badge in the OnGuard database that generated the event.
ID	int64	The ID of the badge.
ID_str	string	A string representation of the badge ID.
ISSUECODE	int32	Issue code of the badge.
LASTCHANGED	datetime (string)	Date the badge was last changed.
LASTPRINT	datetime (string)	Date the badge was last printed.
PASSAGE_MODE	boolean	If true, the cardholder is allowed to use the card twice (within the lock's unlock duration) to place the lock in an unlock mode for an indefinite duration.
PERSONID	int32	Internal ID of the person who owns this badge.
PRINTS	int32	Number of times badge has been printed.
STATUS	int32	Badge status ID. 1 = Active.
TWO_MAN_TYPE	int32	Specifies the two-man rule designation of the cardholder (either Supervisor or Team Member).
TYPE	int32	Badge type ID.
USELIMIT	int32	Imposes a restriction on the number of times a cardholder can use his/her badge at readers marked with the Enforce Use Limit option. A use limit value of zero (0) indicates that a badge has no uses at readers that enforce a use limit. A use limit value of 255 or that is left empty indicates that the badge has unlimited uses.

Cardholder Events

When a Cardholder event occurs, subscribers with proper authorization receive the following properties and their values. For more information, refer to [Lnl_Cardholder](#) on page 206.

Properties for Cardholder Events

Property	Type	Description
ADDR1	string	Cardholder's address.
ALLOWEDVISITORS	boolean	Whether the Allowed visitors checkbox is selected on the Cardholders folder in System Administration.

Properties for Cardholder Events

Property	Type	Description
ASSET_GROUPID	int32	ID of the Asset Group.
BDATE	datetime (string)	Cardholder's birth date, in the format 1968-07-31T00:00:00-04:00.
BUILDING	int32	Cardholder's building.
CITY	string	Cardholder's city.
DATABASEID	int32	The database identifier in an Enterprise system that identifies the system containing the reader to which the badge was last presented.
DEPT	int32	Cardholder's department.
DIVISION	int32	Cardholder's division.
EMAIL	string	Cardholder's email address.
EXT	string	Cardholder's extension.
FIRSTNAME	string	Cardholder's first name.
FLOOR	string	Cardholder's floor.
GUARD	int16	Indicates that the cardholder can be assigned to perform guard tours (1 = guard can perform tours).
ID	int32	Unique cardholder ID.
LASTCHANGED	datetime (string)	Date the record was last changed.
LASTNAME	string	Cardholder's last name.
LOCATION	int32	Cardholder's location.
MIDNAME	string	Cardholder's middle name.
OPHONE	string	Cardholder's office phone number.
PHONE	string	Cardholder's phone number.
PRIMARYSEGMENTID	int32	This property is only visible when cardholders are segmented.
SSNO	string	Cardholder's social security number.
STATE	string	Cardholder's state.
TITLE	int32	Cardholder's title.
VISITOR	boolean	Whether the cardholder is a visitor in the system.
ZIP	string	Cardholder's zip code.

Visitor Events

When a Visitor event occurs, subscribers with proper authorization receive the following properties and their values. For more information, refer to [LnL_Visitor](#) on page 264.

Properties for Visitor Events

Property	Type	Description
ADDRESS	string	Visitor's address.
ASSET_GROUPID	int32	ID of the Asset Group.
CITY	string	Visitor's city.
DATABASEID	int32	The database identifier in an Enterprise system that identifies the system containing the reader to which the badge was last presented.
EMAIL	string	Visitor's email address.
EXT	string	Visitor's extension.
FIRSTNAME	string	Visitor's first name.
GUARD	int16	Indicates that the visitor can be assigned to perform guard tours (1 = guard can perform tours).
ID	int32	Unique visitor ID.
LASTCHANGED	datetime (string)	Date the record was last changed.
LASTNAME	string	Visitor's last name.
MIDNAME	string	Visitor's middle name.
OPHONE	string	Visitor's office phone number.
ORGANIZATION	string	Visitor's organization.
PRIMARYSEGMENTID	int32	This property is only visible when visitors are segmented.
SSNO	string	Visitor's social security number.
STATE	string	Visitor's state.
TITLE	string	Visitor's title.
VISITOR	boolean	Whether the visitor is a visitor in the system.
ZIP	string	Visitor's zip code.

Visit Events

When a Visit event occurs, subscribers with proper authorization receive the following properties and their values. For more information, refer to [LnL_Visit](#) on page 261.

Properties for Visit Events

Property	Type	Description
CARDHOLDERID	int32	The ID for the visitor's host.
ID	int32	Unique visit ID.
LASTCHANGED	datetime (string)	The date and time the visit was last changed, in UTC time.
PURPOSE	string	The purpose of the visit.
SCHEDULED_TIMEIN	datetime (string)	The scheduled time the visitor will arrive for the visit.
SCHEDULED_TIMEOUT	datetime (string)	The scheduled time the visitor will leave from the visit.
STATUS	int16	The status of the visit.
TIMEIN	datetime (string)	The actual time the visitor arrived for the visit, in UTC time.
TIMEOUT	datetime (string)	The actual time the visitor left the visit, in UTC time.
TYPE	int32	System field.
VISIT_EVENTID	int32	The ID of the visit event.
VISIT_KEY	string	A unique identifier assigned to a scheduled visit, used to sign visitors in or out.
VISITORID	int32	The ID of the visitor.

VisitEvent Events

When a VisitEvent event occurs, subscribers with proper authorization receive the following properties and their values. For more information, refer to [LnL_VisitEvent](#) on page 263.

Properties for VisitEvent Events

Property	Type	Description
CardholderID	int32	The host of the visit event.
DatabaseID	int32	The database identifier in an Enterprise system that identifies the system containing the event data.
DelegateID	int32	The person who schedules or maintains the event instead of the host.
ID	int32	Unique visitor event ID.

Properties for VisitEvent Events

Property	Type	Description
LastChanged	datetime (string)	The last time the properties of the visit event changed, in UTC time.
Name	string	The user-friendly name of this object.
Scheduled_TimeIn	datetime (string)	The time the visit event is scheduled to start.
Scheduled_TimeOut	datetime (string)	The time the visit event is scheduled to complete.
SignInLocationID	int32	The ID of the visitor sign in location.

Example Add Cardholder Event

```

1  business_event_class: software_event
2  object_id: 2
3  software_event_object_type: Cardholder
4  software_event_operation_type: Add
5  timestamp: 1460011160000
6  new_ADDR1: 1212 Pittsford-Victor Rd.
7  new_ALLOWEDVISITORS: 1
8  new_ASSET_GROUPID: 0
9  new_BDATE: 01/01/1965
10 new_BUILDING: 0
11 new_CITY: Rochester
12 new_DATABASEID: 1
13 new_DEPT: 0
14 new_DIVISION: 0
15 new_EMAIL: user@abc.com
16 new_EXT: 5555
17 new_FIRSTNAME: William
18 new_FLOOR: 1
19 new_GUARD: 0
20 new_ID: 2
21 new_LASTCHANGED: 1477928433000
22 new_LASTNAME: Smith
23 new_LOCATION: 0
24 new_MIDNAME: Thomas
25 new_OPHONE: 555-555-5555
26 new_PHONE: 555-555-1212
27 new_PRIMARYSEGMENTID: 0
28 new_SSN0: 555-55-5555
29 new_STATE: NY
30 new_TITLE: 0
31 new_VISITOR: 0
32 new_ZIP: 14534

```

Data Classes

For more information about each data class, execute a get type call. For more information, refer to [get type](#) on page 77.

Notes: All class and property access is subject to OnGuard user permissions.

In the following tables, **View** indicates that the property is view only and not editable. **Read** indicates that the property is editable on Add only. **Edit** indicates that the property is always editable.

DatabaseID only appears as a property when the OnGuard system is an Enterprise system. For more information, refer to [get enterprise settings](#) on page 133.

SEGMENTID only appears as a property in data classes that support segmentation when segmentation for that class is enabled. For more information, refer to [get segmentation settings](#) on page 139 and [LnI_Segment](#) on page 252. Restarting the LS OpenAccess service is required when making segmentation changes.

LnI_AccessGroup

Description: An access group defined in the security system.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View

Type	Name	Description	Access
int32	SEGMENTID	Segment to which the access group belongs.	View
string	NAME	Display name.	View

Methods:

```
void AssignGroup([in]int32 badgeKey);
```

Assigns all the access levels in the group to a specific badge.

Parameters:

badgeKey - int32 internal ID of the badge to which the access levels are assigned.

Lnl_AccessLevel

Description: An access level defined in the security system.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	SegmentID	Segment to which the access level belongs.	Read
string	Name	Display name.	Edit
boolean	AvailableForRequest	The access level is available to be requested.	Edit
boolean	HasCommandAuthority	Command authority is enabled for the access level	Edit
boolean	DownloadToIntelligentReaders	Level is download to Intelligent Readers	Edit
boolean	FirstCardUnlock	First Card Unlocks the reader	Edit

Type	Name	Description	Access
int32	EscortMode	If Enable extended options is selected in System Administration > Administration > System Options > Access Levels/ Assets, then EscortMode can be updated using a POST/PUT Lnl_AccessLevel call. Possible values are: • 0 - Not an escort and does not require an escort • 1 - Is an escort • 2 - Requires an escort Note: This property is returned in the response regardless of whether a particular access level has extended options enabled, as long as at least one access level in the system has extended options enabled.	Edit

Lnl_AccessLevelAssignment

Description: An access level assignment defined in the security system.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ACCESSLEVELID	Lnl_AccessLevel.ID - ID of the access level. Key field.	Read
int32	BADGEKEY	Lnl_Badge.BADGEKEY - BadgeKey of the badge. Key field.	Read
datetime (string)	ACTIVATE	Date and time when this assignment will become active.	Read

Type	Name	Description	Access
datetime (string)	DEACTIVATE	Date and time when this assignment will become inactive.	Read

Note: A successful response indicates that the badge and access level assignment have reached the database. The successful response does not indicate that the assignment has reached the access panel. There might be a delay before the assignment reaches the panel.

The following table describes how OpenAccess uses cardholder permissions and Area Access Manager levels to determine which access levels the authenticated OpenAccess user who is making the call can assign.

Does authenticated OpenAccess user have permission group, badge, and "Modify Access Level Assignment" permissions?	Does authenticated OpenAccess user have Area Access Manager levels defined?	The authenticated OpenAccess user can assign these access levels
Yes	Yes	All
Yes	No	All
No	Yes	Only Area Access Manager access levels
No	No	None

Note: If the authenticated OpenAccess user only has Area Access Manager access levels defined, all access levels in the AssignLevel array must be contained within the authenticated OpenAccess user's Area Access Manager access levels. For example, if the authenticated OpenAccess user has access levels 1 and 2, then the authenticated OpenAccess user cannot assign access levels 1, 2, and 3, and the entire access level assignment attempt will fail.

Lnl_AccessLevelManaged

Description: View all access levels that can be managed by Access Manager users.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Access level ID.	View
int32	SegmentID	Segment ID to which the access level belongs.	View

Type	Name	Description	Access
string	Name	Access level name.	View
boolean	AvailableForRequest	True if this access level can be requested.	View

LnI_AccessLevelReaderAssignment

Description: An access level reader assignment defined in the security system.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	AccessLevelID	Access level to which the link belongs. Key field.	View
int32	PanelID	LnI_Panel which is linked to this level. Key field. Reference to LnI_Panel.ID.	View
int32	ReaderID	LnI_Reader ID which is linked to this level. Key field.	View
string	AccessLevelName	Name of the LnI_AccessLevel.	View
boolean	AvailableForRequest	True if this access level can be requested.	View
string	ReaderFriendlyName	The descriptive name for the LnI_Reader.	View
string	ReaderName	The display name of the reader.	View
int32	TimezoneID	LnI_Timezone in which this level is active	View
string	TimezoneName	Name of the LnI_Timezone.	View

LnI_AccessRequest

Description: A request raised by a person for accessing access levels and readers.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	Name	Name of the associated access level or reader.	View
int32	PersonID	Internal ID of the person who requested access to the access level or reader. See Lnl_Person.ID.	View
int32	Type	Request type ID: 0: Reader 1: AccessLevel	View
int32	Status	Request status ID: 0: Submitted 1: Approved 2: OnHold 3: Denied	View
datetime (string)	StartDate	Start date the cardholder requests for access level or reader.	View
datetime (string)	EndDate	End date the cardholder requests for access level or reader.	View
int32	SubmittedByUserID	The user ID of the user who submits the request.	View
int32	ApprovedByUserID	The user ID of the user who approves the request.	View
int32	DeniedByUserID	The user ID of the user who denied the request.	View
int32	OnHoldByUserID	The user ID of the user who put the request on hold.	View
string	SubmittedNote	Notes entered when submitting this request.	View
string	ApprovedNote	Notes entered when approving this request.	View
string	DeniedNote	Notes entered when denying this request.	View
string	OnHoldNote	Notes entered when putting this request on hold.	View
datetime (string)	SubmittedDate	The date and time when the request was submitted.	View
datetime (string)	ApprovedDate	The date and time when the request was approved.	View

Type	Name	Description	Access
datetime (string)	DeniedDate	The date and time when the request was denied.	View
datetime (string)	OnHoldDate	The date and time when the request was put on hold.	View
boolean	EmailCardholder	Whether the cardholder is notified.	View
boolean	EmailAccessManager	Whether the approver is notified.	View

Lnl_AccessLevelRequest

Description: A request raised by a person for accessing access levels.

Abstract: No

Access: View/Add

Superclass: Lnl_AccessRequest

Platforms: OnGuard

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	Name	Name of the associated access level.	View
int32	AccessLevelID	Access level to which access request should be submitted. Key field.	Read
int32	PersonID	Internal ID of the person who requested access for AccessLevel. Key field. See Lnl_Person.ID.	Read
int32	Type	Request type ID: 1: AccessLevel	View
int32	Status	Request status ID: 0: Submitted 1: Approved 2: OnHold 3: Denied	View
datetime (string)	StartDate	Start date the cardholder requests for Accesslevel.	Read
datetime (string)	EndDate	End date the cardholder requests for Accesslevel.	Read
int32	SubmittedByUserID	The user ID of the user who submits the request.	View

Type	Name	Description	Access
int32	ApprovedByUserID	The user ID of the user who approves the request.	View
int32	DeniedByUserID	The user ID of the user who denied the request.	View
int32	OnHoldByUserID	The user ID of the user who put the request on hold.	View
string	SubmittedNote	Notes entered when submitting this request.	Read
string	ApprovedNote	Notes entered when approving this request.	View
string	DeniedNote	Notes entered when denying this request.	View
string	OnHoldNote	Notes entered when putting this request on hold.	View
datetime (string)	SubmittedDate	The date and time when the request was submitted.	View
datetime (string)	ApprovedDate	The date and time when the request was approved.	View
datetime (string)	DeniedDate	The date and time when the request was denied.	View
datetime (string)	OnHoldDate	The date and time when the request was put on hold.	View
boolean	EmailCardholder	Whether the cardholder is notified.	Read
boolean	EmailAccessManager	Whether the approver is notified.	Read

Methods:

`void Approve([in] string Note, [in] boolean EmailCardholder);`

Approves the AccessLevel Request. setting ApprovedDate to current date/time.

`void Deny([in] string Note, [in] boolean EmailCardholder);`

Denies the AccessLevel Request. setting DeniedDate to current date/time.

`void Hold([in] string Note, [in] boolean EmailCardholder);`

Holds the AccessLevel Request. setting OnHoldDate to current date/time.

Parameters:

Note : Notes when the request is approved, denied and put on hold.

EmailCardholder : Whether the cardholder should be notified.

Lnl_Account

Description: A directory account belonging to a person in the security system.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	AccountID	ID of the entry in the external directory. For example, with Microsoft directories, this property would contain the account's security identifier (SID).	Read
string	DirectoryID	Internal ID of the directory to which this account belongs.	Read
int32	PersonID	Internal ID of the person who owns this account. See Lnl_Person.ID.	Read

Lnl_AlarmAckHistory

Description: Records a change in the acknowledgment status of an OnGuard alarm.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
string	AckNote	The text entered by the operator when acknowledging an alarm.	View
int32	AckStatus	The status of the alarm, with possible values: 1: Acknowledged 2: Acknowledged with note 3: Marked in-progress	View
int32	AckTimeUTC	The date and time when the acknowledgment occurred, in the format YYYY-MM-DDTHH:MM:SS[+-]HH:00.	View
int32	ID	The internal ID of the acknowledgment entry.	View

Type	Name	Description	Access
int32	PanelID	The ID of the access panel with which the alarm is associated.	View
int32	SerialNumber	The serial number of the acknowledged alarm.	View
int32	UserID	the user ID of the user who acknowledged the alarm.	View

LnI_AlarmDefinition

Description: Defines how the alarm that is received from the panel is displayed. LnI_AlarmDefinition instances are queried by an end user in order to establish configuration details. This contrasts with LnI_Alarm instances, which come in with all security events that come through the Communication Server.

Note: Text instructions are required in order for an instance from this alarm class to appear in OpenAccess. Text instructions are created using the **System Administration > Monitoring > Alarms > Alarm Configuration** form.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
boolean	AckNotesRequired	True if notes are required when acknowledging this alarm type.	View
boolean	Active	True if the alarm type is configured as Active, meaning the alarm monitoring clients should highlight alarms of this type when they occur.	View
boolean	Aggregate	True if alarms of this type will be aggregated, meaning that alarm monitoring clients should combine all alarms of this type into a single alarm for display purposes.	View

Type	Name	Description	Access
boolean	ChangeResponse	True if it should be allowed for the operator to change the information provided when acknowledging this alarm type.	View
string	Description	Parameter description.	View
boolean	DisplayAlarm	True if this alarm should be displayed.	View
boolean	DisplayMap	True if a map containing the location of this alarm should be shown automatically.	View
boolean	DoNotDeleteOn-Acknowledge	True if alarms of this type should not be deleted from the client view when they are acknowledged.	View
int32	Flags	An integer value representing the combined values of all of the above boolean values.	View
int32	ID	Internal database ID. Key field.	View
boolean	LoginRequiredFor-Acknowledge	True if the operator is required to log in when acknowledging this alarm type.	View
boolean	MustAcknowledge	True if alarms of this type must be acknowledged before they can be deleted.	View
boolean	MustMarkInProgress	True if alarms of this type must be marked "In Progress" before they can be deleted.	View
boolean	PrintAlarm	True if this alarm should be printed.	View
int32	Priority	Alarm priority (0-255)	View
int32	SegmentID	Segment to which the alarm definition belongs.	View
boolean	ShowCardholder	True if the cardholder view should be shown for this alarm type.	View

Type	Name	Description	Access
string	TextInstructionName	Text instruction name.	View
string	TextInstructionData	Text instruction.	View
boolean	VideoVerify	True if the video verification view should be shown for this alarm type.	View
boolean	VisualNotification	True if the occurrence of this alarm type should be highlighted by, for example, bringing the main alarm monitor window to the foreground.	View

Lnl_AlarmInput

Description: Retrieves the hardware status for the device. Inherits from Lnl_Input, described below. Implements the input control methods and represents an alarm input found on an input control module.

Abstract: No

Access: View

Superclass: Lnl_Input

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	AlarmPanelID	The ID of the associated alarm panel.	View
string	HostName	The name of the workstation where the communication server associated with the alarm input's panel is running.	View
int32	InputID	The input number configured for this input.	View
string	Name	The name of the alarm input.	View
int32	PanelID	The ID of the associated access panel. Reference to Lnl_Panel.ID.	View

Methods:

void Mask();

Sends a command to mask a specific alarm input.

void Unmask();

Sends a command to unmask a specific alarm input.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:	
ALRM_STATUS_SECURE	0x00
ALRM_STATUS_ACTIVE	0x01
ALRM_STATUS_GND_FLT	0x02
ALRM_STATUS_SHRT_FLT	0x03
ALRM_STATUS_OPEN_FLT	0x04
ALRM_STATUS_GEN_FLT	0x05

Lnl_AlarmOutput

Description: Retrieves the hardware status for the device. Inherits from Lnl_Output, described below. Implements the relay control methods and represents an alarm relay found on an input or output control module.

Notes: The Activate(), Deactivate(), and Pulse() methods are not supported on Lenel, NGP, or Casi alarm panels when those panels are designated as elevator hardware.

Access panels with a dual reader that are designated as elevator hardware will not generate instances of this class.

Abstract: No

Access: View

Superclass: Lnl_Output

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	AlarmPanelID	The ID number of the associated alarm panel.	View
int32	Duration	The duration of the alarm, in seconds.	View

Type	Name	Description	Access
string	HostName	The name of the workstation where the communication server associated with the alarm output's panel is running.	View
int32	OutputID	The ID number of the associated alarm output.	View
string	Name	The name of the associated alarm output.	View
int32	PanelID	The ID number of the associated access panel. Reference to Lnl_Panel.ID.	View

Methods:

void Activate()

Sends a command to activate a specific alarm output.

void Deactivate()

Sends a command to deactivate a specific alarm output.

void Pulse()

Sends a momentary pulse command to a specific alarm output.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:		
uint32 Status	Description	Device status
ALRM_STATUS_SECURE	Output Secure	0
ALRM_STATUS_ACTIVE	Output Active	1

Lnl_AlarmPanel

Description: Retrieves the hardware status for the device. This class represents the Alarm input or output control module.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	PanelId	The ID of the associated access panel. Key field. Reference to Lnl_Panel.ID.	View
int32	ControlType	The type of alarm panel.	View
string	Name	The name of the associated alarm panel.	View

Methods:

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:		
uint32 Status	Description	Device status
ONLINE_STATUS	Online	0x01
OPTIONS_MISMATCH_STATUS	Options Mismatch	0x02
CABINET_TAMPER	Cabinet Tamper	0x04
POWER_FAIL	Power Failure	0x8

Lnl_Area

Description: An APB area defined in the security system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View

Type	Name	Description	Access
int32	AREATYPE	Type of APB area. Possible values: 0: Other 1: Unknown 2: Local Area 3: Global Area 4: Hazardous Location 5: Safe Location	View
string	NAME	Display name.	View

Methods:

`void MoveBadge();`

Moves a badge from one area into another.

`void MoveBadge([in] int32 areaID, [in] int64 badgeID, [in] string badgeID_str, [in] int32 panelID, [in] int32 readerID, [in] int32 segmentID, [in] datetime UTCTime);`

Parameters:

- *areaID* - This is ID of the area to move the badge to.
- *badgeID* - This is the 64-bit badge ID of the badge you want to move.
- *badgeID_str* - A string representation of the badgeID. You cannot provide both *badgeID* and *badgeID_str* in the same call.
- *panelID* - This is the ID of the panel of the reader responsible for moving the badge to the new area.
- *readerID* - This is the ID of the reader responsible for moving the badge.
- *segmentID* - This is the segment associated with the *panelID*, *readerID*.
- **UTCTime** - The time when the badge was moved to the area.

Lnl_AuthenticationMode

Description: Authentication modes for pivCLASS authenticated readers. Authentication modes specify the authentication mechanism used by the reader to authenticate a cardholder. These modes are configured as assurance profiles in the pivCLASS Validation Server. Use the ID of a retrieved authentication mode when setting reader modes with the `Lnl_Reader` associated class. For more information, refer to [Lnl_Reader](#) on page 239.

Abstract: No

Access: View

Superclass: `Lnl_Element`

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View

Type	Name	Description	Access
string	Name	Name of the authentication mode.	View

Lnl_Badge

Description: A badge in the security system.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	BADGEKEY	Internal database ID. Key field.	View
datetime (string)	ACTIVATE	Badge activate date. Note: Default for ACTIVATE is the current date and time.	Edit
boolean	APBEXEMPT	Whether the badge is APB exempt	Edit
datetime (string)	DEACTIVATE	Badge deactivate date. Note: Default for DEACTIVATE is determined by the configuration for the badge type in System Administration.	Edit
boolean	DEADBOLT_OVERRIDE	If true, the selected cardholder will have deadbolt override privileges, which allows the cardholder to access a door with a deadbolt function mortise lock even when the deadbolt is thrown.	Edit
boolean	DEST_EXEMPT	If true, the badge will not be included in the destination assurance processing and no alarms will be generated if the cardholder violates any of the destination assurance settings.	Edit
int32	EMBOSSSED	Embossed	Edit
boolean	EXTEND_STRIKE_HELD	Use extended strike/held times	Edit

Type	Name	Description	Access
int64	ID	ID of the badge.	Edit
string	ID_Str	A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the ID property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off. Note: This property is only returned when get instances is called with Version 1.2 or later. Note: When adding or modifying an Lnl_Badge, you cannot provide both an ID and an ID_Str.	View
int32	ISSUECODE	Issue code. Note: Default for ISSUECODE is determined by the First Issue Code configured for the badge type in System Administration.	Edit
datetime (string)	LASTCHANGED	Badge last changed	View
datetime (string)	LASTPRINT	Badge last printed	View
boolean	PASSAGE_MODE	If true, the cardholder is allowed to use the card twice (within the lock's unlock duration) to place the lock in an unlock mode for an indefinite duration.	Edit
int32	PERSONID	Internal ID of the person who owns this badge. If you modify Lnl_Badge.PERSONID, all other properties are ignored. For example, if you modify Lnl_Badge.PERSONID and Lnl_Badge.STATUS in the same "PUT /instances" call, the Lnl_Badge.STATUS is ignored. Lnl_Badge.PERSONID cannot be modified if the class of the badge type is "Standard" or "Guest".	Edit

Type	Name	Description	Access
string	PIN	PIN code. Note: You cannot view or search the contents of this property.	Edit
int32	PRINTS	Number of times badge has been printed	View
int32	STATUS	Badge status ID. 1 = "Active". For more information, refer to User-Defined Value Lists on page 270.	Edit
int32	TYPE	Badge type ID. For more information, refer to LnI_BadgeType on page 202.	Edit
int32	USELIMIT	Use limit	Edit

Note: A successful response indicates that the badge and access level assignment have reached the database. The successful response does not indicate that the assignment has reached the access panel. There might be a delay before the assignment reaches the panel.

Methods:

- `void AssignAccessLevel([in] int32[] LevelIn);`
Assigns the access level(s) of a badge. The following table describes how OpenAccess uses cardholder permissions and Area Access Manager levels to determine which access levels a the authenticated OpenAccess user who is making the call can assign.

Does authenticated OpenAccess user have permission group, badge, and "Modify Access Level Assignment" permissions?	Does authenticated OpenAccess user have Area Access Manager levels defined?	The authenticated OpenAccess user can assign these access levels
Yes	Yes	All
Yes	No	All
No	Yes	Only Area Access Manager access levels
No	No	None

Note: If the authenticated OpenAccess user only has Area Access Manager access levels defined, all access levels in the AssignLevel array must be contained within the authenticated OpenAccess user's Area Access Manager access levels. For example, if the authenticated OpenAccess user has access levels 1 and 2, then the authenticated OpenAccess user cannot assign access levels 1, 2, and 3, and the entire access level assignment attempt will fail.

Parameters:

LevelIn - Array that includes all the access level IDs the badge needs to be assigned with, in the format:

– [1, 2, 3]

- void ReplaceAccessLevels([in] int32 SourceBadgekey);

Replaces the access levels assigned to the badge instance with the access levels belonging to the badge with the supplied badgekey.

If no input parameter is provided, this method removes all access level assignments of the badge. This is the recommended approach for deleting all access level assignments from a badge.

Parameters:

SourceBadgekey - The badgekey of the badge from which to copy the access levels.

- void ReplacePIN([in] int32 SourceBadgekey);

Replaces the PIN assigned to the current badge instance with the PIN belonging to the badge with the supplied badgekey.

Parameters:

SourceBadgekey - The badgekey of the badge from which to copy the PIN.

Lnl_BadgeFIPS201

Description: Holds the data imported from FIPS 201 credentials.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	BADGEKEY	Internal database ID of the associated badge record. Key field.	Read
string(hex)	FASCN	Federal Agency Smart Credential Number.	Edit
binary	TWICPrivacyKey	TWIC Privacy Key. The key used to encrypt/decrypt the fingerprints on TWICs.	Edit
int32	TPKAlgorithmId	TWIC Privacy Key algorithm identifier. The algorithm used for encrypting/decrypting the fingerprints on TWICs. Paired with the TWIC Privacy Key.	Edit
string(hex)	UUID	Cardholder's globally unique identifier.	Edit

Type	Name	Description	Access
int32	CredentialType	The type of FIP 201 credential. 0 = Unknown 1 = PIV 2 = TWIC 3 = CAC with PIV Endpoint or Next Generation (NG) applet 4 = CAC without PIV applet 5 = PIV-I or CIV	Edit

Lnl_BadgeLastLocation

Description: Shows at what reader the badge was presented last.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int64	BadgeID	Badge ID. Key field.	View
string	BadgeID_str	A string representation of the badge ID. To accurately display badge ID, web clients should use this property instead of the ID property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off. Note: This property is only returned when get instances is called with Version 1.2 or later.	View
int32	AccessFlag	Shows whether the access was granted. Key field.	View
int32	DatabaseID	The database identifier in an Enterprise system that identifies the system containing the reader to which the badge was last presented. Key field.	View
int32	PanelID	Panel ID where access event occurred. Reference to Lnl_Panel.ID.	View
int32	ReaderID	Reader ID at which access occurred	View

Type	Name	Description	Access
datetime (string)	EventTime	Time at which access occurred	View
int32	EventID	ID of the event associated with the access.	View
int32	EventType	Type of the event associated with access	View
int32	PersonID	LnI_Person for which access occurred	View
int32	IsFromReplication	Shows whether badge last location came over for other region in the system.	View

LnI_BadgeStatus

Description: The status of a badge in the security system.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	NAME	Name of the list value.	Edit

LnI_BadgeType

Description: A badge type in the security system.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	NAME	Name of the badgetype.	View

Type	Name	Description	Access
int32	BadgeIDAllocationType	Indicates the method by which the Badge ID field on the Badge Form is automatically filled in when adding a new badge. 1: Automatic 2: From Cardholder ID 3: Manual entry 5: Internal Cardholder ID 7: FASC-N 8: Import from card	View
int32	BadgeTypeClass	Class of the badgetype Possible values: 0: Standard 1: Temporary 2: Visitor 3: Guest 4: Special Purpose	View
int32	DefaultAccessGroup	A group of access levels to be associated with this badge type.	View
string	DefaultDeactivationDate	Indicates the date on which badges of the specified type will expire.	View
int32	DefaultDeactivationDateType	Indicates the type, or class, assigned to this badge. 0: None 2: Exact 2: After	View
int32	FirstIssueCode	Indicates the first issue code, if used, for the badge (0 or user-specified).	View
boolean	IsDisposable	If true, indicates that the visitor's badge will be a disposable badge.	View
int32	SegmentID	Segment to which the badge type belongs.	View
boolean	AnySegmentCanAssign	Returns true if badge type is made available to any user and any person (no segment restrictions).	View
boolean	BadgeIDAllowEdit	Returns true if badge type allows editing of the badge ID of this type.	View
boolean	UseLatestBadgeDeactivationDate	Indicates whether or not the latest deactivation date of existing badges is used.	View
boolean	UseMobileCredential	Indicates whether or not mobile credentialing is enabled.	View

Methods:

- `void GetRequiredFields([out] string[] RequiredFields);`
Returns a list of field names that this badge type requires a cardholder to have in order to possess a badge of this type.

Lnl_Camera

Description: A camera defined in the system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	PanelID	Lenel NVR ID. Reference to Lnl_Panel.ID. Key field.	View
string	Name	Camera Name.	View
string	CameraID	Recorder-specific unique camera ID. Note: This property is only returned when get instances is called with Version 1.5 or later.	View
string	CameraTypeName	Camera Type Name	View
int32	Channel	Lenel NVR Channel	View
string	VideoStandard	Video Standard (Ex.: NTSC).	View
int32	IPAddress	IP address of the camera	View
int32	Port	Port of the camera	View
int32	HorizontalResolution	Horizontal resolution	View
int32	VerticalResolution	Vertical Resolution	View
int32	MotionBitRate	Motion Bit Rate	View
int32	NonMotionBitRate	Non-motion Bit Rate	View
int32	FrameRate	Frame rate	View
boolean	IsOnline	If true, the camera is online.	View
string	Workstation	Workstation of the host Lenel NVR.	View

Methods:

void GetHardwareStatus([out] uint32 Status)

Retrieves the hardware status for the device. Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

Lnl_CameraDeviceLink

Description: Shows the relationship between a camera and a device (such as a reader). Used for determining if event video is available for the specified device.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	CameraID	The ID of the camera.	View
int32	DeviceID	The ID of the device.	View
int32	DevicePanelID	The ID of the panel to which the device is associated.	View
int32	InputOutputID	The ID of the input or output for this association, if any.	View
int32	VideoRecorderID	The ID of the video recorder to which the camera is associated.	View
int32	ViewOrder	The order, or priority, to be used by clients when displaying video associated with an event, if there are multiple cameras associated with a single device.	View

Lnl_CameraGroup

Description: Camera group definition.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View

Type	Name	Description	Access
string	Name	Group name.	View
int32	SegmentID	Segment to which the camera group belongs.	View

Lnl_CameraGroupCameraLink

Description: An association between a camera and camera group.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	CameraGroupID	Camera group for this link. Lnl_CameraGroup.ID. Key field.	View
int32	PanelID	Panel ID for the camera. Reference to Lnl_Panel.ID. Key field.	View
int32	CameraID	Camera ID. Key field. See Lnl_Camera.ID.	View

Lnl_Cardholder

Description: A cardholder in the security system.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Person

Platforms: OnGuard

Properties: The class has all the properties of the Lnl_Person class, plus any custom fields defined by the end user. In addition, the class has the following properties:

Type	Name	Description	Access
boolean	ALLOWEDVISITORS	Whether this cardholder is allowed to have visitors	Edit
string	ADDR1	The cardholder's address.	Edit
datetime (string)	BDATE	The cardholder's birth date.	Edit

Type	Name	Description	Access
int32	BUILDING	Reference to LnI_BUILDING. For more information, refer to User-Defined Value Lists on page 270.	Edit
string	CITY	The cardholder's city.	Edit
int32	DEPT	Reference to LnI_DEPT. For more information, refer to User-Defined Value Lists on page 270.	Edit
int32	DIVISION	Reference to LnI_DIVISION. For more information, refer to User-Defined Value Lists on page 270.	Edit
string	EMAIL	The cardholder's email address.	Edit
string	EXT	The cardholder's extension.	Edit
string	FLOOR	The cardholder's floor.	Edit
int32	LOCATION	Reference to LnI_LOCATION. For more information, refer to User-Defined Value Lists on page 270.	Edit
string	OPHONE	The cardholder's office phone number.	Edit
string	PHONE	The cardholder's phone number.	Edit
int32	PRIMARYSEGMENTID	This property is only visible when cardholders are segmented.	Read
string	SSNO	Person's identification number.	Edit
string	STATE	The cardholder's state.	Edit
int32	TITLE	Reference to LnI_TITLE. For more information, refer to User-Defined Value Lists on page 270.	Edit
string	ZIP	The cardholder's zip code.	Edit

LnI_DeviceGroup

Description: A group consisting of one or more readers, inputs, outputs, cameras, or remote monitoring devices. A group can contain devices from more than one access panel, and a device can belong to more than one group. In a segmented system, a device group can belong either to one segment or to all segments.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	Name	The name of the device group.	View
int32	SegmentID	The ID of the segment to which the device group belongs (when segmentation is enabled).	View
int32	Type	The type of device group: 0: Reader Group 1: Input Group 2: Output Group 3: Camera Group 4: Monitor Group	View

LnI_Directory

Description: A directory defined in the security system.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
string	ID	Internal database ID. Key field.	View
string	ACCOUNTCATEGORY	Account category.	View
string	ACCOUNTCLASS	Account class.	View
string	ACCOUNTDISPLAYNAMEATTR	Account display name attribute.	View
string	ACCOUNTIDATTR	Account ID attribute.	View

Type	Name	Description	Access
string	ACCOUNTUSERNAMEATTR	Account user name attribute.	View
string	HOSTNAME	Host name or domain.	View
string	NAME	Display name.	View
sint32	PORT	Port	View
string	STARTNODE	Start node.	View
sint32	TYPE	Directory type. Possible values: 0: LDAP 1: Microsoft Active Directory 2: Microsoft Windows NT 4 Domain 3: Windows Local Accounts 4: OpenID Connect	View
boolean	USESSL	Use SSL	View

See the ID CredentialCenter User Guide for more information about directory properties.

Lnl_Element

Description: The base class for many data classes.

Abstract: Yes

Access: None

Superclass: None

Platforms: OnGuard

Properties: None

Lnl_ElevatorTerminal

Description: An elevator terminal defined in the security system. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	IPAddress	The IP address of the associated elevator terminal. Key field	View

Type	Name	Description	Access
int32	PanelId	Reference to Lnl_Panel.ID. Key field.	View
string	Hostname	Host name or domain.	View
int32	Name	The name of the associated elevator terminal.	View

Methods:

`void GetHardwareStatus();`

Status is only retrieved from the hardware when the `UpdateHardwareStatus` is called on the parent ISC.

Possible returned values are:

- 1 = default floor only
- 2 = Access to authorized floors
- 3 = User entry of destination floor
- 4 = Default floor or user entry of destination floor

`void SetAllowedFloors();`

Sends a command to update which floors and doors are accessible via the elevator terminal without supplying security credentials. This method takes a single parameter named `AllowedFloorListID` which corresponds to a Floor List in the OnGuard software. Returns Pass or Fail.

`void SetTerminalMode();`

Sends a command to update the elevator terminal's operational mode for interacting with the cardholder. This method takes the numerical value of a single parameter named `Mode`. Possible values are:

- 1 = Default floor only. When the cardholder presents a valid badge to the elevator reader, or enters a valid PIN code or floor number on the elevator terminal, the system calls the default floor.
- 2 = Access to authorized floors. When the cardholder presents a valid badge to the elevator reader, and then selects an authorized floor, the system calls the authorized floor.
- 3 = User entry of destination floor. The cardholder has the option to select a floor with or without presenting a valid badge to the elevator reader. If the selected floor is an allowed floor, the system calls the floor. If the floor is a non-allowed floor, the cardholder is requested to present a valid badge.
- 4 = Default floor or user entry of destination floor. When the cardholder presents a valid badge to the elevator reader, the system calls the cardholder's default floor. Within a configurable timeout period, the cardholder can override the default floor call by entering another floor number.

Lnl_EventAlarmDefinitionLink

Description: The link between the event type and alarm for a particular device.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	DeviceID	Device ID (ex.: Alarm panel, Reader). Key field.	View
int32	EventParameterID	Event parameter ID. Key field. See Lnl_EventParameter.ID.	View
int32	EventSubtypeDefinitionID	Event Subtype. Key field. See Lnl_EventSubtypeDefinition.ID.	View
int32	EventTypeID	Event Type. Key field. See Lnl_EventType.ID.	View
int32	PanelID	Panel ID (ex.: ISC). Key field. Reference to Lnl_Panel.ID.	View
int32	SecondaryDeviceID	Secondary device ID (ex.: Input, Output). Key field.	View
int32	AlarmDefinitionID	Alarm Definition. See Lnl_AlarmDefinition SubtypeID.	View

Lnl_EventParameter

Description: An event parameter.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	Description	Parameter description.	View
int32	Value	Parameter value	View

Lnl_EventSubtypeDefinition

Description: An event subtype defined in the system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	TypeID	Event Type ID, see Lnl_EventType.ID.	View
int32	SubTypeID	ID within the subtype.	View
string	Description	Sub type description.	View
int32	SupportParameters	Supporting Parameter ID	View
int32	Category	Event subtype category	View

Lnl_EventSubtypeParameterLink

Description: An association between an event subtype and event parameter.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	EventParameterID	Key field. See Lnl_EventParameter.ID.	View
int32	EventSubtypeDefinitionID	Key field. See Lnl_EventSubtypeDefinition.ID.	View

Lnl_EventType

Description: An event type defined in the system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View

Type	Name	Description	Access
string	Description	Event type description.	View

Lnl_GuardTour

Description: A guard tour provides a security guard with a defined set of tasks that must be performed within a specified period of time.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	Name	Guard tour name.	View

Methods:

- **void** LaunchTour([in] **int64** BadgeId, [in] **string** badgeID_str, [in] **int32** MonStationId, [out] **int32** ReturnValue);

Parameters:

- BadgeId - This is the 64-bit badge ID of the badge you want to move.
- badgeID_str - A string representation of the badgeID. You cannot provide both badgeID and badgeID_str in the same call.
- MonStationID - Monitoring station (workstation) ID
- ReturnValue - Result of the guard tour. Possible values:
 - 0: Success
 - 1: Tour already in progress
 - 2: Tour not in progress
 - 3: Invalid tour ID
 - 4: Invalid tour status
 - 5: Invalid badge ID
 - 6: Invalid monitoring station
 - 7: Communication error

Lnl_Holiday

Description: A holiday that is defined in the security system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	SegmentID	Segment to which the holiday belongs.	View
int32	ExtentDays	How many days the holiday lasts	View
datetime (string)	StartDate	Date the holiday starts	View
string	Name	Holiday name.	View

Lnl_HolidayType

Description: A holiday that is defined in the security system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	SegmentID	Segment to which the holiday belongs.	View
string	Name	Holiday name.	View

Lnl_HolidayTypeLink

Description: Defines what holiday type that is associated with a given holiday

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	HolidayID	Holiday. Key field.	View
int32	HolidayTypeID	Holiday type. Key field.	View

Lnl_IncomingEvent

Description: An data class that supports sending incoming events via OpenAccess. This object has no properties; it only has the methods listed below.

Abstract: No

Superclass: Lnl_Element

Platforms: OnGuard

Properties: None

Methods:

- **void** SendIncomingEvent([in] **string** Source, [in] **string** Device, [in] **string** SubDevice, [in] **string** Description, [in] **datetime** Time, [in] **boolean** IsAccessGrant, [in] **boolean** IsAccessDeny, [in] **int64** BadgeID, [in] **string** BadgeID_str, [in] **string(hex)** ExtendedID);

Parameters:

- Source - text representation of the object/device that generated the event
Variable-length Unicode string. This parameter is required. The source must be defined in the OpenAccess Sources folder (in the System Administration application) prior to using the Lnl_IncomingEvent::SendIncomingEvent method. For more information, refer to [Add a Logical Source](#) on page 282.
- Device - text representation of a device associated with a OpenAccess Source that generated the event
Variable-length Unicode string. This parameter is optional. The device must be defined in the OpenAccess Sources folder > OpenAccess Devices tab (in System Administration) prior to using the Lnl_IncomingEvent::SendIncomingEvent method.
- SubDevice - text representation of a sub device associated with a OpenAccess Device that generated the event.
Variable-length Unicode string. This parameter is optional. The device must be defined in the OpenAccess Sources folder > OpenAccess Sub-Devices tab (in System Administration) prior to using the Lnl_IncomingEvent::SendIncomingEvent method.
- Description - text that describes the event
Variable-length Unicode string.
- Time - The time when this event occurred. If this is empty, the current time will be used.
- IsAccessGrant - boolean value that specifies whether the event reported for the OpenAccess Source, Device or Sub-Device will be the “Granted Access” event. This parameter is optional. However, if this parameter is set to true, BadgeID or ExtendedID can be specified to report an “Granted Access” event for a specific OnGuard cardholder. The OpenAccess Source, Device or Sub-Device must be defined in the OpenAccess Sources folder > OpenAccess Devices tab (in the System Administration application) prior to using the Lnl_IncomingEvent::SendIncomingEvent method with the IsAccessGrant parameter set to true. For more information, refer to [Generating Access Granted and Access Denied Events](#) on page 217.
- IsAccessDeny - boolean value that specifies whether the event reported for the OpenAccess Source, Device or Sub-Device will be the “Access Denied” event. This parameter is optional. However, if this parameter is set, then BadgeID or ExtendedID can be specified to report an “Access Denied” event for a specific OnGuard cardholder. The OpenAccess Source, Device or SubDevice must be defined in the OpenAccess Sources folder > OpenAccess Devices tab (in the System Administration application) prior to using the Lnl_IncomingEvent::SendIncomingEvent method with the IsAccessDeny parameter set to

true. For more information, refer to [Generating Access Granted and Access Denied Events](#) on page 217.

- BadgeID - The 64-bit badge ID of the badge in the OnGuard database that generated the event. This parameter is optional and is used in association with all badge related events.
- BadgeID_str - A string representation of the BadgeID. You cannot provide both BadgeID and BadgeID_str in the same call.
- ExtendedID - Extended length string identifier that refers to a PIV-based badge in the OnGuard database that generated the event. Specifies the 128-bit UUID or 200-bit FASC-N. This parameter is optional and is used in association with all badge-related events. This parameter must be in hexadecimal string format. The FASCN or UUID needs to be converted to a binary value that begins with “0x” and includes the values of the FASCN/UUID.

Note: BadgeID is always given precedence over ExtendedID during the search for the badge information to be displayed in Alarm Monitoring.

- **int32** AcknowledgeAlarm([in] **int32** CurrentAckStatus, [in] **int32** SerialNumber, [in] **string** CommServerHostName, [in] **int32** PanelID, [in] **int32** AlarmID, [in] **datetime** AlarmTime, [in] **int32** AckStatus, [in] **string** AckNotes, [out] **int32** SimultaneousAckStatus);

Description:

Allows acknowledgment of alarms received from the system. Most of the parameters can be extracted from the Lnl_LoggedEvent.

Return:

0 - If acknowledgment fails. Examine the SimultaneousAckStatus value to see if the conflict occurred when processing the request.

1 - If acknowledgment succeeds.

Parameters:

- CurrentAckStatus - current acknowledgment status of the alarm to ensure that simultaneous acknowledgment by other means does not interfere with user’s intent. Possible values are:
 - 0 - No. Initial status for an unacknowledged event.
 - 1 - Yes. Acknowledge.
 - 2 - Note. Acknowledge with note.
 - 3 - In-Progress. Mark event as “in-progress”
- SerialNumber - serial number of the event to acknowledge
- CommServerHostName - host name of the Communication server through which the event arrived
- PanelID - Panel ID associated with the event to ensure the integrity of the acknowledgment request
- AlarmID - Event type ID associated with the event to ensure the integrity of the acknowledgment request
- AlarmTime - Time the event occurred to ensure the integrity of the acknowledgment request
- AckStatus - Acknowledgment status to set. See the CurrentAckStatus parameter description for possible values.
- AckNotes - Acknowledgment notes to set. AckStatus must be 2.
- SimultaneousAckStatus - Value greater than 0 if alarm had been acknowledged by other means. Contains the new acknowledgment status if that was the case. See the CurrentAckStatus parameter description for possible values.

Note: Return value of 4 indicates that no simultaneous acknowledgment occurred.

Generating Access Granted and Access Denied Events

The `IsAccessGrant`, `IsAccessDeny`, `Badge ID` and `ExtendedID` parameters can be used to generate access granted and access denied events as follows:

- `IsAccessGrant` and `IsAccessDeny` are mutually exclusive (i.e., either one or the other can be set to true but not both).
- If `IsAccessGrant` or `IsAccessDeny` is set to true, any text that may be specified for the `Description` parameter will be ignored.

Notes: When a user writes a script that invokes the `Lnl_IncomingEvent::SendIncomingEvent` method, he or she may optionally specify the `IsAccessGrant` or `IsAccessDeny` parameters to generate “Granted Access” or “Access Denied” events respectively.

The above functionality will work similarly if the name of the `Source` and `Device` parameters correspond to an `Access` panel and `Reader` configured in the system. If these conditions are met then the “Granted Access” or “Access Denied” events will be reported for the specified `Access` panel and `Reader` based on how the `IsAccessGrant` and `IsAccessDeny` parameters are set.

Using Device and SubDevice in Scripts

A script that invokes the `Lnl_IncomingEvent::SendIncomingEvent` method may optionally include the `Device` and `SubDevice` name. These parameters are reported (to Alarm Monitoring) in the following manner:

- If the `Device` name is empty, the event will only be reported for the `OpenAccess Source`
- If the `Device` name exists and is found in the `OnGuard` database, the event will be reported for the `OpenAccess Device` (i.e., `Controller` and `Device` columns respectively show the `OpenAccess Source` and `OpenAccess Device` that generated the alarm).
- If the `SubDevice` name exists and is found in the `OnGuard` database, the event will be reported for the `OpenAccess Sub-Device` (i.e., `Controller`, `Device`, and `Input/Output` columns respectively show the `OpenAccess Source`, `OpenAccess Device`, and `OpenAccess Sub-Device` that generated the alarm).

Note: The `OpenAccess Source`, `Device`, and `SubDevice` names must all match what has been configured in the `OnGuard` database in order for the event to be reported in Alarm Monitoring.

Lnl_Input

Description: Abstract class that represents any kind of input.

Abstract: Yes

Access: View

Superclass: `Lnl_Element`

Platforms: `OnGuard`

Properties:

Type	Name	Description	Access
string	HostName	The name of the workstation where the communication server associated with the input's panel is running.	View
string	Name	The name of the input.	View
int32	PanelId	The ID of the associated access panel. Reference to Lnl_Panel.ID.	View

Lnl_IntrusionArea

Description: Implements the control methods for the Intrusion Area. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	AreaNumber	The number of the associated intrusion area.	View
int32	AreaType	The type of the associated intrusion area.	View
string	HostName	The name of the workstation where the communication server associated with the intrusion panel is running.	View
string	Name	The name of the associated intrusion area.	View
int32	PanelId	The ID of the associated intrusion panel. Reference to Lnl_Panel.ID.	View

Methods:

```
void Arm([in] int32 armState);
```

armState - the desired arm state of the area. Values include:

Value	Name	Description
1	PerimeterArm	Sends a command to perform a perimeter arm.
2	EntirePartitionArm	Sends a command to perform an entire partition arm.
3	MasterDelayArm	Sends a command to perform a delayed master arm.
4	MasterInstantArm	Sends a command to perform an instant master arm.
5	PerimeterDelayArm	Sends a command to perform a delayed perimeter arm.
6	PerimeterInstantArm	Sends a command to perform an instant perimeter arm.
7	PartialArm	Sends a command to perform a partial arm.
9	AwayArm	Sends a command to perform an away arm.
10	AwayForcedArm	Sends a command to perform an away forced arm.
11	StayArm	Sends a command to perform a stay arm.
12	StayForcedArm	Sends a command to perform a stay forced arm.

void Disarm()

Sends a command to disarm the area.

void SilenceAlarms ()

Sends a command to silence area alarms.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:	
OFFLINE_STATUS	0x00
ONLINE_STATUS	0x01

Lnl_IntrusionDoor

Description: Implements the control methods for the Intrusion Door.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	DeviceId	The ID of the intrusion door. Key field.	View
int32	ID	Internal database ID. Key field.	View
int32	PanelId	The ID of the associated intrusion panel. Key field. Reference to Lnl_Panel.ID.	View
string	HostName	The name of the workstation where the communication server associated with the intrusion panel is running.	View
string	Name	The name of the associated intrusion door.	View

Methods:

void Open()

Sends a command to open the intrusion door.

void SetMode([in] int32 Mode);

Sends a command to change the intrusion door mode.

Parameters:

int32 Mode: Intrusion door mode to be set. Allowed values are:	
INTRUSION_DOOR_MODE_LOCKED	1
INTRUSION_DOOR_MODE_UNLOCKED	2
INTRUSION_DOOR_MODE_SECURED	3
INTRUSION_DOOR_MODE_ENABLED	4
INTRUSION_DOOR_MODE_OFFLINE	5

void GetHardwareStatus([out] uint32 Status);

Retrieves the hardware status for the device. Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – door status:		
uint32 Status	Description	Device status
INTRUSION_DOOR_BIT_LEARN_MODE	Door bit learn mode	0x01
INTRUSION_DOOR_BIT_DIAGNOSTIC_MODE	Door bit diagnostic mode	0x02

uint32 Status – door status:		
uint32 Status	Description	Device status
INTRUSION_DOOR_BIT_NOT_INSTALLED	Door bit not installed	0x04
INTRUSION_DOOR_BIT_SDI_FAILURE	Door bit SDI failure	0x08
INTRUSION_DOOR_BIT_HELD	Door bit held	0x10
INTRUSION_DOOR_BIT_FORCED_OPEN	Door bit forced open	0x20
INTRUSION_DOOR_BIT_UNBLOCKED	Door bit unblocked	0x40

Lnl_IntrusionOutput

Description: Abstract class that inherits from Lnl_Output. Declares the relay control methods and represents an output device of the Intrusion Panel.

Abstract: Yes

Access: View

Superclass: Lnl_Output

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	DeviceId	The ID of the intrusion output. Key field.	View
int32	PanelId	The ID of the associated intrusion panel. Key field. Reference to Lnl_Panel.ID.	View
string	HostName	The name of the workstation where the communication server associated with the intrusion panel is running.	View
string	Name	The name of the intrusion output.	View

Lnl_IntrusionZone

Description: Implements the control methods for the Intrusion Zone.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	DeviceID	The ID of the intrusion zone. Key field.	View
int32	ID	Internal database ID. Key field.	View
int32	PanelID	The ID of the associated intrusion panel. Key field. Reference to Lnl_Panel.ID.	View
string	HostName	The name of the workstation where the communication server associated with the intrusion panel is running.	View
string	Name	The name of the associated intrusion zone.	View

Methods:

`void Bypass()`

Sends a command to open by pass the alarm zone.

`void UnBypass();`

Sends a command to un-bypass the alarm zone.

`void GetHardwareStatus([out] uint32 Status)`

Retrieves the hardware status for the device. Status is only retrieved from the hardware when the `UpdateHardwareStatus` is called on the parent ISC.

uint32 Status – device status:	
OFFLINE_STATUS	0x00
ONLINE_STATUS	0x01

Lnl_LoggedEvent

Description: Represents a hardware event that has been logged to the database.

Notes: When requesting instances of `Lnl_LoggedEvent` with a `get instances` call, a filter is required due to the large number of instances this class usually contains. Also, be careful what you specify as the `order_by` value. If left blank, the key values (`PanelID`, `SerialNumber`) are used, which works well.

You can also specify `Time` as the `order_by` value. If you filter by `Time`, you will improve performance if you also `order_by Time`. However, it is not recommended to use any other combination without an index in place on the `EVENTS` table, as doing so might generate a timeout error. For more information, refer to [Error Messages](#) on page 289.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	SerialNumber	Serial number of the event. Key field.	View
int32	PanelID	Panel at which the event occurred. Key field. Reference to Lnl_Panel.ID.	View
datetime (string)	Time	Time when event occurred.	View
string	Description	Description of the event.	View
int32	DeviceID	Device ID at which event occurred (Lnl_Reader, Lnl_AlarmPanel, and so on.)	View
string(hex)	ExtendedID	Extended identifier of the card (where available) which caused the event.	View
int32	SecondaryDeviceID	Secondary device ID at which event occurred (ex. Lnl_Input).	View
int32	SegmentID	Segment where event occurred.	View
int32	Type	Event type i.e., "duress", "system", etc. Corresponds to Lnl_EventSubtypeDefinition.TypeID and Lnl_EventType.ID.	View
int32	SubType	Event sub-type i.e., "granted", "door forced open", etc. Corresponds to Lnl_EventSubtypeDefinition.SubTypeID.	View
string	EventText	Text associated with event.	View
int64	CardNumber	Card (where available) which caused the event.	View

Type	Name	Description	Access
string	CardNumber_str	A string representation of the Card Number. To accurately display Card Number, web clients should use this property instead of the ID property, since there is a JavaScript limitation in which integer values with 18 digits or more are rounded off. Note: This property is only returned when get instances is called with Version 1.2 or later.	View
int32	IssueCode	Issue code of the card.	View
int32	AssetID	Asset (where available) which caused the event.	View
int32	AccessResult	The level of access that was granted that resulted from reading the card. Possible values: 0: Other 1: Unknown 2: Granted 3: Denied 4: Not Applicable Note: You cannot filter using the AccessResult property. For more information, refer to Searching for Objects on page 39.	View
boolean	CardholderEntered	Whether entry was made by the cardholder.	View
boolean	Duress	Indicates whether this card access indicates an under duress/emergency state.	View
int32	PersonID	Internal ID of the person who is assigned the badge at the time of the access event. See Lnl_Person.ID.	View
int32	Priority	Alarm priority (0 to 255).	View
int32	PriorityColorRed-Value	The red component of the RGB color for the alarm (0 to 255).	View
int32	PriorityColorGreen-Value	The green component of the RGB color for the alarm after it is acknowledged (0 to 255).	View

Type	Name	Description	Access
int32	PriorityColorBlue-Value	The blue component of the RGB color for the alarm (0 to 255).	View
int32	PriorityColorAckRed-Value	The red component of the RGB color for the alarm after it is acknowledged (0 to 255).	View
int32	PriorityColorAck-GreenValue	The green component of the RGB color for the alarm after it is acknowledged (0 to 255).	View
int32	PriorityColorAck-BlueValue	The blue component of the RGB color for the alarm after it is acknowledged (0 to 255).	View

LnI_LogicalDevice

Description: A third-party logical device.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	SourceID	ID of the logical source to which this logical device belongs (LnI_LogicalSource.ID). Key field.	Read
string	Name	Name of the logical device	Edit

LnI_LogicalSource

Description: A third-party logical source.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View

Type	Name	Description	Access
boolean	IsDaylightSaving	Identifies if the logical source follows Daylight Saving Time rules. True = Follows Daylight Saving Time rules	Edit
boolean	IsOnline	Identifies if the logical source is online. True = Is online	Edit
string	Name	Name of the logical source.	Edit
int32	SegmentID	Segment to which the logical source belongs.	Read
int32	WorldTimezoneID	Reference to Lnl_WorldTimezone.ID	Edit

Lnl_LogicalSubDevice

Description: A third-party logical sub-device.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	DeviceID	ID of the logical device to which this logical sub-device belongs (Lnl_LogicalDevice.ID). Key field.	Read
int32	ID	Internal database ID. Key field.	View
int32	SourceID	Reference to Lnl_LogicalSource.ID. Key field.	Read
string	Name	Name of the logical sub-device.	Edit

Lnl_MonitoringZone

Description: A Monitoring zone defined in the system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	Name	Monitoring zone name.	View
int32	DefaultMapID	Map ID assigned as the default map to the monitoring zone. Set to 0 when no map is assigned.	View
int32	SegmentID	Segment to which the monitoring zone belongs.	View

LnI_MonitoringZoneCameraLink

Description: Defines what cameras are associated with a given monitoring zone.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Type	Name	Description	Access
int32	CameraID	Camera ID. Key field. See LnI_Camera.ID.	View
int32	MonitoringZoneID	Monitoring Zone ID. Key field. See LnI_MonitoringZone.ID.	View
int32	PanelID	Panel ID for the camera. Key field. Reference to LnI_Panel.ID.	View

LnI_MonitoringZoneDeviceLink

Description: Defines what devices are associated with a given monitoring zone.

Abstract: No

Access: View/Add/Delete

Superclass: LnI_Element

Platforms: OnGuard

Type	Name	Description	Access
int32	MonitoringZoneID	Monitoring Zone ID. Key field. Required field. See LnI_MonitoringZone.ID.	Read

Type	Name	Description	Access
int32	PanelID	Panel ID for the device. Key field. Required field. Reference to Lnl_Panel.ID.	Read
int32	DeviceID	Device ID. Key field. Set to 0 when the device being linked to a monitoring zone is a panel type. Required.	Read
int32	InputOutputID	Input or output ID. Key field. Set to 0 when the device being linked to a monitoring zone is not an input or output. Required.	Read
boolean	AllDevicesOnPanel	Required. True if all sub-devices are included in this monitoring zone. False if individual sub-devices are to be specified. Required.	Read

Lnl_MonitoringZoneRecorderLink

Description: Defines what Lenel NVR Video Recorders are associated with a given monitoring zone.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_MonitoringZoneDeviceLink

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	MonitoringZoneID	Monitoring Zone ID. Key field. Required field. See Lnl_MonitoringZone.ID.	Read
int32	PanelID	Panel ID for the device. Key field. Required field. Reference to Lnl_Panel.ID.	Read
int32	DeviceID	Device ID. Key field. Set to 0 when the recorder itself is being linked to the monitoring zone and not one of its sub-devices. Required.	Read
int32	InputOutputID	Input or Output ID. Key field. Set to 0 when the device being linked to a monitoring zone is not an input or output. Required.	Read

Type	Name	Description	Access
string	Name	The name of the panel.	View
boolean	AllDevicesOnPanel	Required. True if all sub-devices are included in this monitoring zone. False if individual sub-devices are to be specified.	Read

Note: If **Create/save photo thumbnails** is selected on the **System Administration > Cardholder Options > General Cardholder Options** form, then the thumbnail is automatically created and saved when an Lnl_MultimediaObject is added.

Lnl_MultimediaObject

Description: An image, signature, document, or biometric template belonging to a person in the security system.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
sint32	DATATYPE	Data type. Key field. For possible values, refer to DATATYPE and OBJECTTYPE Pairings on page 231.	Read
sint32	OBJECTTYPE	Object type. Key field. For possible values, refer to DATATYPE and OBJECTTYPE Pairings on page 231.	Read
sint32	PERSONID	Internal ID of the person who owns this object. See Lnl_Person.ID.	Read
binary	DATA	Array of image data.	Read
datetime (string)	LASTCHANGED	Image last changed	View

Notes: DATATYPE and OBJECTTYPE properties must remain paired as shown in [DATATYPE and OBJECTTYPE Pairings](#) on page 231.

The maximum file size that Lnl_MultimediaObject can send by default is less than 1 MB. To allow for larger file sizes, add the **client_max_body** line to the http block in the **nginx.conf** file (located by default in **C:\ProgramData\Lnl\nginx\conf**):

```
http {  
    ...  
    client_max_body_size 10M;  
}
```

This example uses 10 MB as the maximum file size, but you can use any value that you feel is appropriate.

DATATYPE and OBJECTTYPE Pairings

Multimedia Object Type	DATATYPE	OBJECTTYPE
Photo Image	0	1
Photo Image Mask	1	1
Thumbnail	2	1
Signature	0	8
Hand Geometry (RSI)	4	16
LG Iris Code (right eye)	6	64
LG Iris Code (left eye)	7	64
LG Iris Image (right eye)	8	64
LG Iris Image (left eye)	9	64
Bioscrypt Fingerprint Template (primary)	3	32
Bioscrypt Fingerprint Template (secondary)	3	96
Bioscrypt Fingerprint Image (primary)	0	32
Bioscrypt Fingerprint Image (secondary)	0	96
ANSI INCITS 378 Template (primary)	11	112
ANSI INCITS 378 Template (secondary)	12	112
PK_COMP Template (primary)	11	128
PK_COMP Template (secondary)	12	128
Biometric PIN	-1	512
Visitor PDF Document	13	513

Lnl_OffBoardRelay

Description: Inherits from Lnl_Output, and therefore has the same properties. Implements the relay control methods and represents an Off-Board relay connected to the Intrusion Panel. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_IntrusionOutput

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View

Type	Name	Description	Access
int32	DeviceId	The ID of the intrusion output. Key field.	View
int32	PanelId	The ID of the associated intrusion panel. Key field. Reference to Lnl_Panel.ID.	View
string	HostName	The name of the workstation where the communication server associated with the intrusion panel is running.	View
string	Name	The name of the intrusion output.	View

Methods:

void Activate()

Sends a command to activate a specific alarm relay.

void Deactivate()

Sends a command to deactivate a specific alarm relay.

void Toggle();

Toggles the state of the specific alarm relay.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:		
uint32 Status	Description	Device status
ALRM_STATUS_SECURE	Output Secure	0
ALRM_STATUS_ACTIVE	Output Active	1

Lnl_OnBoardRelay

Description: Inherits from Lnl_Output, and therefore has the same properties. Implements the relay control methods and represents an On-Board relay of the Intrusion Panel. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_IntrusionOutput

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	DeviceId	The ID of the on-board relay. Key field.	View
int32	PanelId	The ID of the associated intrusion panel. Key field. Reference to Lnl_Panel.ID.	View
string	HostName	The name of the workstation where the communication server associated with the intrusion panel is running.	View
string	Name	The name.	View

Methods:

`void Activate()`

Sends a command to activate a specific alarm relay.

`void Deactivate()`

Sends a command to deactivate a specific alarm relay.

`void GetHardwareStatus([out] uint32 Status)`

Status is only retrieved from the hardware when the `UpdateHardwareStatus` is called on the parent ISC.

uint32 Status – device status:		
uint32 Status	Description	Device status
ALRM_STATUS_SECURE	Output Secure	0
ALRM_STATUS_ACTIVE	Output Active	1

Lnl_Output

Description: Abstract class that represents any kind of output.

Abstract: Yes

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelId	The ID number of the associated access panel. Reference to Lnl_Panel.ID. Key field.	View
string	HostName	The name of the workstation where the communication server associated with the output's panel is running.	View
string	Name	The name of the associated output.	View

Lnl_Panel

Description: A panel defined in the security system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
boolean	IsDaylightSaving	Whether or not this panel observes Daylight Saving Time	View
boolean	IsOnline	The panel is online.	View
string	NAME	Display name.	View
string	PANELTYPE	Panel type name.	View
string	ComputerName	The hostname of the panel, for panels that are specified using a hostname rather than an IP address.	View

Type	Name	Description	Access
int32	PanelClass	The class of the panel: 1 = access control panel 2 = fire panel 3 = intercom panel 4 = CCTV (video recorder) 5 = offline lock 6 = personal safety device 7 = intrusion panel 8 = switcher 9 = PC 10 = receiver 11 = account 12 = SNMP manager 13 = SNMP agent 14 = OPC connection 15 = OpenIT source 17 = point of sale device 18 = intelligent video server 19 = video monitor 20 = elevator dispatching device 21 = video aux server 22 = caching status proxy	View
string	PrimaryDialupHost-Number	The primary phone number to use when connecting to a server with dial-up access.	View
int32	PrimaryIPAddress	The primary IP address to use when connecting to a server with network access.	View
string	SecondaryDialupHost-Number	The back-up phone number to use when connecting to a server with dial-up access.	View
int32	SEGMENTID	Segment to which the panel belongs.	View
int32	WorldTimezoneID	Time zone of the panel (reference to LnI_WorldTimezone.ID)	View
string	WORKSTATION	Panel workstation name.	View

Methods:

void DownloadFirmware()

Sends a download firmware command to the ISC.

`void DownloadDatabase()`

Sends a command to the ISC to download the cardholder database.

`void ResetUseLimit()`

Sends a command to reset the use limit of all cardholders within the ISC.

`void UpdateHardwareStatus()`

Sends a command to retrieve the status of the Intelligent System controller and all downstream hardware connected to the specific system controller.

`void Connect()`

Used for dial-up only. This command instructs the host to connect to the ISC via dial-up.

`void Disconnect()`

Used for dial-up only. This command instructs the host to send a disconnect command to the ISC.

`void SetClock()`

Sends the current time down to the ISC.

`void GetHardwareStatus([out] uint32 Status)`

Retrieves the hardware status for the device. Status is only retrieved from the hardware when `UpdateHardwareStatus` is called on the parent ISC. If the device is offline, the status is returned with a value of "0".

uint32 Status – device status:		
uint32 Status	Description	Device status
ONLINE_STATUS	Online	0x01
OPTIONS_MISMATCH_STATUS	Options Mismatch	0x02
CABINET_TAMPER	Cabinet Tamper	0x04
POWER_FAIL	Power Failure	0x8
DOWNLOADING_FIRMWARE	Downloading Firmware	0x10

Lnl_Person

Description: A cardholder or visitor in the security system.

Abstract: Yes

Access: View

Superclass: `Lnl_Element`

Platforms: OnGuard

Properties:

Note: The properties listed below with Edit access are editable only through instances of Lnl_Cardholder and Lnl_Visitor.

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	FIRSTNAME	First name.	Edit
datetime (string)	LASTCHANGED	Person last changed	View
string	LASTNAME	Last name.	Edit
string	MIDNAME	Middle name.	Edit
int32	DATABASEID	The database identifier in an Enterprise system that identifies the system containing the cardholder data.	View

Lnl_PersonSecondarySegments

Description: An association between a person and that person's assigned secondary segments. Present only in segmented systems where cardholder or visitor segmentation is enabled.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PERSONID	Reference to Lnl_Person.ID. Cardholder or Visitor ID. Key field.	Read
int32	SEGMENTID	Secondary segment to which the person belongs. Key field.	Read

Lnl_PrecisionAccessGroup

Description: A defined set of unique access privileges for assignment to individual cardholders. Only present if the system is configured to use precision access. For more information, refer to "Precision Access Form" in the *System Administration User Guide*.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	The ID of the precision access group. Key field.	View
string	Name	The name of the precision access group.	View
int32	Type	The type of precision access group. Possible values: 1 (Inclusion), 2 (Exclusion)	View
int32	SegmentID	The ID of the segment associated with the precision access group.	View

Lnl_PrecisionAccessGroupAssignment

Description: An assignment relationship between a badge and a precision access group. Only present if the system is configured to use precision access. For more information, refer to “Precision Access Form” in the *System Administration User Guide*.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	BadgeKey	A key value uniquely identifying a badge. Key field.	Read
int32	PrecisionAccessGroupID	The ID of the precision access group assigned to the badge. Key field.	Read

Lnl_ProhibitedPassword

Description: The prohibited password list defined in the system.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
sint32	ID	Internal database ID. Key field.	View
string	Password	The prohibited password list.	Edit

Lnl_PTZPreset

Description: PTZ presets configured by the OnGuard software.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PresetID	Preset ID. Key field.	View
int32	CameraPreset	Preset number stored on the camera.	View
int32	Channel	Channel ID of the recorder.	View
int32	Duration	Number of seconds, applicable to continuous preset (PresetType=3).	View
Float	Focus	Value of the focus.	View
Float	Iris	Value of the iris.	View
string	Name	Name of the preset.	View
Float	Pan	Value of the pan.	View
int32	PanelID	Value of the recorder.	View
int32	PresetType	Type of PTZ preset. 1 = Absolute 2. = Relative 3 = Continuous 4 = Camera preset	View
Float	Tilt	Value of the tilt.	View
Float	Zoom	Value of the zoom.	View

Lnl_Reader

Description: A reader defined in the security system.

Abstract: No

Access: View/Modify

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	AccessMode	The reader mode setting for when the reader is online and communicating with the access panel. For more information, refer to the Reader Mode table below.	View
int32	Address	The address of the reader (0 to 31).	View
string	Aux1Name	The name of the first auxiliary input.	View
string	Aux2Name	The name of the second auxiliary input.	View
string	Aux3Name	The name of the third auxiliary input.	View
int32	ControlType	The type of reader.	View
int32	ExtendedOpenTime	For Lenel hardware only. Specifies the held open time for badges with the extended strike/held times feature enabled. This field is intended for anyone who needs extra time to proceed through a doorway. Values range from 1 to 131070 seconds.	View
int32	ExtendedStrikeTime	For Lenel hardware only. Specifies the reader strike time for badges with the extended strike/held times feature enabled. This field is intended for anyone who needs extra time to proceed through a doorway. Values range from 1 to 255 seconds.	View
string	FriendlyName	A descriptive name for the reader.	Edit
int32	GatewayAddress	Address of the SimonsVoss gateway to which the reader belongs.	View
string	GatewayHostName	Hostname of the SimonsVoss gateway to which the reader belongs.	View
int32	GatewayIPPort	The port number of the SimonsVoss Gateway to which the reader belongs.	View

Type	Name	Description	Access
string	HostName	The name of the workstation where the communication server associated with this reader's panel is running.	View
bool	IsPairedMaster	If true, indicates that the reader is the master reader of a paired set of readers.	View
bool	IsPairedSlave	If true, indicates that the reader is the slave reader of a paired set of readers.	View
string	Name	Display name.	View
int32	OfflineMode	The reader mode setting for when communication is lost between the reader and the access panel. For more information, refer to the Reader Mode table below.	View
int32	OpenTime	The number of seconds the door can be held open before an alarm is generated. For Lenel hardware, values range from 1 to 131070 seconds. For other types of hardware, values range from 1 to 255 seconds.	View
string	Out1Name	The name of the first reader output.	View
string	Out2Name	The name of the second reader output.	View
int32	PanelID	ID of the panel to which this reader belongs. Key field. Reference to Lnl_Panel.ID.	View
string	PanelTypeName	The panel type name.	View
int32	PortNumber	The number of the port on the access panel to which the reader is attached.	View
int32	ReaderID	Internal database ID. Key field.	View
int32	ReaderNumber	A number that differentiates this reader from other readers using the same port and address. Values typically range from 0 to 7, but may vary depending on reader type.	View
int32	SlaveID	If IsPairedMaster is true, this is the ID of the associated slave reader of the paired set of readers. Reference to Lnl_Reader.ReaderID.	View

Type	Name	Description	Access
int32	StrikeTime	The number of seconds a strike or lock is open (activated) when access is granted. Typically, this is set from 5 to 10 seconds, but possible values range from 1 to 255 seconds.	View
int32	TimeAttendanceType	The time and attendance reader configuration. not used = 0 (or <empty>) Entrance Reader = 1 Exit Reader = 2	View

Methods:

`void OpenDoor()`

Sends a command to open the door for a specific reader.

`void SetMode([in] int32 Mode)`

Sends a command to set the current operating mode of a reader.

`void GetMode ([out] int32 Mode)`

Retrieves current mode of the reader. Mode is only retrieved from the hardware when the `UpdateHardwareStatus` is called on the parent ISC.

Parameters:

int32 Mode: Reader mode to be set. Allowed values are:	
MODE_LOCKED	0x0
MODE_CARDONLY	0x1
MODE_PIN_OR_CARD	0x2
MODE_PIN_AND_CARD	0x3
MODE_UNLOCKED	0x4
MODE_FACCODE_ONLY	0x5
MODE_CYPHERLOCK	0x6
MODE_AUTOMATIC	0x7
MODE_UNLOCK_NEXT_EXIT	0x25
MODE_DEFAULT	0x100

You can set the current mode of the reader to an authentication mode using the ID retrieved with the `Lnl_AuthenticationMode` class. Authentication mode IDs are not static like the system-defined reader modes in the table above.

`void SetBiometricVerifyMode([in] boolean Value)`

Sends a command to enable/disable the biometric mode of verification for a reader.

Note: Using this method requires that you configure at least one biometric type for the reader's controller. You must also configure the desired biometric template type to greater than 0 on the **System Options > Biometrics** tab.

Parameters:

boolean Value: True – enable biometric mode of verification. False – disable biometric mode of verification.

void SetFirstCardUnlockMode([in] boolean Value)

Sends a command to enable/disable first card unlock mode for the reader.

Note: Using this method requires that you enable the First Card Unlock option on the reader's controller.

Parameters:

boolean Value: True – enable first card unlock mode. False – first card unlock mode.

void DownloadFirmware()

Sends a download firmware command to the reader interface module.

void GetHardwareStatus([out] uint32 Status)

Retrieves the hardware status for the device. Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:		
uint32 Status	Description	Device status
RDRSTATUS_ONLINE	Online	0x1
RDRSTATUS_OPTION_MISMATCH	Options Mismatch	0x2
RDRSTATUS_CNTTAMPER	Cabinet Tamper	0x4
RDRSTATUS_PWR_FAIL	Power Failure	0x8
RDRSTATUS_TAMPER	Reader Tamper	0x10
RDRSTATUS_FORCED	Door Forced Open	0x20
RDRSTATUS_HELD	Door Held Open	0x40
RDRSTATUS_AUX	Auxiliary Input 1	0x80
RDRSTATUS_AUX2	Auxiliary Input 2	0x100
RDRSTATUS_AUX3	Auxiliary Input 3	0x400
RDRSTATUS_BIO_VERIFY	Bio Verify	0x800
RDRSTATUS_DC_GND_FLT	DC Ground Fault	0x1000

uint32 Status – device status:		
uint32 Status	Description	Device status
RDRSTATUS_DC_SHRT_FLT	DC Short Fault	0x2000
RDRSTATUS_DC_OPEN_FLT	DC Open Fault	0x4000
RDRSTATUS_DC_GEN_FLT	DC Generic Fault	0x8000
RDRSTATUS_RX_GND_FLT	RX Ground Fault	0x10000
RDRSTATUS_RX_SHRT_FLT	RX Short Fault	0x20000
RDRSTATUS_RX_OPEN_FLT	RX Open Fault	0x40000
RDRSTATUS_RX_GEN_FLT	RX Generic Fault	0x80000
RDRSTATUS_FIRST_CARD_UNLOCK	First Card Unlock Mode	0x100000
RDRSTATUS_EXTENDED_HELD_MODE	Extended Held Mode	0x200000
RDRSTATUS_CIPHER_MODE	Cipher Mode	0x400000
RDRSTATUS_LOW_BATTERY	Low Battery	0x800000
RDRSTATUS_MOTOR_STALLED	Motor Stalled	0x1000000
RDRSTATUS_READHEAD_OFFLINE	Read Head Offline	0x2000000
RDRSTATUS_MRDT_OFFLINE	MRDT Offline	0x4000000
RDRSTATUS_DOOR_CONTACT_OFFLINE	Door Contact Offline	0x8000000
RDRSTATUS_LOCKED	Reader's status is locked	0x80000000
RDRSTATUS_UNLOCKED	Reader's status is unlocked	0x100000000
RDRSTATUS_UNLOCK_NEXT_EXIT	Reader's status will be unlocked after the next exit	0x200000000

Lnl_ReaderInput

Description: Abstract class, inherits from Lnl_Input. Declares the input control methods and represents an auxiliary input found on a reader interface module.

Abstract: Yes

Access: View

Superclass: Lnl_Input

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelId	The ID of the associated panel. Key field. Reference to Lnl_Panel.ID.	View
int32	ReaderId	The ID of the associated reader. Key field.	View
string	HostName	The name of the workstation where the communication server associated with the reader's access panel is running.	View
string	Name	The name of the associated reader input.	View

Lnl_ReaderInput1

Description: Inherits from Lnl_ReaderInput. Declares the input control methods and represents the first auxiliary input found on a reader interface module. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_ReaderInput

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelId	The ID of the associated panel. Key field. Reference to Lnl_Panel.ID.	View
int32	ReaderId	The ID of the associated reader. Key field.	View
string	HostName	The name of the workstation where the communication server associated with the reader's access panel is running.	View
string	Name	The name of the associated reader input.	View

Methods:

void Mask();

Sends a command to mask a specific reader input.

void Unmask();

Sends a command to unmask a specific reader input.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:	
ALRM_STATUS_SECURE	0x00
ALRM_STATUS_ACTIVE	0x01
ALRM_STATUS_GND_FLT	0x02
ALRM_STATUS_SHRT_FLT	0x03
ALRM_STATUS_OPEN_FLT	0x04
ALRM_STATUS_GEN_FLT	0x05
OA_HW_STATUS_MASK_INPUT_MASKED	0x100

Lnl_ReaderInput2

Description: Inherits from Lnl_ReaderInput. Declares the input control methods and represents the second auxiliary input found on a reader interface module. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_ReaderInput

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelId	The ID of the associated panel. Key field. Reference to Lnl_Panel.ID.	View
int32	ReaderId	The ID of the associated reader. Key field.	View
string	HostName	The name of the workstation where the communication server associated with the reader's access panel is running.	View
string	Name	The name of the associated reader input.	View

Methods:

void Mask();

Sends a command to mask a specific reader input.

void Unmask();

Sends a command to unmask a specific reader input.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:	
ALRM_STATUS_SECURE	0x00
ALRM_STATUS_ACTIVE	0x01
ALRM_STATUS_GND_FLT	0x02
ALRM_STATUS_SHRT_FLT	0x03
ALRM_STATUS_OPEN_FLT	0x04
ALRM_STATUS_GEN_FLT	0x05
OA_HW_STATUS_MASK_INPUT_MASKED	0x100

Lnl_ReaderOutput

Description: Abstract class, inherits from Lnl_Output. Declares the relay control methods and represents an auxiliary relay found on a reader interface module.

Abstract: Yes

Access: View

Superclass: Lnl_Output

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelId	The ID of the associated panel. Key field. Reference to Lnl_Panel.ID.	View
int32	ReaderId	The ID of the associated reader. Key field.	View
string	HostName	The name of the workstation where the communication server associated with the reader's access panel is running.	View
string	Name	The name of the associated reader output.	View

Lnl_ReaderOutput1

Description: Inherits from Lnl_ReaderOutput. Implements the relay control methods and represents the first auxiliary relay found on a reader interface module. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_ReaderOutput

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelId	The ID of the associated panel. Key field. Reference to Lnl_Panel.ID.	View
int32	ReaderId	The ID of the associated reader. Key field.	View
string	HostName	The name of the workstation where the communication server associated with the reader's access panel is running.	View
string	Name	The name of the associated reader output.	View

Methods:

void Activate()

Sends a command to activate a specific alarm relay.

void Deactivate()

Sends a command to deactivate a specific alarm relay.

void Pulse()

Sends a momentary pulse command to a specific alarm relay.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:		
uint32 Status	Description	Device status
ALRM_STATUS_SECURE	Output Secure	0
ALRM_STATUS_ACTIVE	Output Active	1

Lnl_ReaderOutput2

Description: Inherits from Lnl_ReaderOutput. Implements the relay control methods and represents the second auxiliary relay found on a reader interface module. Retrieves the hardware status for the device.

Abstract: No

Access: View

Superclass: Lnl_ReaderOutput

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelId	The ID of the associated panel. Key field. Reference to Lnl_Panel.ID.	View
int32	ReaderId	The ID of the associated reader. Key field.	View
string	HostName	The name of the workstation where the communication server associated with the reader's access panel is running.	View
string	Name	The name of the associated reader output.	View

Methods:

void Activate()

Sends a command to activate a specific alarm relay.

void Deactivate()

Sends a command to deactivate a specific alarm relay.

void Pulse()

Sends a momentary pulse command to a specific alarm relay.

void GetHardwareStatus([out] uint32 Status)

Status is only retrieved from the hardware when the UpdateHardwareStatus is called on the parent ISC.

uint32 Status – device status:		
uint32 Status	Description	Device status
ALRM_STATUS_SECURE	Output Secure	0
ALRM_STATUS_ACTIVE	Output Active	1

LnI_ReaderRequest

Description: A request raised by a person for accessing readers.

Abstract: No

Access: View/Add

Superclass: LnI_AccessRequest

Platforms: OnGuard

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	Name	Name of the associated reader.	View
string	ReaderFriendlyName	The descriptive name of the reader for which access is being requested.	View
int32	PanelID	Panel to which access request should be submitted. Key field.	Read
int32	ReaderID	Reader to which access request should be submitted. Key field.	Read
int32	PersonID	Internal ID of the person who requested access to the reader. Key field. See LnI_Person.ID.	Read
int32	Type	Request type ID: 0: Reader	View
int32	Status	Request status ID: 0: Submitted 1: Approved 2: On Hold 3: Denied	View
int32	EventSerialNumber	The serial number of the associated access denied event, if this request was initiated by an access denied event.	Read
datetime (string)	StartDate	Start date the cardholder requests for the reader.	Read
datetime (string)	EndDate	End date the cardholder requests for the reader.	Read
int32	SubmittedByUserID	The user ID of the user who submitted the request.	View
int32	ApprovedByUserID	The user ID of the user who approved the request.	View
int32	DeniedByUserID	The user ID of the user who denied the request.	View

Type	Name	Description	Access
int32	OnHoldByUserID	The user ID of the user who put the request on hold.	View
string	SubmittedNote	Notes entered when submitting this request.	Read
string	ApprovedNote	Notes entered when approving this request.	View
string	DeniedNote	Notes entered when denying this request.	View
string	OnHoldNote	Notes entered when putting this request on hold.	View
datetime (string)	SubmittedDate	The date and time when the request was submitted.	View
datetime (string)	ApprovedDate	The date and time when the request was approved.	View
datetime (string)	DeniedDate	The date and time when the request was denied.	View
datetime (string)	OnHoldDate	The date and time when the request was put on hold.	View
boolean	EmailCardholder	Whether the cardholder is notified.	Read
boolean	EmailAccessManager	Whether the approver is notified.	Read

Methods:

`void Approve([in] string Note, [in] boolean EmailCardholder);`

Approves the Reader Request. setting ApprovedDate to current date/time.

`void Deny([in] string Note, [in] boolean EmailCardholder);`

Denies the Reader Request. setting DeniedDate to current date/time.

`void Hold([in] string Note, [in] boolean EmailCardholder);`

holds the Reader Request. setting OnHoldDate to current date/time.

Parameters:

Note: Notes when the request is approved, denied and put on hold.

EmailCardholder: Whether the cardholder should be notified.

Lnl_RequestableReader

Description: A reader associated with an access level to which cardholders can request access. For more information, refer to the AvailableForRequest property in [Lnl_AccessLevel](#) on page 182.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	PanelID	ID of the panel to which this reader belongs. Key field. Reference to Lnl_Panel.ID.	View
int32	ReaderID	Internal database ID for the requestable reader. Key field.	View
string	Name	Display name for the reader.	View
string	FriendlyName	Descriptive name for the reader.	View

Lnl_Segment

Description: A segment or segment group defined in the security system. Present in segmented systems only.

Abstract: Yes

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	NAME	Display name.	View
string	TYPE	The segment type.	View
string	SERVERNAME	The region server's name. This field is filterable.	View

Lnl_SegmentGroup

Description: A segment group in the security system. Present in segmented systems only. Refer to [Lnl_SegmentGroupMember](#) on page 274 to determine which segments make up a segment group.

Abstract: No

Access: View

Superclass: Lnl_Segment

Platforms: OnGuard

Properties: Same properties as in Lnl_Segment.

Lnl_SegmentUnit

Description: A segment in the security system. Present in segmented systems only.

Abstract: No

Access: View

Superclass: Lnl_Segment

Platforms: OnGuard

Properties: Same properties as in Lnl_Segment.

Lnl_Timezone

Description: A time zone defined in the security system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	SegmentID	Segment ID to which the time zone belongs.	View
string	Name	Name of the timezone.	View

Lnl_TimezoneInterval

Description: A time zone interval used by instances of Lnl_Timezone.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	TimezoneID	Lnl_Timezone of which this interval is a part of. Key field.	View
datetime (string)	StartTime	Time of day when interval becomes active	View

Type	Name	Description	Access
datetime (string)	EndTime	Time of day when interval stops being active	View
boolean	Monday - Sunday	Day of the week when interval is active. There are seven individual boolean properties, one for each day of the week.	View
boolean	HolidayType1 - HolidayType8	Holiday type during which the interval is active. There are eight individual boolean properties, one for each holiday type.	View

Lnl_User

Description: A user defined in the system.

Abstract: No

Access: View/Add /Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Note: When modifying an Lnl_User instance, if the **password** property provided in the modify request is null or an empty string, the existing password is unchanged. For all other properties, providing null or an empty string in the modify request will set that property to null.

Type	Name	Description	Access
string	ID	Internal database ID. Key field.	View
string	LogonID	Internal Account User name.	Edit
string	Password	Internal Account Password. This property cannot be viewed.	Edit
string	FirstName	First Name.	Edit
string	LastName	Last Name.	Edit
boolean	Enabled	Determines whether user is enabled	Edit
boolean	HasInternalAccount	If true, indicates that the user has an internal account.	Edit

Type	Name	Description	Access
boolean	IgnorePasswordExpiration	If true, indicates that this user's password never expires. The sa account and SA delegate accounts are exceptions: this property is always false for the sa user and SA delegate users, and cannot be set to true.	Edit
boolean	SAPowers	If true, indicates that the user is an SA Delegate with all permissions enabled. A user can only be given SA Delegate permissions at the time the user is created by the Root SA or by another SA Delegate user; existing users cannot be converted into SA Delegate users. For security, the first SA Delegate user created should log into the system and disable the Root SA user.	Read
sint32	SystemPermissionGroupID	System User Permission Group. See Lnl_UserPermissionGroup.ID.	Edit
sint32	MonitoringPermissionGroupID	Monitor User Permission Group. See Lnl_UserPermissionGroup.ID.	Edit
sint32	CardPermissionGroupID	Cardholder User Permission Group. See Lnl_UserPermissionGroup.ID.	Edit
sint32	ReportPermissionGroupID	Indicates the Report Permission Group ID. This is a required field, but defaults to 0 which provides no report permissions.	Edit
sint32	FieldPermissionID	Field/Page Access Group. Reference to Lnl_UserFieldPermissionGroup.ID.	Edit
sint32	SegmentID	User's Segment ID This property cannot be viewed. Use Lnl_UserSecondarySegments to see a full list of the user's segments.	Read
sint32	MonitoringZoneID	Monitoring Zone ID. Reference to Lnl_MonitoringZone.ID.	Edit
datetime (string)	Created	Date user was created	View
datetime (string)	LastChanged	Date user was modified	View
string	Notes	Notes associated with the user. This field is not filterable.	Edit

Type	Name	Description	Access
boolean	AutomaticallyCreated	An automatic user is one that has been created in “bulk” using the Bulk User Tool. This property is set to false for all users except those created using the Bulk User Tool. It is included in the application programming interface (API) for filtering only.	View
boolean	PasswordChangeRequired	Determines if the user is forced to change the password at the next login.	Edit
boolean	IsPasswordCaseSensitive	Determines if the user's password is case sensitive.	View
sint32	DatabaseID	The database identifier in an Enterprise system that identifies the replication setting for the User. The value has a default value of 'Local System Only' which matches the default through the OnGuard software.	Edit

LnI_UserAccount

Description: An association between a user and its directory account.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
sint32	ID	Internal database ID. Key field.	View
string	UserID	Internal ID of the user who owns this account. See LnI_User.ID. Key field.	Read
string	AccountID	ID of the entry in the external directory. The ID is the value of the attribute specified in the LnI_Directory.AccountIDAttr property. For example, for Microsoft directories, this property would contain the account's security identifier (SID).	View/Edit

Type	Name	Description	Access
string	DirectoryID	Internal ID of the directory to which this account belongs. See Lnl_Directory.ID.	View/Edit

Lnl_UserPermissionGroup

Description: A user permission group defined in the system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
sint32	ID	Internal database ID. Key field.	View
string	Name	Permission Group name.	View
sint32	Type	Permission Group Type: System = 1 Cardholder = 2 Monitor = 3	View
sint32	SegmentID	Segment to which the user permission group belongs	View
sint32	PTZPriority	PTZ Priority for the users belonging to this group	View
boolean	CanLoginToDataConduit	Shows if the users in this group can login to DataConduit	View
boolean	CanViewLiveVideo	Shows if the users in this group can view live video	View
boolean	CanViewRecordedVideo	Shows if the users in this group can view recorded video	View
boolean	CanSearchVideo	Shows if the users in this group can search video	View
boolean	DevicesExcluded	Shows if the devices in the associated group are excluded	View

Lnl_UserFieldPermissionGroup

Description: A user field permission group defined in the system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
sint32	ID	Internal database ID. Key field.	View
string	Name	Permission Group name.	View
sint32	SegmentID	Segment to which the user field permission group belongs.	View

Lnl_UserPermissionDeviceGroupLink

Description: Describes a link between a device group and a permission.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
sint32	UserPermissionGroupID	User permission group. See Lnl_UserPermissionGroup.ID. Key field.	View
sint32	DeviceGroupID	Device Group ID. See Lnl_DeviceGroup.ID. Key field.	View

Lnl_UserReportPermissionGroup

Description: A user report permission group defined in the system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
sint32	ID	Internal database ID. Key field.	View
string	Name	Permission Group name.	View

Type	Name	Description	Access
sint32	SegmentID	Segment to which the user report permission group belongs.	View
sint32	DatabaseID	The database identifier in an Enterprise system that identifies the replication setting for the group. The value has a default value of 'Local System Only' which matches the default through the OnGuard software.	View

Lnl_UserSecondarySegment

Description: An association between a user and all assigned segments.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
string	UserID	Internal ID of the user Lnl_User.ID.	Read
sint32	SegmentID	A segment to which the user belongs.	Read

Lnl_VideoLayout

Description: Configuration of the matrix view for displaying video channels.

Abstract: No

Access: View

Superclass: None

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	VideoLayoutID	Video layout ID.	View
string	LayoutName	Name of the video layout.	View
int32	VideoTemplateID	Template ID.	View

Type	Name	Description	Access
string	UserID	User ID.	View
int32	WorkstationID	Workstation ID.	View

Lnl_VideoLayoutSource

Description: Source details for the cells in the video layout.

Abstract: No

Access: View

Superclass: None

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	VideoSourceID	Unique ID of the video source.	View
int32	PanelID	VideoRecorderID	View
int32	CameraID	The ID of the camera connected to the video recorder.	View
int32	LayoutID	The layout ID.	View
int32	LayoutCellID	The specific cell in the layout.	View

Lnl_VideoTemplate

Description: A video template for the matrix view of the player window.

Abstract: No

Access: View

Superclass: None

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	TemplateID	Video template ID.	View
string	TemplateName	Video template name.	View
string	TemplateXml	The structure of the template, described in XML.	View

Lnl_Visit

Description: A visit in the security system.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	CARDHOLDERID	LNL_CARDHOLDER.ID - the host	Read
int32	DELEGATEID	The person who schedules or maintains the event on behalf of the host. Optional property.	Edit
boolean	EMAIL_INCLUDE_DEF_RECIPENTS	Whether the default recipients are notified	Edit
boolean	EMAIL_INCLUDE_HOST	Whether the host is notified	Edit
boolean	EMAIL_INCLUDE_VISITOR	Whether the visitor is notified	Edit
string	EMAIL_LIST	A list of semi-colon separated e-mail recipients (other than the visitor, host or defaults) Ex: abc@123.com;xyz@123.com	Edit
datetime (string)	LASTCHANGED	Visit last changed	View
string	NAME	The user-friendly name of this object. Optional property.	Edit
string	PURPOSE	Visit purpose.	Edit
datetime (string)	SCHEDULED_TIMEIN	Scheduled start time	Edit
datetime (string)	SCHEDULED_TIMEOUT	Scheduled end time	Edit
int32	SIGNINLOCATIONID	The ID of the visitor sign-in location. Optional property.	Edit
datetime (string)	TIMEIN	Actual start time	View
datetime (string)	TIMEOUT	Actual end time	View
int32	TYPE	Visit type, values are user-defined	Edit

Type	Name	Description	Access
int32	VISIT_EVENTID	The ID of the visit event. Reference to Lnl_VisitEvent.ID. If this property is empty when calling post Lnl_Visit , a new visit event is created. If a valid Visit_EventID is passed, an additional visitor is added to the event.	Edit
string	VISIT_KEY	A unique identifier assigned to a scheduled visit, used to sign visitors in or out.	View
int32	VISITORID	Lnl_Visitor.ID - the visitor.	Read

Methods:

`void SignVisitOut();`

Signs a visit out, modifying the visit and setting TIMEOUT to current date/time. Any associated badge with the visitor is deactivated and set to the status as configured in the OnGuard software.

`void SignVisitIn([in]int32 BadgeTypeID, [in]string PrinterName, [in]int64 AssignedBadgeID, [in]string AssignedBadgeID_str);`

Signs a visit in, modifying the visit and setting TIMEIN to current date/time. If AssignedBadgeID is set to a valid ID, the badge is automatically assigned to the visitor and made active.

Parameters:

- badgeTypeID - This is the badge type you want to assign the visitor.
- AssignedBadgeID - This is the 64-bit badge ID you want to assign the visitor, a badge already in the system.
- AssignedBadgeID_str - A string representation of the AssignedBadgeID. You cannot provide both AssignedBadgeID and AssignedBadgeID_str in the same call.
- printerName - The name of the printer you want to use to print out the disposable badge

Note: If badgeTypeID is provided so must the printerName (unless there is a default printer set up for the badgeTypeID specified) and AssignedBadgeID will be ignored. If AssignedBadgeID is specified, badgeTypeID and printerName are ignored. See the Visitor Management User Guide for more detailed documentation on visits and signing them in.

Lnl_VisitEmailRecipient

Description: A visit e-mail recipient in the security system.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	RECIPIENTNUMBER	Internal database ID. Key field.	Read
int32	VISITID	LnI_Visit.ID - ID of the visit. Key field.	Read
string	ACCOUNTID	ID of the entry in the external directory. For example, with Microsoft directories, this property would contain the account's security identifier (SID).	Read
string	DIRECTORYID	Internal ID of the directory to which this account belongs.	Read
string	EMAILADDRESS	Recipient e-mail address.	Read
boolean	INCLUDEDEFAULTRECIPIENTS	Whether the default recipients are notified	Read
boolean	INCLUDEHOST	Whether the visit host is notified	Read
boolean	INCLUDEVISITOR	Whether the visitor is notified	Read
int32	PERSONID	LnI_Person.ID - ID of the person receiving the e-mail	Read
int32	SEGMENTID	Segment to which the visit email recipient belongs.	Read

LnI_VisitEvent

Description: A hosted event with visits and visitors.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	The internal database ID	View
string	Name	The user-friendly name of this object.	Edit
int32	CardholderID	The host of the visit event. Reference to LnI_Cardholder.ID.	Edit

Type	Name	Description	Access
int32	DelegateID	The person who schedules or maintains the event instead of the host.	Edit
int32	DatabaseID	The database identifier in an Enterprise system that identifies the system containing the event data.	Edit
datetime (string)	Scheduled_TimeIn	The time the visit event is scheduled to start.	Edit
datetime (string)	Scheduled_TimeOut	The time the visit event is scheduled to complete.	Edit
datetime (string)	LastChanged	The last time the properties of the visit event changed.	View
int32	SignInLocationID	The ID of the visitor sign in location.	Edit

Method:

HRESULT SendEmail([in] int32 ID, [in] Boolean UseSystemDefaults, [in] string Action, [in] Boolean IncludeHost, [in] Boolean IncludeVisitor, [in] Boolean IncludeDefRecipients, [in] string EmailList);

Sends an email to the host, co-hosts, default recipients (if configured), delegate (if visit event is created by the delegate), and individual mails to visitors when a visit event is scheduled with multiple visitors.

Parameters:

- ID - Visit_EventID passed as 'property_value_map'.
- UseSystemDefaults - If true, then emails will be sent as configured in System Administration settings. All other parameters passed to this method are ignored. If false, then emails will be sent as configured by the parameters.
- Action - Add/Modify. 'Add' when visit event is added and 'Modify' when visit event is updated.
- IncludeHost - Whether the host is notified.
- IncludeVisitor - Whether the visitor is notified.
- IncludeDefRecipients - Whether the default recipients are notified.
- EmailList - A list of semi-colon separated e-mail recipients (other than the visitor, host, or defaults).

Lnl_Visitor

Description: A visitor in the security system.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Person

Platforms: OnGuard

Properties: The class has all the properties of the Lnl_Person class, plus custom fields defined by the end user and the following:

Type	Name	Description	Access
string	ADDRESS	The visitor's address.	Edit
string	CITY	The visitor's city.	Edit
string	EMAIL	The visitor's email address.	Edit
boolean	EMAIL_DOCUMENTS	When true, the email sent to a visitor when the visitor is signed in will contain as an attachment the Visitor PDF Document (Lnl_MultimediaObject. Datatype = 13, Lnl_MultimediaObject. Objecttype = 513) associated with the visitor's Lnl_MultimediaObject PERSONID = X. Note: This property is not supported by Visitor Management Front Desk, Visitor Management Host, or Visitor Management.	Edit
string	EXT	The visitor's extension.	Edit
string	OPHONE	The visitor's office phone number.	Edit
string	ORGANIZATION	The visitor's organization.	Edit
int32	PRIMARYSEGMENTID	This property is only available when visitors are segmented.	Read
string	STATE	The visitor's state.	Edit
string	TITLE	The visitor's title.	Edit
string	ZIP	The visitor's zip code.	Edit

Lnl_VisitDelegateAssignment

Description: A visit delegate assignment in the system.

Abstract: No

Access: View/Add/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	HostID	The host. Reference Lnl_Cardholder.ID.	Read
int32	DelegatelD	The delegate. Reference Lnl_Cardholder.ID.	Read

Lnl_VisitSelfServiceStation

Description: The Visitor Self-Service station associated with a sign-in location.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	The internal database ID.	View
string	Name	The Visitor Self Service station name.	Edit
string	UniqueIdentifier	A unique identifier representing the Visitor Self Service device.	Edit
int32	VisitSignInLocationID	A reference to the Lnl_VisitSignInLocation instance corresponding to this station.	Edit

Lnl_VisitSignInLocation

Description: The sign-in location for visits.

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	The internal database ID.	View
string	Name	The visit sign-in location name.	Edit
int32	SegmentID	The ID of the segment to which the sign-in location belongs. This property is only available if segmentation is enabled.	Read
int32	WorldTimezoneID	The time zone of the sign-in location. Reference to Lnl_WorldTimeZone.ID.	Edit

Lnl_Workstation

Description: The workstation used to configure the Monitor Zones used on monitoring stations.

Abstract: No

Access: View

Superclass: None

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	WorkstationID	The ID of the workstation.	View
string	WorkstationName	The name of the workstation.	View
int32	DatabaseID	The database identifier in an Enterprise system that identifies the system containing the workstation data. For more information, refer to Settings on page 130.	View

Lnl_WorldTimezone

Description: A world time zone defined in the security system.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
int32	Bias	The current bias for local time translation on this computer, in minutes.	View
int32	DaylightBias	A bias value that is used during local time translations that occur during daylight time.	View
int32	DaylightDay	DaylightDayOfWeek of the DaylightMonth when the transition from standard time to daylight saving time occurs on this operating system. Example: If the transition day (DaylightDayOfWeek) occurs on a Sunday, then the value "1" indicates the first Sunday of the DaylightMonth, "2" indicates the second Sunday, and so on. The value "5" indicates the last DaylightDayOfWeek in the month.	View
int32	DaylightHour	Hour of the day when the transition from standard time to daylight saving time occurs on an operating system.	View
int32	DaylightMinute	Minute of the DaylightHour when the transition from standard time to daylight saving time occurs on an operating system.	View
int32	DaylightMonth	Minute of the DaylightHour when the transition from standard time to daylight saving time occurs on an operating system. For example, "1" is January, "2" is February, and so on.	View
int32	DaylightSecond	Second of the DaylightMinute when the transition from standard time to daylight saving time occurs on an operating system.	View
int32	DaylightWeek	Week of the DaylightMonth when the transition from standard time to daylight saving time occurs on an operating system.	View
string	DisplayName	The user-friendly name, and how the timezone appears.	View

Type	Name	Description	Access
int32	GMTOffset	In areas of the United States that observe daylight saving time, local residents move their clocks ahead one hour when daylight saving time begins. As a result, their GMT offset would change from GMT - 5h to GMT - 4h. In places not observing daylight saving time, the local GMT offset remains the same all year. Arizona, Puerto Rico, Hawaii, U.S. Virgin Islands, and American Samoa do not observe daylight saving time.	View
boolean	IsDaylightSaving	True if in an area of the United States that observes daylight saving time.	View
int32	StandardBias	Bias value to use when daylight saving time is not in effect. This property is ignored if a value for StandardDay is not supplied. The value of this property is added to the Bias property to form the bias during standard time.	View
int32	StandardDay	StandardDayOfWeek of the StandardMonth when the transition from daylight saving time to standard time occurs on an operating system. If the transition day (StandardDayOfWeek) occurs on a Sunday, then the value "1" indicates the first Sunday of the StandardMonth, "2" indicates the second Sunday, and so on. The value "5" indicates the last StandardDayOfWeek in the month.	View
int32	StandardHour	Hour of the day when the transition from daylight saving time to standard time occurs on an operating system.	View
int32	StandardMinute	Minute of the StandardDay when the transition from daylight saving time to standard time occurs on an operating system.	View

Type	Name	Description	Access
int32	StandardMonth	Month when the transition from daylight saving time to standard time occurs on an operating system. For example, "1" is January, "2" is February, and so on.	View
int32	StandardSecond	Second of the StandardMinute when the transition from daylight saving time to standard time occurs on an operating system.	View
int32	StandardWeek	Week of the StandardMonth when the transition from daylight saving time to standard time occurs on an operating system.	View
string	Windows_TZID	The unique name that Windows uses to identify the timezone in the registry.	View

User-Defined Value Lists

Description: Any user-defined list in the system, populated via List Builder. Some examples include:

- Lnl_BUILDING
- Lnl_DEPT
- Lnl_DIVISION
- Lnl_LOCATION
- Lnl_TITLE
- Lnl_VISITTYPE

Abstract: No

Access: View/Add/Modify/Delete

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description	Access
int32	ID	Internal database ID. Key field.	View
string	NAME	Name of the list value.	Edit
int32	SEGMENTID	Segment to which the user-defined value list belongs.	Read

Association Classes

When using a filter to get instances of an association class, configure the filter as shown in this example:

```
type_name=LnI_AccessLevelGroupAssignment and
filter=AccessGroup="LnI_AccessGroup.ID=1"
```

This filter provides all access levels that belong to the access group with ID = 1.

LnI_AccessLevelGroupAssignment

Description: An association between an access level and the group in which it belongs.

Abstract: No

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:LnI_AccessLevel	ACCESSLEVEL	Reference to the access level
ref:LnI_AccessGroup	ACCESSGROUP	Reference to the access group

LnI_BadgeOwner

Description: An association between a badge and the person who owns it.

Abstract: Yes

Access: View

Superclass: LnI_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:LnI_Badge	BADGE	Reference to the badge
ref:LnI_Person	PERSON	Reference to the person

LnI_CardholderAccount

Description: An association between an account and the cardholder with which it is associated.

Abstract: No

Access: View

Superclass: LnI_PersonAccount

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Account	ACCOUNT	Reference to the account
ref:Lnl_Cardholder	PERSON	Reference to the cardholder

Lnl_CardholderBadge

Description: An association between a badge and the cardholder who owns it.

Abstract: No

Access: View

Superclass: Lnl_BadgeOwner

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Badge	BADGE	Reference to the badge
ref:Lnl_Cardholder	PERSON	Reference to the cardholder

Lnl_CardholderMultimediaObject

Description: An association between a multimedia object and the cardholder who owns it.

Abstract: No

Access: View

Superclass: Lnl_MultimediaObjectOwner

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_MultimediaObject	MULTIMEDIAOBJECT	Reference to the multimedia object
ref:Lnl_Cardholder	PERSON	Reference to the cardholder

Lnl_DirectoryAccount

Description: An association between an account and the directory in which it is stored.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Account	ACCOUNT	Reference to the account
ref:Lnl_Directory	DIRECTORY	Reference to the directory

Lnl_MultimediaObjectOwner

Description: An association between a multimedia object and the person who owns it.

Abstract: Yes

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_MultimediaObject	MULTIMEDIAOBJECT	Reference to the multimedia object
ref:Lnl_Person	PERSON	Reference to the person

Lnl_PersonAccount

Description: An association between an account and the person with which it is associated.

Abstract: Yes

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Account	ACCOUNT	Reference to the account
ref:Lnl_Person	PERSON	Reference to the person

Lnl_ReaderEntersArea

Description: An association between a reader and the APB area to which it allows entry.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Area	AREA	Reference to the APB area
ref:Lnl_Reader	READER	Reference to the reader

Lnl_ReaderExitsArea

Description: An association between a reader and the APB area to which it allows departure from.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Area	AREA	Reference to the APB area
ref:Lnl_Reader	READER	Reference to the reader

Lnl_SegmentGroupMember

Description: An association between a segment unit and the segment group of which the unit is a member. Present in segmented systems only.

Abstract: No

Access: View

Superclass: Lnl_Element

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_SegmentGroup	GROUP	Reference to the segment group
ref:Lnl_SegmentUnit	MEMBER	Reference to the segment unit

Lnl_VisitorAccount

Description: An association between an account and the visitor with which it is associated.

Abstract: No

Access: View

Superclass: Lnl_PersonAccount

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Account	ACCOUNT	Reference to the account
ref:Lnl_Visitor	PERSON	Reference to the visitor

Lnl_VisitorBadge

Description: An association between a badge and the visitor who owns it.

Abstract: No

Access: View

Superclass: Lnl_BadgeOwner

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_Badge	BADGE	Reference to the badge
ref:Lnl_Visitor	PERSON	Reference to the visitor

Lnl_VisitorMultimediaObject

Description: An association between a multimedia object and the visitor who owns it.

Abstract: No

Access: View

Superclass: Lnl_MultimediaObjectOwner

Platforms: OnGuard

Properties:

Type	Name	Description
ref:Lnl_MultimediaObject	MULTIMEDIAOBJECT	Reference to the multimedia object
ref:Lnl_Visitor	PERSON	Reference to the visitor

Using OpenAccess to Send Alarms to OnGuard

OpenAccess provides the capability of sending alarms to the Alarm Monitoring application. These alarms are also logged to the OnGuard database just like other alarms.

It is necessary to first setup a Logical Source using System Administration before using this capability of OpenAccess. OpenAccess will use this source as the device to display alarms for in Alarm Monitoring. For more information, refer to [Add a Logical Source](#) on page 282.

Note: Starting with OnGuard 7.6, the preferred method for using OpenAccess to send alarms to OnGuard is with the **post send_incoming_events** call. For more information, refer to [post send_incoming_events](#) on page 109.

Note: In order to receive logical source events, add at least one online panel to the same monitor zone as the source.

After configuring the Logical Source, you should also add any Logical Device and Logical Sub-Device downstream devices in System Administration. Use of devices and sub-devices is optional. OnGuard uses devices and sub-devices to report alarms for Logical Source child and sub-child devices in Alarm Monitoring. For more information, refer to [Add a Logical Device](#) on page 284 and [Add a Logical Sub-Device](#) on page 286.

Sending alarms to Alarm Monitoring is very simple.

Note: To use the following example, change “localhost” to the Fully Qualified Domain Name (FQDN) of your server.

Here is an example using an HTTP request:

```
1 POST localhost/api/access/onguard/openaccess/execute_method
2 Header:
3     Session-Token : 12345-67890-12345-67890
4     Application-Id : SUPPLIED_APPLICATION_ID
5 Body:
6 {
7     "type_name" : "LnI_IncomingEvent",
8     "property_value_map" :
9     {
10     },
```

```
11     "method_name" : "SendIncomingEvent",
12     "in_paramter_value_map" :
13     {
14         "Description" : "Test event from OpenAccess",
15         "Source" : "Logical Source 6"
16     }
17 }
```

The above sample will display and log an alarm with the description “Test Event From OpenAccess” from controller name “Logical Source 6”. This sample assumes System Administration was used to create a Logical Source called “Logical Source 6” and demonstrates how to send an alarm to Alarm Monitoring. The Source refers to the logical source setup in System Administration. The Description property is the actual text of the alarm that will display in Alarm Monitoring and be logged into the OnGuard database.

The `Lnl_IncomingEvent` object has no properties and currently supports the methods “SendIncomingEvent” and “AcknowledgeAlarm”. For more information, refer to [Lnl_IncomingEvent](#) on page 215.

The OpenAccess SendIncomingEvent method allows the ability to generate Access Granted and Access Denied events for a Logical Source, Device and Sub-Device. This is made possible via the following additional optional parameters that may be specified to the SendIncomingEvent method: IsAccessGrant, IsAccessDeny, BadgeID, and ExtendedID.

If ‘IsAccessGrant’ is set to true, the ‘Granted Access’ event will be reported for the Logical Source, Device or Sub-Device specified in the script. Similarly, if ‘IsAccessDeny’ is set to true, the ‘Access Denied’ event will be reported. If both of these are set to true, the method will fail since only of these can be set to true at a given time (i.e., they are mutually exclusive). For more information, refer to [Generating Access Granted and Access Denied Events](#) on page 217.

The process is similar if the name of the Source and Device parameters correspond to the name of an access panel and reader respectively. OnGuard checks to see if the Logical Source name provided matches a Logical Source. If not, then a check is made to see if it matches the name of a Lenel access panel. If so, OnGuard checks the Device parameter and see if it matches the name of a reader assigned to the access panel. If these conditions are met, the ‘Granted Access’ or ‘Access Denied’ events are reported based on how ‘IsAccessGrant’ and ‘IsAccessDeny’ are set.

The BadgeID or ExtendedID parameter can be specified when either ‘IsAccessGrant’ or ‘IsAccessDeny’ are set to true to report an event for a specific OnGuard cardholder. BadgeID is not required when using ‘IsAccessGrant’ or ‘IsAccessDeny’.

OpenAccess is an advanced application integration service that allows real time, bidirectional integration between OnGuard and third party IT sources. OpenAccess allows System Administrators to develop scripts and/or applications that allow events in one domain (security or IT) to cause appropriate actions in the other.

Logical Sources Folder

Note: In order to receive logical source events, add at least one online panel to the same monitor zone as the source.

The Logical Sources folder is found in System Administration and allows System Administrators to add, modify and delete third-party Logical Sources, Devices, and Sub-Devices. After third-party sources are added, users can send the incoming events to OnGuard via OpenAccess, and view third-party events in Alarm Monitoring.

To send an event to OnGuard via OpenAccess, System Administrators must:

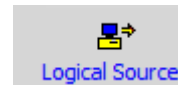
- Define the incoming source in the Logical Sources folder
- Use the `Lnl_IncomingEvent::SendIncomingEvent` method

Note: The Logical Sources method has four parameters: the source, description, device (optional), and sub-device (optional). The source of the Logical Sources method must match the source name on the Logical Sources form. If the optional parameters are used, the device of the Logical Sources method must match the device name on the Logical Devices form, and the sub-device must match the sub-device name on the Logical Sub-Devices form.

- Have at least one panel (non-system Logical Source) configured and marked online so that the Communications Server will work properly with Logical Sources. The panel does not need to exist or actually be online in Alarm Monitoring; it simply needs to exist and show up in the System Status view. Once this is configured, events can be received successfully by Alarm Monitoring from Logical Sources.

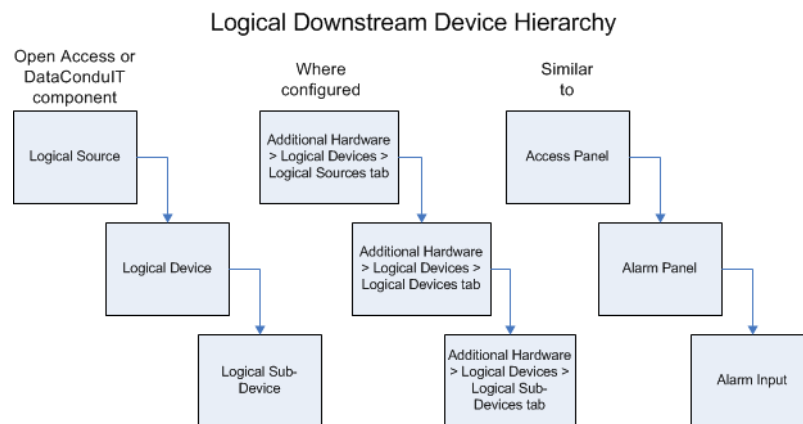
This folder is displayed by selecting **Logical Sources** from the **Additional Hardware** menu, or by selecting the Logical Sources toolbar button in System Administration or ID CredentialCenter.

Toolbar Shortcut



Logical Source Downstream Devices

A Logical Source may have Logical Device or Logical Sub-Device downstream devices. A Logical Device is a child of a Logical Source, similar to how an alarm panel is a child of an access panel. A Logical Sub-Device is a sub-child device of a Logical Device, similar to how an alarm input is a sub-child of an alarm panel. The following diagram illustrates this hierarchy.



Logical Devices and Logical Sub-Devices also display in Alarm Monitoring in the System Status Tree. For example, a Logical Source named “Tivoli” with a Logical Device named “Tivoli device” and a Logical Sub-Device named “Tivoli sub-device” would display in Alarm Monitoring in the following manner:



User Permissions Required

Add, Modify, and Delete Logical Sources, Devices, and Sub-Devices

The add, modify, and/or delete Logical Sources permissions, determine what functions a user can perform on Logical Sources, Logical Devices, and Logical Sub-Devices in the Logical Sources folder. These permissions are located in **Administration > Users > System Permission Groups tab > Additional Data Sources sub-tab** in System Administration or ID CredentialCenter.

Trace Logical Sources, Devices, and Sub-Devices

In addition, user permissions are required to trace Logical Sources, Logical Devices, and Logical Sub-devices in Alarm Monitoring. These permissions are located in **Administration > Users > Monitor Permission Groups** tab > **Monitor** sub-tab in System Administration or ID CredentialCenter.

Logical Sources Form

The screenshot shows the 'Logical Sources' window with three tabs: 'Logical Sources', 'Logical Devices', and 'Logical Sub-Devices'. The 'Logical Sources' tab is active, displaying a table with columns 'Name' and 'Segment'. The table lists three items: 'HP OpenView' (EnterpriseMas), 'IBM Websphere Message Adapter' (EnterpriseMas), and 'Tivoli' (EnterpriseMas). To the right of the table, there are fields for 'Name' (set to 'IBM Websphere Message Adapter'), 'Online' (checked), 'World time zone' (set to '(GMT-05:00) Eastern Time (US & Canada)'), and 'Daylight savings' (checked). At the bottom, there are buttons for 'Add', 'Modify', 'Delete', 'Help...', a 'Multiple Selection' checkbox, a status indicator '1 of 3 selected', and a 'Close' button.

Listing window

Lists Logical Source names.

Name

Identifies the name of the Logical Source. This is a “friendly” name assigned to each Logical Source to make it easy to identify.

Online

The Logical Source is always online and ready for use. This status does not apply to the Logical Source.

World time zone

Select the world time zone for the selected access panel’s geographical location. The selections in the drop-down list are listed sequentially, and each includes:

- The world time zone’s clock time relative to Greenwich Mean Time. For example, (GMT+05:00) indicates that the clock time in the selected world time zone is 5 hours ahead of the clock time in Greenwich, England.
- The name of one or more countries or cities that are located in that world time zone.

Daylight savings

Select this check box if Daylight Savings Time is enforced in the selected access panel’s geographical location.

Add

Click this button to add a Logical Source.

Modify

Click this button to modify a Logical Source.

Delete

Click this button to delete a Logical Source.

Help

Click this button to display online help for this form.

Multiple Selection

If selected, more than one entry in the listing window can be selected simultaneously. The changes made on this form will apply to all selected Logical Sources.

Close

Click this button to close the Logical Sources folder.

Logical Sources Form Procedures

Use the following procedures on this form.

Add a Logical Source

1. From the ***Additional Hardware*** menu, select ***Logical Sources***. The Logical Sources folder opens.
2. On the Logical Sources tab, click [Add].
3. If segmentation is not enabled, skip this step. If segmentation is enabled:
 - a. The Segment Membership window opens. Select the segment to which this Logical Source will be assigned.
 - b. Click [OK].
4. In the **Name** field, type a name for the Logical Source.
5. Select whether the Logical Source will be online.
6. Select the world time zone and daylight savings options as you see fit.
7. Click [OK].

IMPORTANT: In addition to having a Logical Source configured, there must be at least one panel (non-system Logical Source) configured and marked online so that the Communications Server will work properly with Logical Sources. The panel does not need to exist or actually be online in Alarm Monitoring; it simply needs to exist and show up in the System Status view. Once this is set up, events can be received successfully by Alarm Monitoring and event subscribers from Logical Sources.

Modify a Logical Source

1. From the ***Additional Hardware*** menu, select ***Logical Sources***.
2. On the Logical Sources tab, select the entry you want to modify from the listing window.
3. Click [Modify].
4. Make any changes.

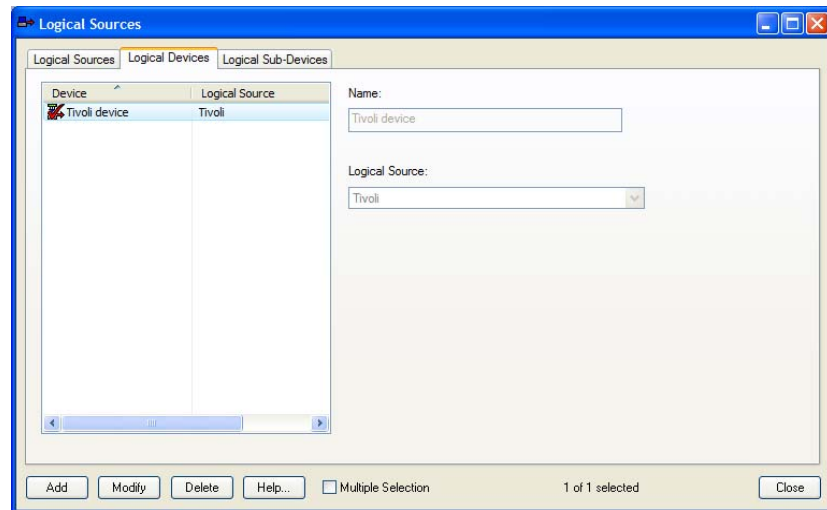
5. Click [OK].
6. A prompt to confirm that you want to make the modification displays. Click [OK].

Delete a Logical Source

To suspend a Logical Source without deleting it, take it offline.

1. From the **Additional Hardware** menu, select **Logical Sources**.
2. On the Logical Sources tab, select the entry you want to delete from the listing window.
3. Click [Delete].
4. Click [OK].
5. A prompt to confirm that you want to make the deletion will be displayed. Click [OK].

Logical Devices Form



Listing window

Lists Logical Device names.

Name

Identifies the name of the Logical Device. This is a “friendly” name assigned to each Logical Device to make it easy to identify.

Logical Source

Select the Logical Source that is the parent of the child device being configured. Logical Sources are configured on the Logical Sources tab (Additional Hardware > Logical Sources > Logical Sources tab).

Add

Click this button to add a Logical Device.

Modify

Click this button to modify a Logical Device.

Delete

Click this button to delete a Logical Device.

Help

Click this button to display online help for this form.

Multiple Selection

If selected, more than one entry in the listing window can be selected simultaneously. The changes made on this form will apply to all selected Logical Devices.

Close

Click this button to close the Logical Sources folder.

Logical Devices Form Procedures

Use the following procedures on this form.

Add a Logical Device

Prerequisite: Before a Logical Device can be configured, its parent Logical Source must first be configured.

Note: If segmentation is enabled, the segment of the Logical Source will be used as the segment for the Logical Device.

1. From the **Additional Hardware** menu, select **Logical Sources**. The Logical Sources folder opens.
2. Click the Logical Devices tab.
3. Click [Add].
4. In the **Name** field, type a name for the Logical Device.
5. Select the Logical Source that is the parent of the Logical Device.

Note: The Logical Source must be configured on the Logical Sources tab.

6. Click [OK].

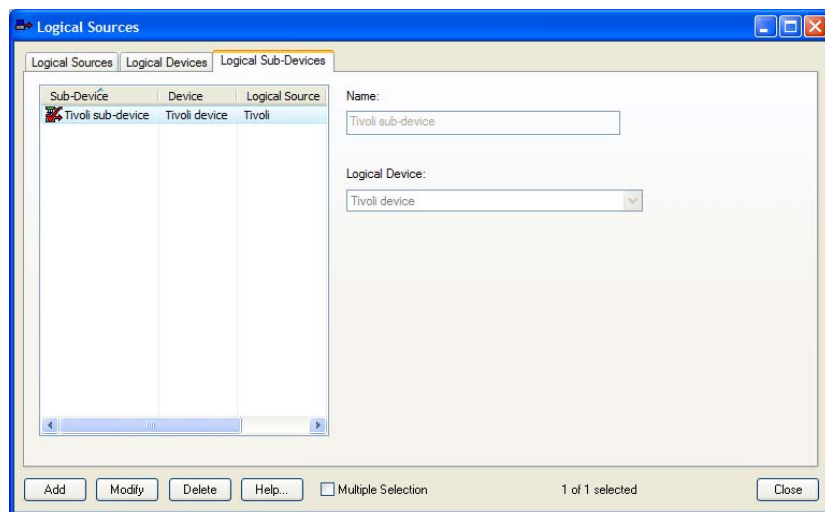
Modify a Logical Device

1. From the **Additional Hardware** menu, select **Logical Sources**.
2. Click the Logical Devices tab.
3. Select the entry you want to modify from the listing window.
4. Click [Modify].
5. Make any changes.
6. Click [OK].
7. A prompt to confirm that you want to make the modification displays. Click [OK].

Delete a Logical Device

1. From the **Additional Hardware** menu, select **Logical Sources**.
2. Click the Logical Devices tab.
3. Select the entry you want to delete from the listing window.
4. Click [Delete].
5. Click [OK].
6. A prompt to confirm that you want to make the deletion will be displayed. Click [OK].

Logical Sub-Devices Form



Listing window

Lists Logical Sub-Device names, along with the parent Logical Device and Logical Source.

Name

Identifies the name of the Logical Sub-Device. This is a “friendly” name assigned to each Logical Sub-Device to make it easy to identify.

Logical Device

Select the Logical Device that is the parent of the child Sub-Device being configured. Logical Devices are configured on the Logical Devices tab (**Additional Hardware > Logical Sources > Logical Devices** tab).

Add

Click this button to add a Logical Sub-Device.

Modify

Click this button to modify a Logical Sub-Device.

Delete

Click this button to delete a Logical Sub-Device.

Help

Click this button to display online help for this form.

Multiple Selection

If selected, more than one entry in the listing window can be selected simultaneously. The changes made on this form will apply to all selected Logical Sub-Devices.

Close

Click this button to close the Logical Sources folder.

Logical Sub-Devices Form Procedures

Use the following procedures on this form.

Add a Logical Sub-Device

Prerequisite: Before a Logical Sub-Device can be configured, its parent Logical Source and Logical Device must be configured.

Note: If segmentation is enabled, the segment of the Logical Source will be used as the segment for the Logical Sub-Device.

1. From the **Additional Hardware** menu, select **Logical Sources**. The Logical Sources folder opens.
2. Click the Logical Sub-Devices tab.
3. Click [Add].
4. In the **Name** field, type a name for the Logical Sub-Device.
5. Select the Logical Device that is the parent of the Logical Sub-Device.

Note: The Logical Device must be configured on the Logical Devices tab.

6. Click [OK].

Modify a Logical Sub-Device

1. From the **Additional Hardware** menu, select **Logical Sources**.
2. Click the Logical Sub-Devices tab.
3. Select the entry you want to modify from the listing window.
4. Click [Modify].
5. Make any changes.
6. Click [OK].
7. A prompt to confirm that you want to make the modification displays. Click [OK].

Delete a Logical Sub-Device

1. From the **Additional Hardware** menu, select **Logical Sources**.
2. Click the Logical Sub-Devices tab.
3. Select the entry you want to delete from the listing window.
4. Click [Delete].

5. Click [OK].
6. A prompt to confirm that you want to make the deletion will be displayed. Click [OK].

This section describes how to use the OpenAccess Tool and other techniques to troubleshoot issues with the LS OpenAccess service.

It is also useful to understand the OpenAccess architecture. For more information, refer to [OpenAccess Architecture](#) on page 26.

Enabling Verbose Logging

For more information, refer to [Enabling Verbose Logging](#) on page 33.

Testing if the LS OpenAccess Service is Online

For a quick test to see if the LS OpenAccess service is configured and online, create a client that supports the **get version** request/response. A **get version** response confirms that the service is online. For more information, refer to [get version](#) on page 56.

Error Messages

This section defines how the LS OpenAccess service communicates errors to the client. If an error occurs, the response will include an entry named **error** which is a key/value map. The response may otherwise contain only standard response headers.

The error is a string in a period-delimited hierarchical string that follows the platform namespace. For example:

```
"error":
{
  "code": "openaccess.general.invalidapplicationid",
  "message": "You are not licensed for OpenAccess."
}
```

Name	Type	Required	Description
code	string	yes	The error code, which is a string with a full namespace.
message	string	no	An optional human-readable message to display after the translated error code. The message is sent in the client locale, if possible.
...	...	no	Other optional fields, as defined along with the error code.

For more information about error codes, refer to [Errors List](#) on page 290.

Errors List

Notes: This section does not contain every OpenAccess error code that might be logged. Only the most common error codes are listed.

The error code sent to the client generally contains less detail than is logged at the server. Check the server logs for more information.

If the LS OpenAccess service cannot connect to the database, that can cause many of the OpenAccess errors. Confirm that the service has a database connection.

Error Code	Root Cause and Resolution	HTTP Error Code
openaccess.general.missingrequestitem	When a required request item is not present in the request, the name of the missing item is part of the message.	400
openaccess.general.exception	General exception. Refer to server logs for details.	500
openaccess.general.invalidrequestitem	The operation failed because of an invalid request item input. Details provided in the error message.	400
system.invalid_field	The operation failed because of an invalid request item input. Details provided in the error message.	400
openaccess.general.decodingfailed	Failed to generate binary data from base-64 string.	400
openaccess.general.invalidapplicationid	You are not licensed to use OpenAccess with the provided application ID. The application ID is not valid.	401

openaccess.general.invaliddb-connection	The database connection is not functioning. The request cannot be fulfilled. Try again later.	503
openaccess.general.invalid-sessiontoken	The provided session token is not recognized as a previously-authenticated token to the service.	401
openaccess.general.invalid-typename	Failed to retrieve type details. Type name specified is not valid. Refer to server logs for details.	400
openaccess.general.invalid-userpassword	The operation failed because the new password you created does not meet the password policies. Details are provided in the error message.	400
openaccess.authentication.failedtoauthenticate	Authentication failed. Could be caused by invalid credentials. Refer to server logs for details.	401
openaccess.authentication.invalidinternallogin	Authentication of an internal user account failed because of invalid credentials.	400
openaccess.authentication.invalidthirdpartyauthlicense	The OpenID Connect feature is not licensed. Acquire a valid license to use this feature.	400
openaccess.authentication.passwordexpired	The user password is expired.	400
openaccess.getinstances.maxpagesizeexceeded	The maximum page size is 100.	400
openaccess.editinstance.error	The add/modify/delete operation failed. Details will be provided in the error message.	500
openaccess.execute-method.error	Execution of the method failed. Details provided in the error message.	500
system.insufficient_privilege	The user is not the owner of the event subscription.	400
system.missing_field	When a required request item is not present in the request, the name of the missing item is part of the message.	400
system.parse	The filter specified is invalid.	400
system.http_error_code	A timeout occurred because the request took longer than allowed, as configured with the request_timeout property in the openaccess.ini file (for more information, refer to Timeout Property on page 25). Also, the request might be malformed or contain invalid parameters.	40_ (400, 404, 408, and so on)

system.insufficient_privilege	The user logged into OpenAccess does not have the permissions required to perform the requested operation.	403
system.not_implemented	When an unsupported operation is attempted (for example, you try to delete an instance of a type that does not support delete).	501

Warning List

Note: This section does not contain every OpenAccess warning. Only the most common warnings are listed.

Warning Code	Root Cause and Resolution
openaccess.warning.passwordexpiration	Users receive this warning during authentication if their passwords are almost expired. The following policy settings are used when the authentication response contains this warning: • is_expiration_reminders_enabled • expiration_first_reminder_days • expiration_reminder_days For more information, refer to get password policy settings on page 134.

Starting the OpenAccess Tool

The OpenAccess Tool is a sample client used for troubleshooting purposes. To start the tool, navigate to **Program Files (x86)\OnGuard**, and then double-click **OpenAccessTool.exe**.

Notes: To run the OpenAccess Tool, you will be prompted to enter a valid Application ID. Contact LenelS2 OnGuard Technical Support if you do not have an Application ID.

The Event Generator is another useful troubleshooting tool. Use Event Generator to create “fake” events that can be received by event subscribers. For more information, refer to [Appendix A: Event Generator](#) on page 299.

Using the OpenAccess Tool

Creating Instances

To create an instance:

1. In the OpenAccess tool, select the Instances tab.
2. In the Types drop-down, select the type you want to create.
3. Click [Create]. The listing window populates with the properties assigned to the type.

4. Double-click each property in the listing window you want to define and enter the value.
5. When you are finished defining property values, click [Submit].
6. Click [Instances] to view the existing instances.

Modifying Instances

To modify an instance:

1. In the OpenAccess tool, select the Instances tab.
2. In the Types drop-down, select the type you want to modify.
3. Click [Instances] to view the existing instances.
4. Select an instance you want to modify. The listing window populates with the properties assigned to that instance.
5. Click [Modify].
6. Double-click each property in the listing window you want to modify and enter the new value.
7. When you are finished modifying property values, click [Submit].
8. Click [Instances] to view the existing instances.

Deleting Instances

To delete an instance:

1. In the OpenAccess tool, select the Instances tab.
2. In the Types drop-down, select the type you want to delete.
3. Click [Instances] to view the existing instances of that type. The listing window populates with the existing instances.
4. Select the instance you want to delete.
5. Click [Delete].
6. A dialog opens asking if you want to delete the instance. Click [OK].
7. A dialog indicates that the instance was deleted successfully. Click [OK].

Authentication Expiration Warning for OpenAccess Tool

The OpenAccess Tool warns the user with a dialog when the Authentication token has expired, and forces the user to log back into the tool to receive a new token. By default, the Authentication token expires 8 hours after you logged in. For more information, refer to [Authentication](#) on page 32.

Symptoms and Solutions

Errors Connecting to the Message Broker

There are errors connecting to the Message Broker when it is running on a server not connected to any domain (only local workgroup).

For information about certificates and how to correct these errors, refer to the “OnGuard and the Use of Certificates” appendix in the *OnGuard Installation Guide*.

SSL/TLS Secure Channel Errors

The OpenAccess Tool generates errors similar to “The underlying connection was closed: Could not establish trust relationship for the SSL/TLS secure channel.”

All applications using the LS OpenAccess service must reference the OpenAccess API in a way that exactly matches the certificate name. If the certificate name uses the server’s Fully Qualified Domain Name (FQDN), then applications must access OpenAccess using the server’s FQDN. Likewise, if the certificate name does not use the server’s FQDN, then applications must access OpenAccess by not using the server’s FQDN.

Note: The OpenAccess Tool uses the OpenAccess location configured on the **System Administration > System Options** form.

For information about certificates and how to correct these errors, refer to the “OnGuard and the Use of Certificates” appendix in the *OnGuard Installation Guide*.

CORS Errors When Accessing the OpenAccess API from a Web Application

There are Cross-Origin Resource Sharing (CORS) errors when accessing the OpenAccess API from a web application.

For more information, refer to [Cross-Origin Resource Sharing](#) on page 48.

CORS Errors When Running the Cardholder Sample Web Application

There are CORS errors when running the Cardholder Sample web application.

The Getting Started chapter provides details on how to load the cardholder sample web application properly. See Sample Applications on page 34.

The Using OpenAccess chapter provides details about CORS. See Cross-Origin Resource Sharing on page 48.

Errors After Updating the nginx.conf File

*There are errors accessing the OpenAccess API after updating the **nginx.conf** file.*

Perform the following steps to troubleshoot the NGINX configuration:

1. Verify NGINX is running by checking for two running **nginx.exe** processes. Also point a web browser to <https://<Fully Qualified Domain Name of server>:8080>. If the default NGINX page loads, the web server is running. If the default NGINX page loads on the server but fails to load on the client, there is a problem with the connection between the client and server.
2. Review the NGINX error log (**C:\ProgramData\Ln\nginx\logs\error.log**). For more verbose logging, add the following line near the top of the **C:\ProgramData\Ln\nginx\conf\nginx.conf** file. Refer to http://nginx.org/en/docs/nginx_core_module.html#error_log for details about the NGINX error log directive:

```
error_log logs/error.log info;
```

Event Subscribers Do Not Receive Any Events

Event subscribers are not receiving any events.

Confirm the following:

- The LS Event Context Provider is running.

- There is an online panel in your default monitoring zone. For more information, refer to [Add a Logical Source](#) on page 282.
- Verify the filter you used to subscribe to events. Also verify that the property names are valid. For more information, refer to [Using Event Filters with Subscriptions](#) on page 43.

Note: The Event Generator is a useful troubleshooting tool. Use Event Generator to create “fake” events that can be received by event subscribers. For more information, refer to [Appendix A: Event Generator](#) on page 299.

Event Subscribers Do Not Receive Software Events

Event subscribers are not receiving software events.

Confirm that on the **System Administration > Administration > System Options** form, the **Generate software events** checkbox is checked.

Cannot Log Into OpenAccess Using Manual Single Sign-On

Manual single sign-on does not work with OpenAccess, after specifying the directory, user name, and password.

Confirm the following:

- The user name and password are correct.
- The specified directory is configured correctly in System Administration on the **Administration > Directories** form.
- Also on the Directories form, confirm that the **Enable single sign-on** and **Allow manual single sign-on** checkboxes are selected.

Note: OpenAccess does not work with directories of type **Windows Local Accounts** because local accounts do not support manual single sign-on. To work around this, create a directory of type **Microsoft Windows NT 4 Domain** and enter the machine name in the **Domain** field.

Cannot Get Cardholders From Active Directory with Administrator Account

Use **Domain.exe** located in the **TroubleShooting** directory in the **DataConduIT** documentation file structure to determine if this may be the problem. If the NT4Domain is different from the W2KDomain, update the LNL_DIRECTORY.DIR_HOSTNAME in the Access Control database to match the NT4Domain. In case this is Oracle, use all upper case.

A sample SQL query to do this follows; it assumes the NT4Domain name is “Lenel” from **Domain.exe** and that the directory to be updated is LNL_DIRECTORYID = 1.

```
update ln1_directory set dir_hostname = 'LENEL' where  
ln1_directoryid=1
```

Alternatively, add both the fully qualified Active directory and the NT 4 Domain directory.

Unsuccessful OpenAccess Operations From Behind a Network Proxy

An error occurs when performing OpenAccess operations from behind a network proxy.

If you are using OpenAccess to perform OpenAccess operations from behind a network proxy (for example, issue mobile credentials, or using a third-party provider for OnGuard authentication, or

performing any other operation that requires OpenAccess to reach an external network location) and are behind a network proxy, an error might occur. To resolve this error, on the server where the LS OpenAccess service is running, change the logon account for the LS OpenAccess service from Local System to a user whose account has the correct proxy settings configured.

LS OpenAccess Service Does Not Start in a Cluster Environment

The LS OpenAccess service does not start when installed in a cluster environment.

For information on how to troubleshoot this issue, refer to the *Using Microsoft Cluster Services with OnGuard* guide.

Appendices

The Event Generator is a utility that is used to generate events without having “live” or online hardware connected to a system; it enables customers who wish to generate events without purchasing hardware to do so.

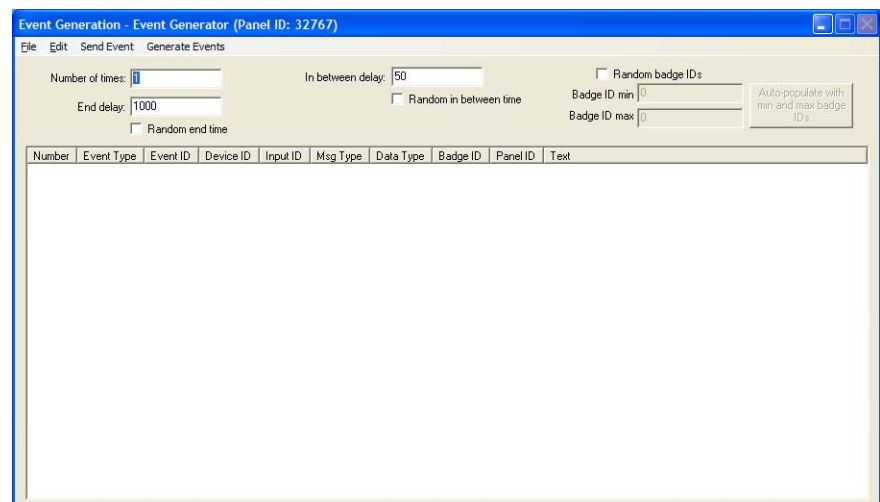
The Event Generator is available on the LenelS2 Web site: <https://partner.lenel.com/downloads/onguard/software>.

Note: When accessing the **Downloads** section at <https://partner.lenel.com>, make sure to select the version of OnGuard that is currently installed.

It is also available on the OnGuard Software Development Kit (SDK) media image.

Event Generator Main Window

The Event Generator Main Window displays automatically when the Communication Server is run as an application after the Event Generator is set up. To correctly set up the Event Generator, refer to [Required Event Generator Files](#) on page 307.



Number of times

Number of times each event in the listing window will be generated

End delay

Amount of time that will elapse after the last event is sent

Random end time

If selected, the **End delay** value specified will be ignored, and instead a random time will be used

In between delay

Amount of time that will elapse between events that are sent

Random in between time

If selected, the **In between delay** value specified will be ignored, and instead a random time will be used

Random badge IDs

If selected, badge ID numbers will be randomly generated. This check box must be selected for **Badge ID min**, **Badge ID max**, and [Auto-populate with min and max badge IDs] to be enabled and available for selection.

Badge ID min

The lowest badge ID that is allowed to be randomly selected. Badge IDs will be randomly determined, but will fall in the range between the specified badge ID min and max.

Badge ID max

The highest badge ID that is allowed to be randomly selected. Badge IDs will be randomly determined, but will fall in the range between the specified badge ID min and max.

Auto-populate with min and max badge IDs

Automatically populates the **Badge ID min** and **Badge ID max** fields with values appropriate for your particular database

Listing window

Lists events that have been added, along with the event type, event ID, device ID, input ID, message type, data type, badge ID, Panel ID, and text associated with each.

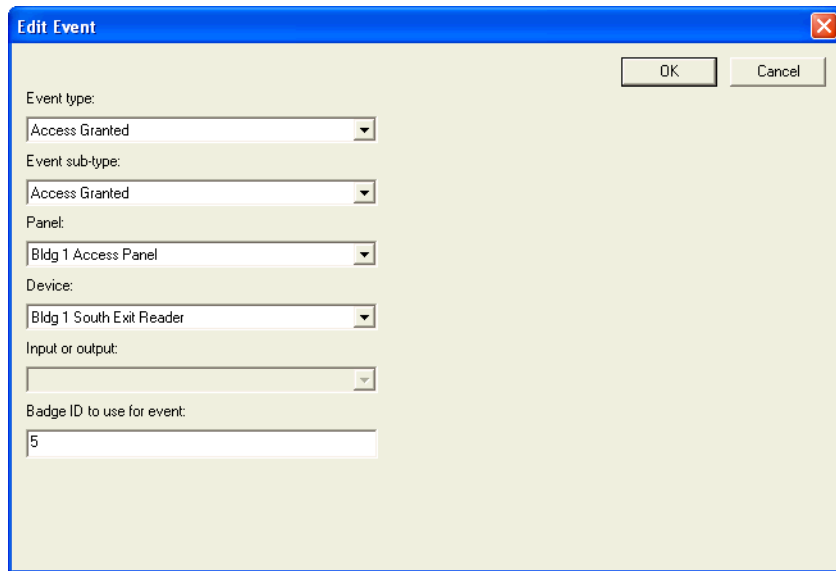
Edit Event (Simple) Window

The Edit Event (Simple) window is used to add new events or modify existing events using the minimum number of required parameters.

Only non-receiver/intrusion events in the OnGuard system are available in the Edit Event (Simple) window. For receiver/intrusion events, use the Edit Event (Advanced) window.

The Edit Event (Simple) window opens when you select either:

- **Edit > Create Event > Create Event (Simple)**, or
- **Edit > Modify Event > Modify Event (Simple)** when an event is selected



The screenshot shows a window titled "Edit Event" with a close button in the top right corner. Inside the window, there are several fields with dropdown menus and one text field. The fields are labeled as follows: "Event type:" with a dropdown menu showing "Access Granted"; "Event sub-type:" with a dropdown menu showing "Access Granted"; "Panel:" with a dropdown menu showing "Bldg 1 Access Panel"; "Device:" with a dropdown menu showing "Bldg 1 South Exit Reader"; "Input or output:" with a dropdown menu that is currently empty; and "Badge ID to use for event:" with a text field containing the number "5". In the top right corner of the window, there are two buttons: "OK" and "Cancel".

Event type

Lists all non-receiver/intrusion events in the OnGuard system. For receiver/intrusion events, use the Advanced user interface.

Event sub-type

Lists sub-categories of the selected event type.

Panel

Lists all available panels for the selected event type. The event will be generated for the selected panel.

Device

Lists all available readers for the selected event type (if applicable). The event will be generated for the selected reader.

Input or output

Lists all available inputs and outputs for the selected event type (if applicable). The event will be generated for the selected input or output.

Badge ID to use for event

The entered badge ID will be used in generating the event (if applicable).

OK

If adding a new event, the event will be added. If modifying an event, the modifications will be saved.

Cancel

Closes the Edit Event (Simple) window without adding or modifying any events.

Edit Event (Advanced) Window

The Edit Event (Advanced) window is used to add new events or modify existing events using advanced parameters.

In the Edit Event (Advanced) window, both non-receiver/intrusion and receiver/intrusion events are available. In the Edit Event (Simple) window, only non-receiver/intrusion events are available.

The Edit Event (Advanced) window opens when you select either:

- **Edit > Create Event > Create Event (Advanced)**, or
- **Edit > Modify Event > Modify Event (Advanced)** when an event is selected

The fields available on this window for the data type change depending on which data type is selected. For example, if the **EVENT_DATA_TYPE_STATUS** data type is selected, the **New status**, **Old status**, and **Comm status** fields are displayed and active.

There are six custom data fields: data1, data2, data3, data4, data5, and data6. If a data type uses custom fields, then the field names are displayed instead of data1, data2, data3, etc.

When a data type contains less than six custom data fields, the extra fields are disabled. For example:

- **New status** = data1
- **Old status** = data2
- **Comm status** = data3
- data4, data5 and data6 are not used and are disabled

Event type

Lists all categories of events in the OnGuard system. This field is used in combination with the **Event category** drop-down to filter what events are listed in the **Events** drop-down.

Event category

Allows the events in the **Events** drop-down listbox to be filtered based on the category. Non-receiver/intrusion events and receiver/intrusion events are available in this drop-down; in the Simple user interface only non-receiver/intrusion events are available.

Events

Lists all events for the selected event type and event category.

Parameterized

Select this check box to generate an event that uses event parameters.

Note: Not all events support parameters. For more information on event parameters, refer to the OpenDevice Events Guide in the OnGuard Software Development Kit (***Program Files (x86)\OnGuard Software Development Kit\OpenDevice***).

Parameter

Enter the parameter value associated with the event to generate. For more information, refer to the OpenDevice Events Guide for events that have the **sb_EventParam** listed.

Message type

Indicates the message type of the event. The available choices are: Event, Status, Video. Most messages will be of the Event type. Status messages are for messages which pass back status information and will not display in Alarm Monitoring. Video events are special events used by video.

Data type

Indicates the type of additional data to be used with the message. For example, some messages can have a badge ID and a specific data type will be used for these so this information can be passed back.

The fields available on this window for the data type change depending on which data type is selected. For example, if the **EVENT_DATA_TYPE_STATUS** data type is selected, the **New status**, **Old status**, and **Comm status** fields are displayed and active.

There are six custom data fields: data1, data2, data3, data4, data5, and data6. If a data type uses custom fields, then the field names are displayed instead of data1, data2, data3, etc.

When a data type contains less than six custom data fields, the extra fields are disabled. For example:

- **New status** = data1
- **Old status** = data2
- **Comm status** = data3
- data4, data5 and data6 are not used and are disabled

If your event does not have additional data, use the **EVENT_DATA_TYPE_STATUS**.

For more information, refer to [Custom Data Fields Displayed for Each Data Type Setting](#) on page 304.

Associated event text

If selected, the text field will become enabled. Indicates if the message is to have associated text with it.

Text

Enter text to be associated with the event

Device ID

This is a downstream device ID that can be used to represent the event is from a downstream device instead of just from a panel. OnGuard uses a three tiered device ID in the format P-D-I; this is the second value.

Input ID

This is a downstream input ID that can be used to represent that the event is from a downstream device instead of just for a panel or its downstream device. OnGuard uses a three tiered device ID in the format P-D-I; this is the third value.

Override Event Generator's panel ID

This checkbox can be used to override the event generator's panel ID so that you can generate an event that is from a different panel.

Panel ID

If the Override Event Generator's panel ID option is being used, you will need to specify the panel ID that will be used for the event in replacement for the event generator's panel ID.

Generate Receiver Account event

Select this check box to generate an event that would be sent from a burglary/intrusion panel to a Central Station receiver connected to the OnGuard software.

This check box is only available when `EVENT_DATA_TYPE_RECEIVER` is selected from **Data type**. When this box is checked, the **Account Number** and **Event Code Template** fields become available.

Account Number

Enter the account number for the receiver. This number is then displayed in Alarm Monitoring under the Controller column.

Event Code Template

Select the event code format that is used to decode the receiver account event data. This is the same field in *System Administration > Additional Hardware > Receivers > Event Code Templates* tab.

Note: When using the **Event Code Template** drop-down list, the **Event type**, **Event category**, and **Events** drop-down lists are not used.

OK

If adding a new event, the event will be added. If modifying an event, the modifications will be saved.

Cancel

Closes the Edit Event (Advanced) window without adding or modifying any events

Custom Data Fields Displayed for Each Data Type Setting

Data type	Custom data fields and descriptions
<code>EVENT_DATA_ASSET</code>	Badge ID - Card number associated with the asset event.
<code>EVENT_DATA_TYPE_AREAAPB</code>	Area APB ID - Area anti-passback ID.
<code>EVENT_DATA_TYPE_CA</code> (Card Access)	Badge ID - Card number associated with the card event. Issue code - Issue code associated with the card. Bio score - Biometric score for biometric card events.

Custom Data Fields Displayed for Each Data Type Setting

Data type	Custom data fields and descriptions
EVENT_DATA_TYPE_CNA (Card No Access)	Badge ID - Card number associated with the event.
EVENT_DATA_TYPE_FC (Facility Code)	Facility code - Facility code associated with the event. Issue code - Issue code.
EVENT_DATA_TYPE_INTERCOM	Intercom data - Special intercom data associated with the event. Line number - Line number used by special intercom events.
EVENT_DATA_TYPE_INTRUSION	Area ID - Area ID for the intrusion event. User ID - User ID associated with the intrusion event.
EVENT_DATA_TYPE_RECEIVER	Receiver ID - ID of the receiver. Line number - Line number on the receiver. Area ID - Area ID for the event. User ID - User ID associated with the event. Event Code - Event code for the event. The Event Code depends on the selection made from the Event Code Template drop-down list. For example, if SIA is selected from the Event Code Template drop-down list, enter "BA" in the Event Code field for a Burglary Alarm event.
EVENT_DATA_TYPE_STATUS	New status - New status, which is dependent on the type of message. Old status - Old status, which is dependent on type of message. Comm status - Communication status, which is dependent on the type of message. If your event really does not have additional data, you can use the EVENT_DATA_TYPE_STATUS.
EVENT_DATA_TYPE_STATUSREQUEST	Status type - Type of status request. OnGuard has a number of pre-defined types. Status - Status associated with the status type. These values depend on the type of status.
EVENT_DATA_TYPE_TRANSMITTER	Transmitter ID - Transmitter ID associated with the transmitter event
EVENT_DATA_TYPE_VIDEO	Channel - Channel number associated with the video event

Event Generator Menus

File

Save Events

Saves the event list as a file with an EVT extension. This is generally done after the event configuration has been completed.

Load Events

Enables you to load a previously saved event configuration.

Edit

Create Event

Contains a sub-menu of options that are used to create events.

- **Create Event (Advanced):** Enables you to create an event using additional advanced parameters that are not available in the simple mode.
- **Create Event (Simple):** Enables you to create an event using the least number of parameters possible.

Modify Event

Contains a sub-menu of options that are used to modify events.

- **Modify Event (Advanced):** For a selected event, displays the basic parameters and enables you to change them.
- **Modify Event (Simple):** For a selected event, displays advanced parameters and enables you to change them.

Delete Event

Used to delete a selected event. A confirmation message is displayed before the actual deletion occurs.

Clear Events

Clears all events listed in the main window. Make sure to save the events before executing this command if you wish to use the events in the future; otherwise, you will need to recreate them.

Send Event

This option in the Edit menu performs the same function as **Send Event**. For more information, refer to [Send Event](#) on page 306.

Generate Events

This option in the Edit menu performs the same function as **Generate Events**. For more information, refer to [Generate Events](#) on page 307.

Send Event

Generates a single selected event, which is then sent to Alarm Monitoring.

Generate Events

Generates multiple events according to the configured frequency settings, and sends them to Alarm Monitoring.

Required Event Generator Files

To use the Event Generator, you will need the following files:

- **EventGeneratorSetupTool.exe**
- **LnlEventGeneratoru.dll**
- (Optional) **EventGenerator.chm**

These files are copied to the <**Windows Configured Program Files Location**>\OnGuard Software Development Kit directory when the SDK software is installed. Typically, this directory is **C:\Program Files (x86)\OnGuard Software Development Kit\EventGenerator**.

You will need to manually copy the files listed above to the OnGuard installation directory, which is typically **C:\Program Files (x86)\OnGuard**. Although the **EventGenerator.chm** file is not required for the Event Generator to run, we recommend that you copy this as well, since this contains the online help for the Event Generator application. All of these files are also located on the OnGuard SDK media image in the **program files (x86)\OnGuard Software Development Kit\Event Generator** directory.

You must also manually register the **LnlEventGeneratoru.dll**. For more information, refer to [Registering the LnlEventGeneratoru.dll](#) on page 308.

Setting Up the Event Generator

1. Install the OnGuard SDK software.
2. Copy the **EventGeneratorSetupTool.exe**, **LnlEventGeneratoru.dll**, **EventGenerator.chm** files from the Software Development Kit to your hard drive.

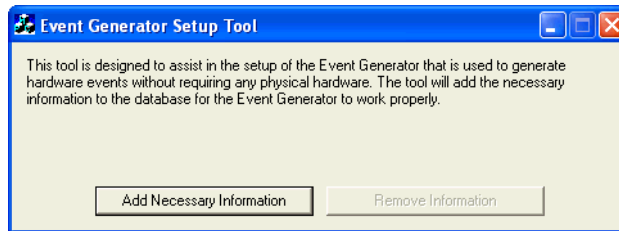
Copy from **C:\Program Files (x86)\OnGuard Software Development Kit\EventGenerator** directory to **C:\Program Files (x86)\OnGuard** directory

Note: If you receive an information message stating that the **LnlEventGeneratoru.dll** already exists in the **C:\Program Files (x86)\OnGuard** directory, replace the file.

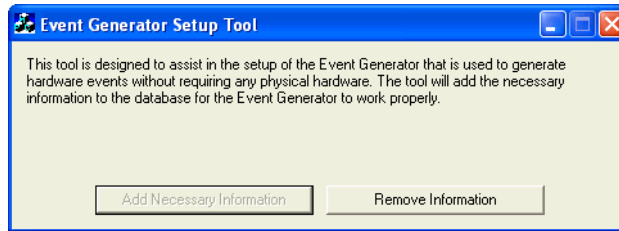
3. Register the **LnlEventGeneratoru.dll**. For more information, refer to [Registering the LnlEventGeneratoru.dll](#) on page 308.
4. In the OnGuard software, add hardware such as access panels, readers, and so on. Keep in mind this hardware does not have to be “online”; it might even be hardware that doesn’t really exist.
5. Run the Event Generator Setup Tool. To do this, navigate to the **EventGeneratorSetupTool.exe** file in your OnGuard installation directory (**C:\Program Files (x86)\OnGuard**) and double-click it.

Note: If you receive an error saying that the **LnlFCDBu.dll** file could not be found in the specified path, register the **LnlEventGeneratoru.dll**. For more information, refer to [Registering the LnlEventGeneratoru.dll](#) on page 308.

6. Click [Add Necessary Information].



7. The [Add Necessary Information] button will then become grayed out. At this point, you can close the Event Generator Setup Tool.

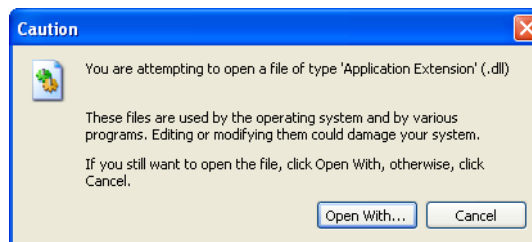


8. Run the Communication Server as an application. To do this:
 - a. Open the Communication Server.
For more information, refer to “Using OnGuard in the Supported Operating Systems” in the Installation Guide.
 - b. Right-click on the  icon in the system tray, and then select **Open Communication Server**. The Communication Server will open in one window, and the Event Generator will open in another window.

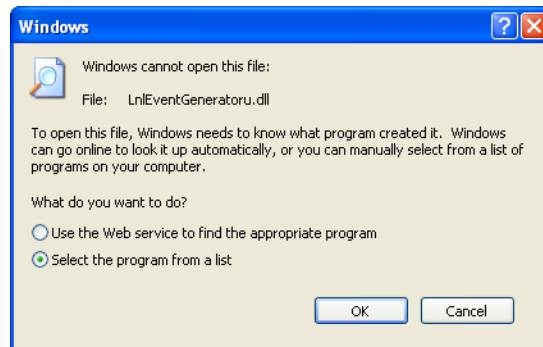
Registering the LnlEventGeneratoru.dll

One way to register the **LnlEventGeneratoru.dll** file is the following:

1. Navigate to the **LnlEventGeneratoru.dll** file in the OnGuard installation directory.
2. Right-click on the file, select **Open With > Choose Program**.
3. A warning message displays, indicating the potential danger of opening dll files. Click [OK].



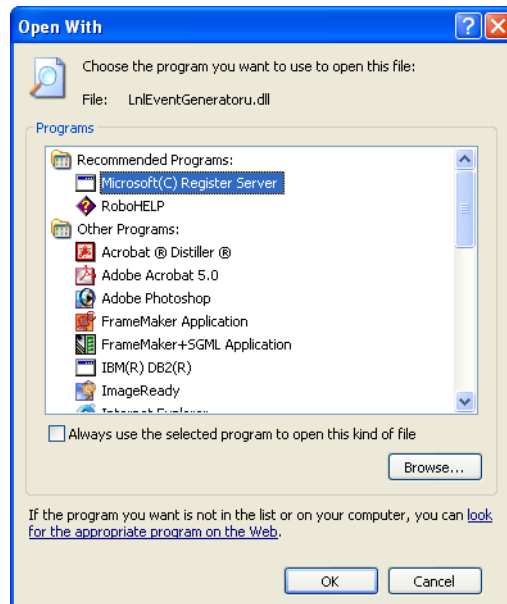
4. Click [Open With...].
5. Select the **Select the program from list** radio button, then click [OK].



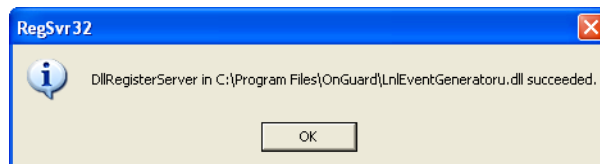
6. The Open With window opens. Click [Browse...], navigate to **C:\Windows\system32**, and then double-click on the **regsvr32.exe** file.

Note: Run the **regsvr32.exe** file as an administrator. Otherwise, an error message will appear.

7. In the Open With window, Microsoft Register Server will now be highlighted. Click [OK].



The following message is displayed, indicating that the file was successfully registered:



8. The **LnIEventGeneratoru.dll** file is now registered. If you were setting up Event Generator, return to [Setting Up the Event Generator](#) on page 307.

Adding an Event to the Event Generator

A Simple user interface and an Advanced user interface are available for adding events to the Event Generator. Only non-receiver/intrusion events are available in the Simple user interface; both non-receiver/intrusion events and receiver/intrusion events are available in the Advanced user interface.

Adding an Event Using the Simple User Interface

To add a new event to be generated using the Simple user interface:

1. From the **Edit** menu in the Event Generator main window, select **Create Event > Create Event (Simple)**.
2. When the Edit Event (Simple) window appears, select the desired **Event type**. Depending on your selection, the other drop-down lists will be enabled/disabled accordingly.
3. Once you've filled in all necessary items, click [OK].
4. Repeat these steps for all the events you wish to create.

Adding an Event Using the Advanced User Interface

To add a new event to be generated using the Advanced user interface:

1. From the **Edit** menu in the Event Generator main window, select **Create Event > Create Event (Advanced)**.
2. When the Edit Event (Simple) window appears, select the desired **Event type**. Depending on your selection, the other drop-down lists will be enabled/disabled accordingly.
3. Once you've filled in all necessary items, click [OK].
4. Repeat these steps for all the events you wish to create.

Generating Events

Events are generated differently depending on whether you are generating a single event or multiple events.

Generating a Single Event

Select the event you wish to generate from the list of events and then select **Edit > Send Event**. You should see that event in Alarm Monitoring.

Generating Multiple Events

1. In the Event Generator main window, enter a value in the **Number of times** field. This will be the number of times each event in the list is generated.
2. Either fill in the **End delay and In between delay** fields with new values, stay with defaults, or select to use a random time for one or both using the check boxes.
3. You can also select to use random cardholders along with these events, by clicking the **Random badge IDs** check box. To save time you can click [Auto-populate with min and max badge IDs], and then the fields will be automatically filled with the proper numbers from your database.
4. Click **Edit > Generate Events**.

Saving an Event List

After you have completed your event configuration, you can save the event list by doing the following:

1. From the **File** menu, select **Save Events**.
2. Navigate to the location where you wish to save the event list, enter a file name, and then click [Save]. The event list will be saved in a file with the extension EVT.

Loading an Event List

To load a previously saved list:

1. From the **File** menu, select **Load Events**.
2. Navigate to the event list that you wish to load, select the EVT file, and then click [Open].

Closing the Event Generator

To close the Event Generator, simply exit the Communication Server. After a short delay, the Event Generator window will close as well. You cannot close the Event Generator manually while the Communication Server is running; if you attempt to do so, the following error message will be displayed:



Additional Copyright and Licensing Information

This appendix provides copyright and licensing information for libraries, encoding algorithms, templates, and so on used by the LS OpenAccess web service and the REST API.

Entity Framework

Apache License

Version 2.0, January 2004

<http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License.

Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License.

Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution.

You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

- 1) You must give any other recipients of the Work or Derivative Works a copy of this License; and
- 2) You must cause any modified files to carry prominent notices stating that You changed the files; and
- 3) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
- 4) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices

contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions.

Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks.

This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty.

Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability.

In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability.

While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

LinqToQuery

Copyright (c) 2013 Peter Smith

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Antlr

ANTLR 4 License, viewable at www.antlr.org/license.html

Copyright (c) 2012 Terence Parr and Sam Harwell. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the author nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Newtonsoft.Json

The MIT License (MIT)

Copyright (c) 2007 James Newton-King

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

SignalR

Copyright © Microsoft Open Technologies, Inc. All rights reserved. Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Index

A

Access Denied events.....	166
Access Granted events.....	165
access level	127
Add	
Event to the Event	
Generator.....	310
Logical Device.....	284
Logical Source.....	282
Logical Sub-Device	286
Alarms	
sending.....	277
Test Event	278
Architecture	
OpenAccess.....	26
Area Control events.....	167
Asset events	167
Association classes.....	271
Lnl_AccessLevelGroupAssignment ..	271
Lnl_BadgeOwner	271
Lnl_CardholderAccount	271
Lnl_CardholderBadge	272
Lnl_CardholderMultimediaObject.....	272
Lnl_DirectoryAccount	272
Lnl_MultimediaObjectOwner.....	273
Lnl_PersonAccount	273
Lnl_ReaderEntersArea.....	273
Lnl_ReaderExitsArea	274
Lnl_SegmentGroupMember.....	274
Lnl_VisitorAccount	274
Lnl_VisitorBadge	275
Lnl_VisitorMultimediaObject.....	275
Lnl_VisitSelfServiceStation	266
Authorization.....	32

B

Badges	41
Biometric events	168
brute force attack	49

C

Caching user credentials.....	17, 32
Cardholders.....	41, 123
Class definition	25
Classes	
association	271
data	181
Client definition	25
Closing the Event Generator.....	311
Command and control classes and	
methods	
Lnl_AlarmOutput.....	193
Lnl_AlarmPanel	194
Lnl_Input	217
Lnl_IntrusionArea.....	218
Lnl_IntrusionDoor	219
Lnl_IntrusionOutput	221
Lnl_IntrusionZone	221
Lnl_OffBoardRelay	231
Lnl_OnBoardRelay.....	232
Lnl_Output	233
Lnl_ReaderInput.....	244
Lnl_ReaderInput1	245
Lnl_ReaderInput2	246
Lnl_ReaderOutput.....	247
Lnl_ReaderOutput1	248
Lnl_ReaderOutput2	249
Common event properties	162, 174
Confirm installed version of OnGuard.....	16
ConnectionHeartbeat.....	160
Controller-based events	164

CORS	48	Lnl_PrecisionAccessGroup	
CreateSubscription	155	Assignment	238
Cross-Origin Resource Sharing	48	Lnl_ProhibitedPassword	238
Custom configuration		Lnl_PTZPreset	239
authenticated token inactivity		Lnl_Reader	239
timeout	17	Lnl_Segment	252
authenticated token timeout	17	Lnl_SegmentGroup	252
badge printing deletion		Lnl_SegmentUnit	253
properties	22	Lnl_Timezone	253
brute force attack protection	17	Lnl_TimezoneInterval	253
caching properties	20	Lnl_User	254
internal lockout properties	19	Lnl_UserAccount	256
issue mobile badges	17	Lnl_UserFieldPermissionGroup	257
openaccess.ini	19	Lnl_UserPermissionDeviceGroup	
		Link	258
D		Lnl_UserPermissionGroup	257
Data classes	181	Lnl_UserReportPermissionGroup	258
Lnl_AccessGroup	181	Lnl_UserSecondarySegment	259
Lnl_AccessLevel	182	Lnl_VideoLayoutSource	260
Lnl_AccessLevelAssignment	183	Lnl_VideoRecorder	260
Lnl_AccessLevelManaged	184	Lnl_VideoTemplate	260
Lnl_AccessLevelReaderAssignment	185	Lnl_Visit	261
Lnl_Account	188	Lnl_VisitDelegateAssignment	265
Lnl_AlarmAckHistory	189	Lnl_VisitEmailRecipient	262
Lnl_AlarmDefinition	190	Lnl_Visitor	264
Lnl_AlarmInput	192	Lnl_VisitSignInLocation	266
Lnl_Badge	197	Lnl_Workstation	267
Lnl_BadgeFIPS201	200	Lnl_WorldTimezone	267
Lnl_BadgeLastLocation	201	user-defined value lists	270
Lnl_BadgeStatus	202	Delete	
Lnl_BadgeType	202	Logical Device	285
Lnl_Camera	204	Logical Source	283
Lnl_CameraDeviceLink	205	Logical Sub-Device	286
Lnl_CameraGroup	205	Deploy	
Lnl_CameraGroupCameraLink	206	LS Message Broker Service	29
Lnl_Cardholder	206	Directory accounts	41
Lnl_DeviceGroup	208		
Lnl_Directory	208	E	
Lnl_Element	209	Enabling Verbose Logging	289
Lnl_ElevatorTerminal	209	Event API Reference	155
Lnl_EventAlarmDefinitionLink	210	Event filters	71
Lnl_EventParameter	211	Event Generator	
Lnl_EventSubtypeDefinition	211	add an event to the	
Lnl_EventSubtypeParameterLink	212	Event Generator	310
Lnl_EventType	212, 213	closing	311
Lnl_HolidayType	214	generating a single event	310
Lnl_HolidayTypeLink	214	generating events	310
Lnl_IncomingEvent	215	generating multiple events	310
Lnl_LoggedEvent	222	main window	299
Lnl_LogicalSource	225	menus	306
Lnl_MonitoringZone	226	saving an event list	311
Lnl_MonitoringZoneCameraLink	227	setting up	307
Lnl_MonitoringZoneDeviceLink	227	Event queues	26
Lnl_MonitorZoneRecorderLink	228	Event subscriptions, See Subscriptions	
Lnl_MultimediaObject	229	Events	
Lnl_Panel	234	Access Denied	166
Lnl_Person	236	Access Granted	165
Lnl_PersonSecondarySegments	237	add an event to the	
Lnl_PrecisionAccessGroup	237, 251	Event Generator	310

Alarm Acknowledgment Activity	173	Lnl_BadgeStatus	202
Area Control	167	Lnl_BadgeType	202
Asset	167	Lnl_Camera	204
Biometric	168	Lnl_CameraDeviceLink	205
common properties	162, 174	Lnl_CameraGroup	205
controller-based event properties	164	Lnl_CameraGroupCameraLink	206
generating	310	Lnl_Cardholder	206
generating multiple	310	Lnl_CardholderAccount	271
generating single	310	Lnl_CardholderBadge	272
hardware	162	Lnl_CardholderMultimediaObject	272
Intercom	168	Lnl_DeviceGroup	208
Intrusion	169	Lnl_Directory	208
loading an event list	311	Lnl_DirectoryAccount	272
saving an event list	311	Lnl_Element	209
software	174	Lnl_ElevatorTerminal	209
status	169	Lnl_EventAlarmDefinitionLink	210
Transmitter	169	Lnl_EventParameter	211
transmitter	169	Lnl_EventSubtypeDefinition	211
Video	169	Lnl_EventSubtypeParameterLink	212
		Lnl_EventType	212
G		Lnl_GuardTour	213
Generating a single event	310	Lnl_Holiday	213
Generating Access Granted and Access		Lnl_HolidayType	214
Denied events	217	Lnl_HolidayTypeLink	214
Generating events	310	Lnl_IncomingEvent	215
Generating multiple events	310	Lnl_Input	217
Getting started	29	Lnl_IntrusionArea	218
		Lnl_IntrusionDoor	219
H		Lnl_IntrusionOutput	221
Hardware events	162	Lnl_IntrusionZone	221
		Lnl_LoggedEvent	222
I		Lnl_LogicalDevice	225
Intercom events	168	Lnl_LogicalSource	225
Intrusion events	169	Lnl_LogicalSubDevice	226
		Lnl_MonitoringZone	226
J		Lnl_MonitoringZoneCameraLink	227
JSON	25	Lnl_MonitoringZoneDeviceLink	227
		Lnl_MonitoringZoneRecordLink	228
L		Lnl_MultimediaObject	229
Lnl_AccessGroup	181	Lnl_MultimediaObjectOwner	273
Lnl_AccessLevel	182	Lnl_OffBoardRelay	231
Lnl_AccessLevelAssignment	183	Lnl_OnBoardRelay	232
Lnl_AccessLevelGroupAssignment	271	Lnl_Output	233
Lnl_AccessLevelReaderAssignment	185	Lnl_Panel	234
Lnl_AccessLevelRequest	187	Lnl_Person	236
Lnl_AccessRequest	185	Lnl_PersonAccount	273
Lnl_Account	188	Lnl_PersonSecondarySegments	237
Lnl_AlarmAckHistory	189	Lnl_PrecisionAccessGroup	237
Lnl_AlarmDefinition	189	Lnl_PrecisionAccessGroupAssignment	238
Lnl_AlarmInput	192	Lnl_ProhibitedPassword	238
Lnl_AlarmOutput	193	Lnl_PTZPreset	239
Lnl_AlarmPanel	194	Lnl_Reader	239
Lnl_Area	195	Lnl_ReaderEntersArea	273
Lnl_AuthenticationMode	196	Lnl_ReaderExitsArea	274
Lnl_Badge	197	Lnl_ReaderInput	244
Lnl_BadgeFIPS201	200	Lnl_ReaderInput1	245
Lnl_BadgeLastLocation	201	Lnl_ReaderInput2	246
Lnl_BadgeOwner	271	Lnl_ReaderOutput	247
		Lnl_ReaderOutput1	248

Lnl_ReaderOutput2	249	delete event_subscriptions with	
Lnl_ReaderRequest	250	id	72
Lnl_RequestableReader	251	delete instance	91
Lnl_Segment	252	delete queue/{id}	58
Lnl_SegmentGroup	252	delete user managed_access_levels	113
Lnl_SegmentGroupMember	274	delete user preferences	122
Lnl_SegmentUnit	253	delete user segments	119
Lnl_Timezone	253	execute_method	92
Lnl_TimezoneInterval	253	get auth_data	106
Lnl_User	254	get authorized warning settings	130
Lnl_UserAccount	256	get badge print_request	82
Lnl_UserFieldPermissionGroup	257	get cardholder	132
Lnl_UserPermissionDeviceGroupLink	258	get cardholder_from_directory	123
Lnl_UserPermissionGroup	257	get cardholders	93
Lnl_UserReportPermissionGroup	258	get console layout	129
Lnl_UserSecondarySegment	259	get count	79
Lnl_VideoLayout	259	get credentials	107
Lnl_VideoRecorder	260	get directories	60
Lnl_VideoTemplate	260	get directory_accounts	123
Lnl_Visit	261	get directory_accounts_matching_	
Lnl_VisitDelegateAssignment	265	cardholders	124
Lnl_VisitEmailRecipient	262	get editable_segments	116
Lnl_Visitor	263	get enterprise	133
Lnl_VisitorAccount	274	get event_subscriptions	65
Lnl_VisitorBadge	275	get event_subscriptions with id	68
Lnl_VisitorMultimediaObject	266, 275	get feature_availability	56
Lnl_VisitSignInLocation	266	get identity_provider_url	64
Lnl_Workstation	267	get instance	80
Lnl_WorldTimezone	267	get keepalive	56
LnlEventGeneratoru.dll		get logged_events	73
location	307	get logged_in_user	110
registering	307	get managers_of_access_level	115
Loading an event list	311	get map devices	144
Logical Sources		get map icon groups	148
licenses required	280	get map icons	152
user permissions required	280	get maps	141
LS Message Broker service		get password policy	134
deploying	29	get queue	57
LS OpenAccess Service		get queue/{id}	57
overview	15	get segmentation	139
using the API	39	get session	64
M		get susp_expiration	59
Menus for Event Generator	306	get type	77
Message Broker		get types	76
See Also LS Message		get user	113
Broker service		get user managed_access_levels	111
Method		get user preferences	119
add authentication	61	get user segments	117
add badge print_request	83	get version	56
add event_subscriptions	69	get video settings	141
add instance	88	get video_recorders	104
add partner_values	58	get visit settings	140
add user managed_access_levels	112	get visitors	99
add user segments	118	modify event_subscriptions	71
bulk modify instance property	90	modify instance	89
delete authentication	63	modify partner_values	59
delete badge print_request	85	modify user	114
delete console cards with id	128	post console cards	127
		post send_incoming_events	109

post user preferences	121	SignalR	155
put access_level	108, 127	Software events	174
put console layout	129	SSL	15, 30
put password policy	136	StartManaging	159
put update_cardholder_with_		Status events	169
directory_account_ property	126	StopManaging	159
put user password	115	StopSubscription	159
put user preferences	120	Subscriptions	71
Modify		event filters	71
Logical Device	284	event queues	26
Logical Source	282	overview	26
Logical Sub-Device	286	using event filters	71
ModifySubscription	157	Swagger specification and	
Multimedia objects	42	documentation	38
O		T	
Object/instance definition	25	Test Event From alarm	278
OnBusinessEventReceived	160	Transmitter events	169
OnConnectionFromMessageBusLost	161	Troubleshooting	289
OnConnectionToMessageBusEstablished ...	161	U	
OnExceptionRaised	161	User-defined list values	42
OnGuard		User-defined value lists	270
confirm installed version	16	V	
OnManagementEvent	161	Verbose Logging	
OpenAccess		Enabling	289
custom configuration	19	version	49
user credential caching	17, 32	Video events	169
OpenAccess Architecture	26	Visitors	41
OpenAccess Tool		Visits	41
starting	292	W	
using	292	Web Event Bridge	155
openaccess.ini			
custom configuration	19		
P			
Person definition	25		
PIN code	41		
properties	162, 174		
R			
Reference	181		
Registering the LnlEventGeneratoru.dll	307		
Response headers	38		
REST API Reference	53		
S			
Sample applications	34		
sample C# applications	35		
sample Java application	37		
sample web applications	34		
Sample code			
retrieve error information	277		
Saving an event list	311		
SDK definition	25		
Secure Socket Layer	15, 30		
Security identifier	41		
Sending alarms to OnGuard	277		
Setting up the Event Generator	307		



1212 Pittsford-Victor Road
Pittsford, New York 14534 USA
Tel 866.788.5095 Fax 585.248.9185
www.LenelS2.com

